

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



Đặng Thị Thanh Trúc

**PHƯƠNG PHÁP XÂY DỰNG PHẦN MỀM TỰ
ĐỘNG DỰA TRÊN ĐẶC TẢ CA SỬ DỤNG ỨNG
DỤNG VIBE CODING**

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY
Ngành: Công nghệ thông tin

HÀ NỘI - 2025

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



Đặng Thị Thanh Trúc

**PHƯƠNG PHÁP XÂY DỰNG PHẦN MỀM TỰ
ĐỘNG DỰA TRÊN ĐẶC TẢ CA SỬ DỤNG ỨNG
DỤNG VIBE CODING**

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY
Ngành: Công nghệ thông tin

Cán bộ hướng dẫn: TS. Trần Hoàng Việt

HÀ NỘI - 2025

**VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY**



Dang Thi Thanh Truc

**METHOD FOR AUTOMATED SOFTWARE
CONSTRUCTION BASED ON USE CASE
SPECIFICATION USING VIBE CODING**

**BACHELOR'S THESIS
Major: Information Technology**

Supervisor: Ph.D Tran Hoang Viet

Hanoi - 2025

LỜI CAM ĐOAN

Tôi tên là Đặng Thị Thanh Trúc, sinh viên lớp QH-2021-I/CQ-I-IT2, ngành Công nghệ thông tin hệ chuẩn. Tôi xin cam đoan: Khóa luận tốt nghiệp với đề tài “Phương pháp xây dựng phần mềm tự động dựa trên đặc tả ca sử dụng ứng dụng Vibe coding” là của tôi, chưa từng được nộp như một khóa luận tốt nghiệp nào tại Trường Đại học Công Nghệ hoặc bất kỳ trường đại học nào khác. Những gì tôi viết ra không có sự sao chép từ các tài liệu, không sử dụng kết quả của người khác mà không trích dẫn cụ thể. Đây là công trình nghiên cứu cá nhân tôi tự phát triển với sự hướng dẫn của TS. Trần Hoàng Việt, không sao chép mã nguồn của người khác. Nếu vi phạm những điều trên, tôi xin chấp nhận tất cả những truy cứu về trách nhiệm theo quy định của Trường Đại học Công Nghệ.

Hà Nội, ngày 29 tháng 11 năm 2025

Sinh viên

Đặng Thị Thanh Trúc

LỜI CẢM ƠN

Lời đầu tiên, tôi xin gửi lời cảm ơn chân thành đến thầy Trần Hoàng Việt. Thầy đã tận tình giảng dạy, truyền đạt những kiến thức quý báu cho tôi trong suốt quá trình học tập vừa qua. Bên cạnh đó, thầy còn trang bị cho tôi phương pháp tư duy và phương pháp nghiên cứu khoa học hiệu quả để hoàn thành tốt khóa luận.

Ngoài ra, tôi xin gửi lời cảm ơn đến các thầy cô, cán bộ, anh chị và bạn bè tại Trường Đại học Công nghệ đã tạo điều kiện thuận lợi cho tôi được học tập, nghiên cứu và thực hiện khóa luận.

Cuối cùng, tôi xin được tỏ lòng biết ơn sâu sắc đến gia đình tôi. Họ đã nuôi dưỡng, dạy dỗ tôi trưởng thành và luôn động viên, ủng hộ tôi trong suốt thời gian học tập vừa qua.

Hà Nội, ngày 29 tháng 11 năm 2025

Sinh viên

Đặng Thị Thanh Trúc

TÓM TẮT

Khóa luận nghiên cứu và đề xuất một phương pháp phát triển phần mềm tự động dựa trên đặc tả ca sử dụng nhằm rút ngắn thời gian xây dựng hệ thống, giảm chi phí và nâng cao tính nhất quán giữa yêu cầu và mã nguồn. Phương pháp được xây dựng theo hai pha: pha thứ nhất chuyển đổi đặc tả ca sử dụng, tài liệu thiết kế và mẫu cấu trúc dự án thành một đặc tả kỹ thuật dạng “mẫu lệnh lập trình”; pha thứ hai sử dụng đặc tả này kết hợp với bộ mã nguồn nền để tự động sinh ra các tệp mã hoàn chỉnh. Đồng thời, hệ thống có khả năng nhận diện và xử lý một số ngoại lệ thường gặp như lỗi định tuyến, thiếu cấu hình hay không khớp tham số, từ đó giảm thiểu mức độ can thiệp thủ công. Để đánh giá hiệu quả của phương pháp, nghiên cứu đã tiến hành thực nghiệm trên một hệ thống gồm nhiều ca sử dụng với luồng nghiệp vụ rõ ràng, xây dựng đặc tả chi tiết cho từng ca sử dụng và đối chiếu kết quả sinh mã với yêu cầu ban đầu. Các tiêu chí đánh giá tập trung vào số lần cần chỉnh sửa, độ khớp giữa mã sinh ra và đặc tả nghiệp vụ, cũng như tính đầy đủ của các thành phần được tạo tự động. Kết quả thực nghiệm cho thấy phương pháp có khả năng sinh mã nguồn có cấu trúc tốt, đáp ứng đúng yêu cầu và giảm đáng kể công việc lặp lại của lập trình viên. Qua đó, nghiên cứu khẳng định tính khả thi của hướng tiếp cận phát triển phần mềm tự động dựa trên đặc tả ca sử dụng, đồng thời mở ra triển vọng ứng dụng trong thực tiễn nhằm tối ưu hóa quy trình phát triển phần mềm và nâng cao năng suất đội ngũ kỹ thuật.

Từ khóa: phát triển phần mềm tự động, đặc tả ca sử dụng, tối ưu hóa quy trình phát triển phần mềm, sinh mã nguồn tự động, hệ thống hỗ trợ lập trình.

ABSTRACT

This thesis presents a method for automated software development based on use case specifications, aiming to shorten system development time, reduce costs, and improve consistency between requirements and source code. The proposed method consists of two phases: the first phase transforms use case specifications, design documents, and project templates into a technical specification in the form of a “coding prompt”; the second phase uses this specification along with a base code repository to automatically generate complete source code files. The system can also detect and handle common exceptions such as routing errors, missing configurations, or mismatched parameters, minimizing manual intervention. To evaluate the method, experiments were conducted on a system with multiple use cases and well-defined workflows, generating coding prompts for each case and comparing the resulting code with the original requirements. Evaluation criteria focused on the number of manual corrections, the alignment between generated code and business specifications, and the completeness of automatically created components. Experimental results show that the method can produce well-structured code that meets requirements and significantly reduces repetitive tasks for developers. These results demonstrate the feasibility of automated software development based on use case specifications and highlight its potential for practical application in optimizing software development processes and enhancing engineering productivity.

Keywords: *automated software development, automated source code generation, special use case, system support installer.*

Mục lục

Lời cam đoan	i
Lời cảm ơn	ii
Tóm tắt	iii
Abstract	iv
Thuật ngữ	xii
Chương 1 Giới thiệu	1
1.1 Đặt vấn đề	1
1.2 Mục tiêu của khoá luận	2
1.3 Cấu trúc của khoá luận	2
Chương 2 Kiến thức nền tảng	4
2.1 Định nghĩa	4
2.2 Công cụ và nền tảng hỗ trợ	4
2.2.1 Cursor	4
2.2.2 Google Gemini	5
Chương 3 Phương pháp nghiên cứu	7
3.1 Tổng quan phương pháp	7
3.2 Pha 1 – Sinh Coding Prompt	9
3.2.1 Đầu vào của pha 1	9

3.2.2	Quy trình xử lý	14
3.2.3	Đầu ra của pha 1	17
3.3	Pha 2 – Sinh mã nguồn	18
3.3.1	Đầu vào pha 2	18
3.3.2	Quy trình sinh mã	22
3.3.3	Đầu ra của Pha 2	24
3.4	Xử lý ngoại lệ	24
3.4.1	Lỗi liên quan đến giao tiếp mạng và bảo mật (CORS)	25
3.4.2	Lỗi cấu hình và định tuyến API	25
3.4.3	Lỗi mô hình dữ liệu	26
3.4.4	Lỗi xung đột mã nguồn và cấu trúc	26
3.4.5	Lỗi sai lệch giữa giao diện và logic	26
Chương 4	Thực nghiệm	28
4.1	Mô tả hệ thống	28
4.1.1	Danh sách ca sử dụng	29
4.1.2	Mô tả mã nguồn nền tảng ban đầu	30
4.2	Đặc tả ca sử dụng chi tiết	37
4.2.1	Đăng nhập	37
4.2.2	Đăng kí	41
4.2.3	Xem danh sách giao dịch	46
4.2.4	Tạo giao dịch mới	52
4.2.5	Xem danh sách tài khoản ngân hàng	58
4.3	Phần mềm được sinh ra	63
4.3.1	Giao diện phần mềm	63
4.3.2	Mã nguồn và các file sinh tự động	73

4.4	Đánh giá kết quả	80
4.4.1	Bảng thống kê các lần chạy	81
4.4.2	Đánh giá hiệu quả	82
Chương 5	Kết luận	84
5.1	Tóm tắt kết quả đạt được	84
5.2	Hạn chế và thách thức của Vibe Coding	84
5.3	Định hướng phát triển trong tương lai	85
	Tài liệu tham khảo	86
	Phụ lục: Đặc tả ca sử dụng chi tiết	88

Danh sách hình vẽ

3.1	Sơ đồ tổng quan phương pháp xây dựng phần mềm tự động.	7
3.2	Quy trình Pha 1 để sinh Coding Prompt.	15
3.3	Giao diện công cụ gọi API Gemini.	18
3.4	Các bước xử lý ở Pha 2.	23
4.1	Sơ đồ kiến trúc tổng thể của hệ thống.	31
4.2	Cấu trúc thư mục Backend của dự án.	32
4.3	Cấu trúc thư mục Frontend của dự án.	34
4.4	Giao diện màn hình đăng nhập.	64
4.5	Giao diện màn hình đăng ký tài khoản.	64
4.6	Giao diện màn hình xem danh sách tài khoản.	65
4.7	Giao diện màn hình xóa tài khoản.	65
4.8	Giao diện màn hình xem chi tiết tài khoản.	66
4.9	Giao diện màn hình chỉnh sửa thông tin tài khoản.	66
4.10	Giao diện màn hình xem danh sách giao dịch.	67
4.11	Giao diện màn hình thêm giao dịch mới.	68
4.12	Giao diện màn hình tổng quan chi tiêu.	69
4.13	Giao diện màn hình xem mục tiêu hàng tháng.	70
4.14	Giao diện màn hình thêm mục tiêu.	71
4.15	Giao diện màn hình xem danh sách hóa đơn.	72

4.16	Giao diện màn hình chỉnh sửa mục tiêu.	72
4.17	Mô tả màn hình mã nguồn sinh tự động trong thư mục Backend.	76
4.18	Mô tả mã nguồn sinh tự động trong thư mục Frontend.	80

Danh sách bảng

3.1	Phân tích các thành phần cốt lõi của đặc tả ca sử dụng.	10
4.1	Đặc tả ca sử dụng Đăng nhập hệ thống	38
4.2	Đặc tả ca sử dụng Đăng ký tài khoản	42
4.3	Đặc tả ca sử dụng Xem danh sách giao dịch	46
4.4	Đặc tả ca sử dụng Tạo giao dịch mới	52
4.5	Đặc tả ca sử dụng Xem danh sách tài khoản ngân hàng	58
4.6	Danh sách các thư mục và tệp mới được tạo trong Backend	73
4.7	Danh sách các thư mục và tệp mới được tạo trong Frontend	76
4.8	Thống kê thời gian thực thi cho từng ca sử dụng	81
1.1	Đặc tả ca sử dụng Thêm tài khoản ngân hàng mới	88
1.2	Đặc tả ca sử dụng Chỉnh sửa thông tin tài khoản	93
1.3	Đặc tả ca sử dụng Xoá thông tin tài khoản	98
1.4	Đặc tả ca sử dụng Xem chi tiết thông tin tài khoản	102
1.5	Đặc tả ca sử dụng Xem chi tiêu hàng tháng	107
1.6	Đặc tả ca sử dụng Xem chi tiết chi tiêu theo danh mục	111
1.7	Đặc tả ca sử dụng Xem danh sách hoá đơn sắp tới	116
1.8	Đặc tả ca sử dụng Xem danh sách Mục tiêu	120
1.9	Đặc tả ca sử dụng Tạo Mục tiêu mới	124

1.10	Đặc tả ca sử dụng Điều chỉnh Mục tiêu hàng tháng	129
1.11	Đặc tả ca sử dụng Xem Biểu đồ Tổng kết Tiết kiệm	134

Thuật ngữ

Từ viết tắt	Từ đầy đủ	Ý nghĩa
HTTP	HyperText Transfer Protocol	Giao thức truyền tải dữ liệu giữa client và server trên web
API	Application Programming Interface	Giao diện lập trình cho phép các hệ thống hoặc module giao tiếp với nhau
CRUD	Create – Read – Update – Delete	Các thao tác cơ bản trên dữ liệu trong hệ thống phần mềm
DOM	Document Object Model	Mô hình biểu diễn cấu trúc của tài liệu HTML/XML dưới dạng cây
IDE	Integrated Development Environment	Môi trường tích hợp hỗ trợ lập trình và phát triển phần mềm
AI	Artificial Intelligence	Trí tuệ nhân tạo
LLM	Large Language Model	Mô hình ngôn ngữ lớn dùng trong xử lý ngôn ngữ tự nhiên
UI	User Interface	Giao diện người dùng
ORM	Object–Relational Mapping	Cơ chế ánh xạ giữa đối tượng trong code và bảng trong cơ sở dữ liệu
SDK	Software Development Kit	Bộ công cụ hỗ trợ lập trình và tích hợp tính năng
JSON	JavaScript Object Notation	Định dạng dữ liệu nhẹ, dùng phổ biến trong API và truyền tải dữ liệu
SQL	Structured Query Language	Ngôn ngữ truy vấn dùng cho hệ quản trị cơ sở dữ liệu quan hệ
JWT	JSON Web Token	Cơ chế xác thực dựa trên token mã hóa
OOP	Object-Oriented Programming	Lập trình hướng đối tượng
CSS	Cascading Style Sheets	Ngôn ngữ mô tả giao diện và định dạng trang web
CORS	Cross-Origin Resource Sharing	Cơ chế cho phép chia sẻ tài nguyên giữa các domain khác nhau
DTO	Data Transfer Object	Đối tượng dùng để truyền dữ liệu giữa các tầng trong hệ thống
CSDL	Cơ sở dữ liệu	Hệ thống lưu trữ và quản lý dữ liệu
MCP	Model Context Protocol	Giao thức quản lý ngữ cảnh dùng cho mô hình AI

Chương 1

Giới thiệu

1.1 Đặt vấn đề

Trong bối cảnh nhu cầu phát triển phần mềm ngày càng tăng cao, các doanh nghiệp và nhóm kỹ thuật đều hướng đến mục tiêu rút ngắn thời gian xây dựng sản phẩm, giảm chi phí và bảo đảm chất lượng. Tuy nhiên, quá trình phát triển phần mềm truyền thống vẫn đòi hỏi nhiều công sức trong việc phân tích yêu cầu, thiết kế kiến trúc, lập trình và kiểm thử. Một trong những thách thức lớn nhất là bảo đảm sự nhất quán giữa đặc tả yêu cầu ban đầu và sản phẩm phần mềm cuối cùng. Điều này dẫn đến nhu cầu nghiên cứu các phương pháp tự động hóa dựa trên trí tuệ nhân tạo nhằm hỗ trợ hoặc thay thế một phần các hoạt động lập trình thủ công.

Trong xu hướng đó, mô hình "phát triển phần mềm tự động" (Automated Software Development) dựa trên đặc tả ca sử dụng đang nhận được sự quan tâm đặc biệt. Việc chuyển đổi trực tiếp từ tài liệu đặc tả thành mã nguồn mở ra khả năng giảm thiểu sai lệch yêu cầu, tăng tốc độ phát triển và nâng cao năng suất của lập trình viên. Nhiều mô hình và công cụ hỗ trợ đã xuất hiện, trong đó có Vibe Coding – một cách tiếp cận mới cho phép mô hình AI phân tích luồng nghiệp vụ và sinh mã tự động thông qua các tương tác tự nhiên.

Đề tài khóa luận này tập trung nghiên cứu phương pháp phát triển phần mềm tự động dựa trên đặc tả ca sử dụng, đồng thời khai thác khả năng của công cụ Vibe Coding kết hợp với môi trường Cursor để kiểm nghiệm tính khả thi của việc tự động hóa quy

trình phát triển. Đầu vào của hệ thống là các đặc tả ca sử dụng chuẩn hóa; từ đó, mô hình sẽ phân tích luồng nghiệp vụ, bóc tách logic và tự động sinh ra cấu trúc mã nguồn ban đầu. Việc nghiên cứu không chỉ đóng góp về mặt lý thuyết khi đề xuất một phương pháp tiếp cận có hệ thống, mà còn có ý nghĩa thực tiễn trong việc hỗ trợ lập trình viên giảm tải công việc lặp lại, nâng cao độ chính xác và tối ưu quy trình phát triển phần mềm.

1.2 Mục tiêu của khóa luận

Khóa luận hướng đến mục tiêu nghiên cứu và đề xuất một phương pháp phát triển phần mềm tự động giúp rút ngắn thời gian xây dựng hệ thống, giảm chi phí và đảm bảo tính nhất quán giữa yêu cầu nghiệp vụ và mã nguồn. Trọng tâm của nghiên cứu là tận dụng đặc tả ca sử dụng như nguồn thông tin đầu vào để tạo ra các thành phần phần mềm một cách tự động, hạn chế tối đa thao tác thủ công và các lỗi phát sinh trong quá trình phát triển. Phương pháp được định hướng theo hướng chuẩn hóa đặc tả, mô tả lại yêu cầu dưới dạng có thể xử lý tự động và kết hợp với bộ mã nguồn nền để sinh ra cấu trúc hệ thống cùng các tệp mã cần thiết. Đồng thời, khóa luận đặt mục tiêu kiểm chứng khả năng áp dụng của phương pháp thông qua thực nghiệm trên một hệ thống mẫu, đánh giá mức độ chính xác của mã sinh ra và khả năng đáp ứng yêu cầu nghiệp vụ. Thông qua việc xây dựng và kiểm chứng phương pháp, khóa luận kỳ vọng góp phần định hướng một cách tiếp cận mới trong tự động hóa phát triển phần mềm, giúp nâng cao hiệu quả và năng suất cho các dự án thực tế.

1.3 Cấu trúc của khóa luận

Khóa luận được tổ chức thành 5 chương với nội dung chính như sau:

Chương 1: Đặt vấn đề trình bày bối cảnh nghiên cứu, các khái niệm nền tảng liên quan, cùng với các công cụ và nền tảng hỗ trợ được sử dụng trong quá trình thực hiện khóa luận. Chương cũng giới thiệu cấu trúc tổng thể của luận văn.

Chương 2: Kiến thức nền tảng giới thiệu về các công nghệ, các công cụ cùng với các thư viện sử dụng để nghiên cứu phương pháp.

Chương 3: Phương pháp nghiên cứu mô tả tổng quan phương pháp phát triển

phần mềm tự động dựa trên đặc tả ca sử dụng, mô tả chi tiết Pha 1 – Sinh coding prompt từ đặc tả ca sử dụng và Pha 2 – Sinh mã nguồn dựa trên chuỗi prompt đã được tạo.

Chương 4: Thực nghiệm trình bày quá trình kiểm chứng phương pháp thông qua hệ thống thử nghiệm và phân tích hiệu quả của phương pháp đề xuất thông qua các thống kê thực nghiệm.

Chương 5: Kết luận tổng kết những kết quả đạt được, nhận định các hạn chế của phương pháp và định hướng phát triển trong tương lai.

Chương 2

Kiến thức nền tảng

2.1 Định nghĩa

Vibe Coding là một phương pháp phát triển phần mềm mới, trong đó lập trình viên hoặc nhà phân tích mô tả yêu cầu hệ thống, logic xử lý hoặc hành vi mong muốn bằng ngôn ngữ tự nhiên. Các mô tả này được mô hình AI hiểu và tự động tạo ra mã nguồn tương ứng [10]. Thay vì tập trung vào cú pháp, cấu trúc hay chi tiết kỹ thuật của từng dòng code, người dùng chỉ cần diễn đạt “ý muốn lập trình” (the vibe), còn AI chịu trách nhiệm chuyển hóa ý tưởng thành hiện thực dưới dạng mã chạy được. Phương pháp này giúp rút ngắn đáng kể thời gian phát triển, giảm lỗi phát sinh do thao tác thủ công, đồng thời tăng tính nhất quán giữa đặc tả và phần mềm triển khai [11]. Vibe Coding mở ra cách tiếp cận mới, nơi con người định hướng ở mức ý tưởng và kiến trúc, còn máy móc đảm nhận phần hiện thực hóa kỹ thuật.

2.2 Công cụ và nền tảng hỗ trợ

2.2.1 Cursor

Cursor là một môi trường phát triển tích hợp (IDE) được xây dựng dựa trên Visual Studio Code nhưng được mở rộng mạnh mẽ nhờ khả năng tích hợp trực tiếp với các mô hình ngôn ngữ lớn (LLM) [1, 2, 3]. Điểm nổi bật của Cursor là Composer, một tính năng

cho phép lập trình viên “trò chuyện” với toàn bộ mã nguồn theo ngữ cảnh dự án. Thông qua Composer, người dùng có thể yêu cầu IDE phân tích cấu trúc hệ thống, sinh mã mới, tối ưu logic, thực hiện refactor đa tệp và tìm–sửa lỗi phức tạp mà không cần thao tác thủ công [4]. Điều này giúp Cursor hoạt động như một trợ lý lập trình toàn cục (global coding assistant), có khả năng hiểu mối liên hệ giữa các lớp, module, service, controller và kiến trúc tổng thể của dự án.

Cursor hỗ trợ sử dụng các mô hình AI hàng đầu như Claude 3.5 Sonnet, Claude 3.7, GPT-4 và các frontier LLM khác. Dù danh sách mô hình không xuất hiện trong nghiên cứu học thuật, thông tin này được xác nhận trong tài liệu kỹ thuật và đánh giá người dùng [1, 3]. Công cụ này còn tích hợp nhiều thao tác nâng cao như tạo file tự động, sinh tài liệu kỹ thuật, kiểm tra tính nhất quán giữa các thành phần, và đề xuất cải thiện kiến trúc mã, hỗ trợ tối ưu cho các dự án quy mô lớn.

Tuy nhiên, để khai thác tối đa khả năng của Cursor, lập trình viên cần có hiểu biết tốt về cấu trúc dự án, quy trình phát triển và cách tổ chức mã. Nhận định này phù hợp với các báo cáo và chia sẻ kinh nghiệm thực tế trong cộng đồng kỹ thuật [4, 3]. Do đó, công cụ này phù hợp nhất với lập trình viên từ mức trung cấp trở lên.

Trong mô hình Vibe Coding, mã nguồn được sinh ra tự động dựa trên đặc tả use case, khiến yêu cầu tích hợp, kiểm tra và đồng bộ mã giữa các giai đoạn trở nên quan trọng. Cursor trở thành công cụ tối ưu để hỗ trợ quy trình này vì Composer có thể hiểu ngữ cảnh toàn dự án, tự động ghép nối mã sinh sẵn vào hệ thống hiện hữu, hỗ trợ refactor và sửa lỗi nhanh chóng [5, 10]. Nhờ đó, Cursor không chỉ là IDE mà còn là công cụ duy trì tính nhất quán và tăng tốc quá trình phát triển phần mềm tự động trong Vibe Coding.

2.2.2 Google Gemini

Google Gemini là dòng mô hình trí tuệ nhân tạo đa phương thức (multimodal AI) tiên tiến do Google phát triển, đại diện cho bước tiến lớn trong lĩnh vực AI tổng hợp [12]. Khác với các mô hình truyền thống chỉ xử lý một dạng dữ liệu, Gemini được thiết kế để hiểu và tạo nội dung từ nhiều loại dữ liệu cùng lúc như văn bản, hình ảnh, âm thanh, video và mã nguồn [13]. Nhờ khả năng này, Gemini có thể không chỉ trả lời câu hỏi hay tóm tắt nội dung mà còn giải quyết các nhiệm vụ sáng tạo, phân tích kỹ thuật, hỗ trợ lập trình và xử lý thông tin phức tạp.

Một trong những điểm mạnh cốt lõi của Gemini là khả năng suy luận theo ngữ cảnh (contextual reasoning), cho phép mô hình hiểu rõ mục đích người dùng và đưa ra phản hồi chính xác. Khả năng này được đề cập trong các bài viết phân tích về Gemini 2.0 và hướng dẫn chuyên sâu [14]. Với kiến trúc tối ưu hóa hiệu suất và khả năng mở rộng, Gemini xử lý khối lượng dữ liệu lớn mà vẫn duy trì độ chính xác cao.

Gemini còn được tích hợp sâu với hệ sinh thái Google như Search, Workspace, Android và Google Cloud, giúp người dùng tận dụng AI vào công việc hằng ngày như tạo tài liệu, phân tích dữ liệu, viết mã hoặc kiểm tra logic phần mềm một cách trực quan và hiệu quả [12]. Vì vậy, Gemini mở ra nhiều ứng dụng tiềm năng trong giáo dục, kinh doanh, nghiên cứu và lập trình.

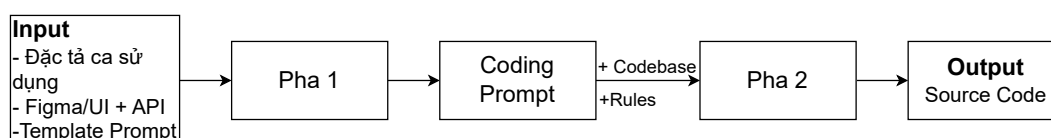
Với sự kết hợp giữa khả năng đa phương thức và sức mạnh suy luận, Gemini trở thành một nền tảng AI toàn diện, đóng vai trò như đối tác thông minh hỗ trợ con người giải quyết bài toán phức tạp và thúc đẩy sự sáng tạo trong nhiều lĩnh vực [13, 14].

Chương 3

Phương pháp nghiên cứu

3.1 Tổng quan phương pháp

Trong nghiên cứu này, phương pháp xây dựng phần mềm bán-tự động được tổ chức theo hai pha chính, tạo thành một chu trình từ việc hiểu và phân tích yêu cầu đến sinh ra mã nguồn hoàn chỉnh. Hai pha này hoạt động liên tục với nhau, mỗi pha đảm nhận một vai trò riêng nhưng gắn kết chặt chẽ để tối ưu hóa quá trình phát triển phần mềm. Tổng quan phương pháp xây dựng phần mềm tự động được mô tả trong Hình 3.1:



Hình 3.1: Sơ đồ mô tả tổng quan phương pháp xây dựng phần mềm tự động dựa vào đặc tả ca sử dụng ứng dụng Vibe Coding.

Pha đầu tiên là Pha 1 – Sinh Coding Prompt, tập trung vào việc phân tích và chuyển hóa dữ liệu đầu vào. Trong pha này, hệ thống nhận các thành phần đầu vào đa dạng nhưng thiết yếu cho việc sinh mã, bao gồm đặc tả ca sử dụng, thiết kế giao diện từ Figma, template chuẩn và hợp đồng API. Những dữ liệu này ban đầu thường rời rạc và thiếu cấu trúc thống nhất, trong khi đó, quá trình sinh mã yêu cầu một định dạng rõ ràng, đầy đủ

thông tin nghiệp vụ cũng như các trạng thái giao diện và luồng tương tác người dùng.

Vai trò của Pha 1 là hợp nhất tất cả các thông tin này thành một tập Coding Prompt được tối ưu cho công cụ sinh mã. Coding Prompt vừa đóng vai trò là bản đặc tả kỹ thuật chuẩn hóa, vừa cung cấp bối cảnh đầy đủ để mô hình ngôn ngữ hiểu và sinh ra mã tương ứng. Mỗi Coding Prompt thường bao gồm: mục tiêu nghiệp vụ cần triển khai, các bước xử lý logic đã phân tích, các ràng buộc dữ liệu và quy tắc nghiệp vụ, cũng như các yêu cầu về giao diện người dùng và cách tương tác với hệ thống. Nhờ Pha 1, các yêu cầu từ nhiều nguồn khác nhau được chuẩn hóa thành một đầu vào duy nhất, giảm thiểu sự mơ hồ và rủi ro lỗi trong quá trình sinh mã.

Pha thứ hai là Pha 2 – Sinh mã nguồn, chịu trách nhiệm hiện thực hóa Coding Prompt thành mã nguồn có thể chạy được. Khác với Pha 1, pha này không chỉ dựa vào văn bản yêu cầu mà còn dựa vào ngữ cảnh của mã nguồn nền tảng hiện tại. Mã nguồn nền tảng, hay Cobebase, đóng vai trò quan trọng để đảm bảo rằng mã sinh ra tuân thủ cấu trúc dự án, quy tắc đặt tên, và nguyên tắc kiến trúc đã thiết lập trước đó. Mã nguồn nền tảng có thể bao gồm các module đã tồn tại, định nghĩa mô hình dữ liệu, các thành phần giao diện cơ bản, các thư mục cấu hình, cũng như các hàm tiện ích và thư viện đã được tích hợp. Khi được cung cấp cùng với Coding Prompt, Cursor hoặc môi trường sinh mã có thể phân tích tổng thể, xác định phần nào cần tạo mới, phần nào cần chỉnh sửa, và đảm bảo các module mới sẽ tích hợp liền mạch với hệ thống hiện hữu.

Quy trình sinh mã trong Pha 2 diễn ra theo các bước logic tuần tự. Trước tiên, công cụ sẽ nạp toàn bộ ngữ cảnh dự án, bao gồm cây thư mục, nội dung thư mục, và các mối quan hệ giữa các module. Dựa trên ngữ cảnh này và Coding Prompt, công cụ xác định các điểm chèn hoặc cập nhật mã mới. Khi tạo mã mới, công cụ không chỉ sinh nội dung theo yêu cầu mà còn áp dụng các quy tắc kiến trúc tổng quát, chẳng hạn như nguyên tắc phân tầng (tầng nghiệp vụ, tầng dữ liệu, tầng giao diện), nguyên tắc tách module, quy ước đặt tên thư mục – lớp – biến, và các nguyên tắc xử lý lỗi thống nhất.

Một điểm quan trọng trong Pha 2 là cơ chế tự kiểm tra và sửa lỗi. Mỗi đoạn mã sinh ra được đánh giá về tính hợp lệ, bao gồm cả cú pháp, logic, tính toàn vẹn của dữ liệu và sự tuân thủ kiến trúc. Nếu phát hiện bất kỳ sai lệch hoặc lỗi nào, hệ thống sẽ tự sinh sub-prompt để chỉnh sửa, đồng thời cập nhật mã đã sinh. Cơ chế này đảm bảo rằng kết quả cuối cùng không chỉ đúng về mặt chức năng mà còn duy trì được sự đồng nhất về phong cách và kiến trúc dự án.

Kết quả cuối cùng của Pha 2 là tập hợp các thư mục mã nguồn hoàn chỉnh hoặc được cập nhật, sẵn sàng để build hoặc chạy trực tiếp trong hệ thống. Tùy theo từng dự án, kết quả này có thể bao gồm các module nghiệp vụ, các thành phần giao diện, các hàm xử lý dữ liệu, hoặc các đoạn logic tích hợp API. Nhờ cấu trúc theo hai pha, phương pháp này vừa đảm bảo độ chính xác và nhất quán, vừa tăng tốc độ triển khai, giảm thiểu các lỗi do lập trình thủ công, đồng thời giúp lập trình viên tập trung vào các quyết định kiến trúc quan trọng và mở rộng chức năng thay vì viết từng dòng code cơ bản. Nhìn chung, phương pháp hai pha cho phép xây dựng phần mềm một cách có cấu trúc và tuần tự, giảm thiểu rủi ro và tăng tính ổn định của dự án. Pha 1 đảm bảo tất cả yêu cầu nghiệp vụ và giao diện được chuẩn hóa và mô tả chính xác, trong khi Pha 2 hiện thực hóa các yêu cầu này thành mã nguồn thực sự, tuân thủ các quy tắc kiến trúc tổng quát và có khả năng tự kiểm tra và sửa lỗi. Sự phối hợp chặt chẽ giữa hai pha tạo ra một luồng phát triển phần mềm hiệu quả, giảm thiểu lỗi, đồng thời duy trì tính mở rộng và khả năng bảo trì lâu dài.

3.2 Pha 1 – Sinh Coding Prompt

Pha đầu tiên trong quy trình tự động hoá lập trình bằng mô hình ngôn ngữ lớn (LLM) tập trung vào nhiệm vụ then chốt: chuyển đổi toàn bộ thông tin đặc tả nghiệp vụ thành một “Coding prompt” hoàn chỉnh, có cấu trúc, và sẵn sàng để tích hợp vào môi trường phát triển code như Cursor. Pha này đóng vai trò nền tảng, bởi chất lượng đầu ra của prompt ảnh hưởng trực tiếp đến tính đúng đắn, hiệu suất, và khả năng bảo trì của mã nguồn được sinh ra ở các pha kế tiếp. Đây là cầu nối trung gian giữa miền nghiệp vụ (Business Domain) và miền kỹ thuật (Code Domain).

3.2.1 Đầu vào của pha 1

Để đảm bảo mô hình AI có khả năng suy luận chính xác và bao quát, hệ thống tiếp nhận và xử lý đồng thời bốn luồng thông tin đầu vào khác nhau. Sự kết hợp này nhằm loại bỏ các điểm mù về ngữ cảnh mà một nguồn đơn lẻ thường mắc phải.

Đặc tả ca sử dụng

Đặc tả ca sử dụng được xem là thành phần cốt lõi, cung cấp toàn bộ logic nghiệp vụ của hệ thống. Nó mô tả chi tiết các tác nhân, hành vi, điều kiện và kết quả mong đợi trong từng kịch bản sử dụng, từ đó làm rõ cách thức tương tác giữa người dùng và hệ thống. Trong ngữ cảnh sinh mã tự động, tài liệu này không chỉ là một văn bản mô tả đơn thuần mà còn là tập hợp các ràng buộc hành vi, cung cấp cơ sở để máy có thể chuyển đổi trực tiếp các kịch bản nghiệp vụ thành mã nguồn hoạt động. Việc này giúp đảm bảo tính nhất quán giữa yêu cầu và triển khai, giảm thiểu lỗi do hiểu nhầm và rút ngắn thời gian phát triển phần mềm một cách đáng kể. Đồng thời, đặc tả ca sử dụng còn là công cụ tham khảo quan trọng cho các bước kiểm thử, bảo trì và mở rộng hệ thống sau này. Phân tích thành phần được mô tả trong bảng 3.1 :

Thành phần	Mô tả
Tác nhân	Định danh đối tượng tương tác với hệ thống, ví dụ: User, Admin, System.
Luồng sự kiện chính	Mô tả trình tự chuẩn để hoàn thành chức năng, cung cấp logic cơ bản cho hệ thống.
Luồng thay thế và Ngoại lệ	Bao gồm các kịch bản rẽ nhánh như mất kết nối, dữ liệu sai định dạng, giúp code sinh ra có khả năng chịu lỗi.
Tiền điều kiện và Hậu điều kiện	Xác định trạng thái hệ thống trước và sau khi thực thi, hỗ trợ việc sinh các đoạn mã kiểm tra và quản lý trạng thái.

Bảng 3.1: Phân tích các thành phần cốt lõi của đặc tả ca sử dụng.

Thiết kế UI/ Figma

Thiết kế giao diện người dùng (UI) thông qua công cụ Figma đóng vai trò quan trọng trong việc cung cấp ngữ cảnh thị giác và bổ sung những khía cạnh mà đặc tả ca sử dụng không thể mô tả trực tiếp, bao gồm bố cục, thành phần giao diện, trạng thái hiển thị, hành vi tương tác và cách điều hướng giữa các màn hình. Việc tích hợp Figma vào quy trình phát triển giúp các mô hình trợ lý lập trình hình dung một cách chính xác cách người dùng tương tác với hệ thống trong thực tế, từ đó nâng cao độ chính xác của mã nguồn được sinh ra.

Bằng cách phân tích các màn hình Figma, mô hình có thể xác định các component

cơ bản của giao diện, phân rã màn hình thành những thành phần nguyên tử như button, input, modal hoặc sidebar. Đồng thời, mô hình nhận diện các trạng thái hiển thị khác nhau của từng component, bao gồm hover, active, disabled, loading hoặc error, từ đó đảm bảo rằng mọi biến thể giao diện đều được xử lý đầy đủ. Ngoài ra, Figma cung cấp thông tin về hành vi tương tác của người dùng, cho phép mô hình ánh xạ các sự kiện như onClick, onSubmit hay onScroll vào các hàm xử lý sự kiện (event handlers) tương ứng trong mã nguồn. Các yếu tố hình ảnh này giúp hạn chế tình trạng hiểu sai hoặc bỏ sót chi tiết, đặc biệt trong các chức năng có nhiều trạng thái động, đồng thời tăng cường khả năng mô hình hóa chính xác các luồng nghiệp vụ và trải nghiệm người dùng.

Đặc tả API (API Contract)

Trong các dự án phần mềm có kiến trúc backend độc lập, đặc tả API đóng vai trò như một thỏa thuận ràng buộc chính xác giữa frontend và backend về cấu trúc và hành vi dữ liệu. Đặc tả này xác định một cách chi tiết các thông tin quan trọng của API, bao gồm đường dẫn endpoint, phương thức HTTP được sử dụng, các tham số truy vấn (query parameters) và tham số đường dẫn (path parameters), nội dung của request body, cũng như cấu trúc và kiểu dữ liệu của response body. Bên cạnh đó, đặc tả còn mô tả các mã lỗi có thể phát sinh và ý nghĩa của từng mã, cùng với các điều kiện nghiệp vụ liên quan mà dữ liệu phải tuân thủ. Việc cung cấp đầy đủ và chính xác những thông tin này cho phép các công cụ, bao gồm cả các mô hình trợ lý lập trình, xác định chính xác thời điểm gọi API, dữ liệu cần gửi đi, dữ liệu nhận lại và cách xử lý các trường hợp lỗi một cách hợp lý. Nhờ đó, đặc tả API không chỉ đảm bảo tính đồng bộ giữa frontend và backend mà còn nâng cao độ ổn định, khả năng mở rộng và tính tin cậy của toàn bộ hệ thống.

Template Prompt chuẩn hoá

Để đảm bảo đầu ra có cấu trúc nhất quán và giảm thiểu mâu thuẫn giữa các prompt sinh ra từ nhiều ca sử dụng khác nhau, pha tổng hợp thông tin sử dụng một template coding prompt cố định. Template này định nghĩa các phần thông tin mà mỗi prompt bắt buộc phải chứa, bao gồm mô tả mục tiêu của chức năng, công nghệ dự án đang sử dụng (React, React Native, Next.js, Spring Boot, NestJS...), quy chuẩn lập trình (Clean Code, đặt tên biến, cấu trúc thư mục...), yêu cầu về API, validation, xử lý lỗi, khả năng mở

rộng và tái sử dụng, cũng như định nghĩa format đầu ra mà Cursor cần tuân thủ. Template chất lượng cao đóng vai trò như một “khung” thống nhất, đảm bảo mọi prompt sinh ra đều có phong cách nhất quán, giúp mô hình sinh code dễ hiểu, dễ thực thi và duy trì tính đồng bộ trong toàn dự án. Cụ thể, template được chia thành bốn nhóm prompt chính:

Prompt A: Backend API Service. Trong giai đoạn phát triển backend, trọng tâm là xây dựng các dịch vụ và kho dữ liệu trung gian nhằm xử lý các yêu cầu API, định nghĩa mô hình dữ liệu và quản lý logic nghiệp vụ của hệ thống. Các service và repository được thiết kế tách biệt giữa việc truy xuất dữ liệu và xử lý logic, đảm bảo tính mở rộng, dễ bảo trì và dễ kiểm thử. Mô hình dữ liệu (models) được định nghĩa chính xác để phản ánh cấu trúc dữ liệu cần quản lý và trao đổi với client. Việc tách riêng logic nghiệp vụ khỏi giao diện giúp hệ thống duy trì sự ổn định, đảm bảo rằng mọi xử lý phức tạp đều được thực hiện ở phía server trước khi trả dữ liệu về client.

Cấu Trúc Chi Tiết Của Prompt A: Sinh Mã Backend (NestJS Service/Controller)

Mục tiêu: Xây dựng dịch vụ API, logic nghiệp vụ và xử lý lỗi phía server.

“Tạo endpoint [Phương thức API] [Đường dẫn API] trong [Tên Controller] NestJS. Endpoint này PHẢI được bảo vệ bằng JwtAuthGuard.

ĐỊNH DẠNG REQUEST: [Dán Request Body Mẫu hoặc Query Params Mẫu]. [Tên Service]Service phải viết phương thức xử lý chính. Logic: [Mô tả chi tiết các bước nghiệp vụ, bao gồm validation].

*ĐỊNH DẠNG RESPONSE THÀNH CÔNG: [Dán Response Mẫu thành công].
Xử lý Lỗi: Nếu [Tình huống lỗi xảy ra], throw [Loại Exception NestJS] với
JSON lỗi: [Mô tả Response lỗi chính xác].”*

Prompt B: Cấu trúc và Giao diện (UI). Giai đoạn thiết kế giao diện tập trung vào xây dựng khung cơ bản cho các thành phần UI và tổ chức cấu trúc thư mục theo tiêu chuẩn dự án. Các component giao diện được khởi tạo với trạng thái ban đầu, sẵn sàng cho việc tích hợp dữ liệu từ backend và logic xử lý phức tạp ở các giai đoạn tiếp theo. Việc phân tách thư mục, tách riêng các màn hình, component, style và tài nguyên giúp mã nguồn rõ ràng, thuận tiện cho bảo trì và mở rộng. Giao diện được thiết kế nhất quán về phong cách, màu sắc, typography và bố cục, đảm bảo trải nghiệm người dùng mạch lạc, trực quan và thống nhất.

Cấu Trúc Chi Tiết Của Prompt B: Sinh Mã Frontend (React Component & UI/MCP)

Mục tiêu: Xây dựng giao diện theo đúng thiết kế Figma.

“Tạo component [Tên Component FE] bằng React TypeScript và Tailwind CSS. Component này PHẢI hiển thị [Mô tả các phần tử UI chính]. STRICTLY sử dụng giao diện thiết kế từ Figma MCP tại liên kết: [Figma Link/Selection ID]. Đảm bảo bố cục và màu sắc khớp 100% với thiết kế.”

Prompt C: Logic Chính và Quản lý Trạng thái. Sau khi xây dựng UI cơ bản, giai đoạn tiếp theo là tích hợp dữ liệu từ backend vào giao diện và quản lý trạng thái ứng dụng nhằm đảm bảo sự đồng bộ và chính xác của dữ liệu trong toàn bộ ca sử dụng. Các API service từ backend được gọi từ các component UI, dữ liệu nhận về được xử lý và hiển thị hợp lý. Đồng thời, các trạng thái loading, success và error được quản lý rõ ràng. Việc quản lý trạng thái, dù là local state hay global state thông qua context hoặc các thư viện quản lý trạng thái, giúp đồng bộ hóa dữ liệu giữa UI và backend, đảm bảo mọi thay đổi đều được phản ánh chính xác. Trước khi gửi dữ liệu lên server hoặc hiển thị trên UI, hệ thống thực hiện kiểm tra tính hợp lệ, định dạng dữ liệu và các điều kiện nghiệp vụ, giảm thiểu rủi ro lỗi và nâng cao độ tin cậy.

Cấu Trúc Chi Tiết Của Prompt C: Sinh Mã Frontend Logic (State & Kết nối API)

Mục tiêu: Kết nối giao diện với API và triển khai luồng xử lý thành công.

“Tiếp tục làm việc trên [Tên Component FE]. Thêm các biến trạng thái cần thiết. Triển khai hàm bắt đầu bộ [Tên hàm xử lý] để gửi request tới API [Phương thức và đường dẫn đã tạo trong Prompt D].

NỘI DUNG REQUEST: Chuẩn bị payload dựa trên [Dán Request Body Mẫu đã cho] từ state của Form.

Nếu thành công, sử dụng cấu trúc [Dán Response Mẫu thành công] để thực hiện [Mô tả các bước sau thành công, ví dụ: Lưu accessToken, Điều hướng, Cập nhật state].”

Prompt D: Xử lý Lỗi và Kiểm thử. Để đảm bảo hệ thống hoạt động ổn định và đáng tin cậy, việc xử lý lỗi và kiểm thử được thực hiện song song với phát triển logic

chính. Tất cả thao tác gọi API đều được bao bọc trong các khối xử lý ngoại lệ, cho phép phân loại và xử lý các tình huống bất thường như lỗi kết nối, lỗi server, dữ liệu đầu vào không hợp lệ, tài nguyên không tìm thấy hay lỗi xác thực và ủy quyền. Thông báo lỗi được thiết kế hiển thị rõ ràng, thân thiện và dễ hiểu với người dùng, sử dụng các cơ chế như toast, modal hoặc thông báo inline nhằm tránh gián đoạn trải nghiệm.

Cấu Trúc Chi Tiết Của Prompt D: Xử lý Ngoại lệ (Client-side Validation & Error Handling)

Mục tiêu: Hoàn thiện xử lý lỗi, kiểm tra dữ liệu phía client và trạng thái tải.

“Tinh chỉnh hàm [Tên hàm xử lý] trong Component FE:

Trạng thái Loading: Vô hiệu hóa/Hiển thị Spinner khi isLoading là true.

Lỗi API: Nếu API trả về lỗi (ví dụ: 400/500), hiển thị thông báo lỗi chính xác: [Thông báo lỗi chi tiết hoặc thông báo lỗi chung] tại [Vị trí hiển thị lỗi].

Validation FE: Thêm kiểm tra đầu vào phía client (ví dụ: [Mô tả Rule Validation FE, ví dụ: check password match, field not empty]) trước khi gọi API.”

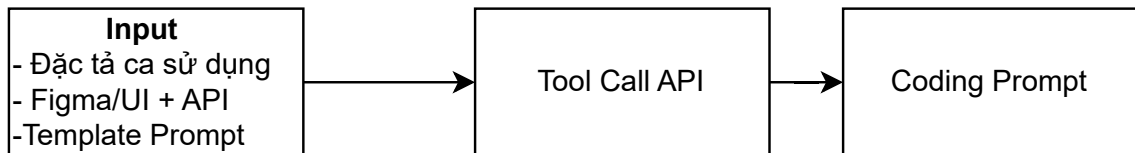
Hệ thống được kiểm thử ở nhiều mức độ khác nhau. Kiểm thử đơn vị (unit test) đảm bảo các hàm, service hoặc module hoạt động đúng với dữ liệu chuẩn và xử lý ngoại lệ chính xác. Kiểm thử tích hợp (integration test) đánh giá sự tương tác giữa UI và backend, đảm bảo dữ liệu được xử lý và hiển thị đúng, đồng thời lỗi được phát hiện và xử lý phù hợp. Việc ghi nhận lỗi thông qua logging hoặc các công cụ quản lý lỗi chuyên nghiệp giúp phân tích nguyên nhân sự cố và nâng cao độ tin cậy. Kết hợp xử lý lỗi và kiểm thử toàn diện là yếu tố then chốt để giảm thiểu rủi ro, nâng cao tính ổn định của ứng dụng và tối ưu trải nghiệm người dùng.

Nhờ việc sử dụng template cố định, quá trình sinh code trở nên nhất quán, dễ kiểm soát và có thể áp dụng cho nhiều chức năng khác nhau mà vẫn giữ được phong cách lập trình thống nhất trong toàn dự án.

3.2.2 Quy trình xử lý

Trong hệ thống, toàn bộ dữ liệu đầu vào (ca sử dụng, template, API, thiết kế giao diện) được chuyển vào một công cụ nội bộ đóng vai trò như lớp trung gian xử lý – tổng hợp trước khi gửi yêu cầu đến mô hình Gemini. Công cụ này được triển khai dưới dạng

một API service sử dụng Node.js/Express, với nhiệm vụ chuẩn hóa dữ liệu, hợp nhất ngữ cảnh và tạo prompt cuối cùng để mô hình Gemini sinh mã nguồn. Mã nguồn công cụ mọi người có thể tham khảo tại đường dẫn (https://github.com/thanhttruc/tool_call_api). Hình 3.2 mô tả quy trình trung gian trong Pha 1 để sinh Coding Prompt:



Hình 3.2: Mô tả quy trình Pha 1 xử lý để sinh Coding Prompt.

Đoạn code 1 : Công cụ Gọi API Gemini để sinh Coding Prompt

```
{      // Tạo prompt từ usecase và template
const prompt = '
Dựa trên đặc tả usecase và template đầu ra sau đây,
hãy tạo kết quả phù hợp:

**Promt Gốc:**
${usecase}
**Template Đầu Ra:**
${template}
Hãy tạo kết quả theo đúng template và đáp ứng
các yêu cầu trong usecase.';
// Gọi Gemini API
const model = genAI.getGenerativeModel
    ({ model: 'gemini-2.5-pro' });
const result = await model.generateContent(prompt);
const response = await result.response;
const text = response.text();
// Trả về kết quả
res.json({
    success: true,
```

```

    result: text,
    usecase: usecase,
    template: template
  });
} catch (error) {
  console.error('Error calling Gemini API:', error);
  res.status(500).json({
    error: 'Lỗi khi gọi Gemini API',
    message: error.message
  });}

```

Đoạn code trên mô tả công cụ xử lý trung gian được thiết kế và triển khai trên nền tảng Node.js kết hợp với khung làm việc Express.js, đóng vai trò là lớp giao tiếp kỹ thuật thiết yếu giữa dữ liệu đặc tả đầu vào và mô hình sinh mã Gemini. Với đặc tính nhẹ, linh hoạt và khả năng hỗ trợ mạnh mẽ các cơ chế RESTful, hệ thống đảm bảo khả năng mở rộng và tính ổn định trong môi trường phát triển phân tán. Công cụ này không chỉ đơn thuần là một cầu nối truyền tải dữ liệu mà còn thực hiện chức năng chuẩn hóa, xác thực và quản lý an toàn thông tin. Việc tích hợp thư viện dotenv cho phép quản lý các cấu hình nhạy cảm, đặc biệt là khóa API, tách biệt hoàn toàn khỏi mã nguồn, qua đó tăng cường tính bảo mật và khả năng thích ứng linh hoạt khi triển khai trên đa dạng các môi trường hạ tầng.

Quy trình cốt lõi của công cụ tập trung vào việc tổng hợp và chuyển đổi các yêu cầu nghiệp vụ trừu tượng thành các chỉ thị kỹ thuật chính xác (coding prompt) cho mô hình AI. Trước khi thực hiện gọi API, hệ thống tiến hành phân tích sâu chuỗi dữ liệu đầu vào bao gồm đặc tả ca sử dụng và bản thiết kế giao diện (UI). Giai đoạn này thực hiện việc bóc tách luồng nghiệp vụ, ánh xạ các tương tác từ giao diện người dùng (UI) sang logic xử lý backend và nhận diện các điểm kết nối API cần thiết. Thông qua việc tái hiện tư duy lập trình hệ thống, công cụ xác định rõ ràng các điều kiện tiên quyết, quy tắc xác thực dữ liệu, cơ chế xử lý lỗi và các trạng thái phản hồi. Kết quả của quá trình phân tích này là một tập hợp dữ liệu có cấu trúc chặt chẽ, liên kết logic giữa hành vi người dùng và xử lý hệ thống, đóng vai trò là ngữ cảnh đầu vào quan trọng để định hướng mô hình sinh mã.

Giai đoạn tương tác với mô hình ngôn ngữ lớn được thực hiện thông qua SDK Google Generative AI, kết nối trực tiếp đến phiên bản mô hình gemini-2.5-pro. Tại đây, các dữ liệu đã qua xử lý được tổng hợp vào một khuôn mẫu (template) chuẩn hóa, bao gồm đầy đủ các ràng buộc về kiến trúc phần mềm (như Clean Architecture), quy chuẩn viết mã (coding style) và cấu trúc dự án. Việc sử dụng cơ chế xử lý bất đồng bộ (asynchronous) trong quá trình gọi API giúp tối ưu hóa hiệu năng hệ thống, cho phép xử lý song song nhiều yêu cầu phức tạp mà không gây tắc nghẽn tài nguyên máy chủ. Mô hình Gemini, dựa trên prompt kỹ thuật chi tiết này, sẽ thực hiện sinh mã nguồn với độ chính xác cao, đảm bảo tính nhất quán giữa đặc tả nghiệp vụ ban đầu và sản phẩm mã hóa cuối cùng.

Tổng thể, công cụ xử lý trung gian thiết lập một quy trình khép kín từ việc tiếp nhận yêu cầu, phân tích logic, đến việc tương tác với trí tuệ nhân tạo để tự động hóa quy trình phát triển phần mềm. Bằng cách chuẩn hóa dữ liệu đầu vào và kiểm soát chặt chẽ đầu ra, hệ thống không chỉ giảm thiểu các sai sót tiềm ẩn do yếu tố con người mà còn nâng cao đáng kể chất lượng mã nguồn, tạo tiền đề vững chắc cho việc tích hợp các tính năng mở rộng và duy trì tính khả thi của dự án trong dài hạn.

3.2.3 Đầu ra của pha 1

Kết quả của Giai đoạn 1 là một coding prompt hoàn chỉnh, sẵn sàng để nạp trực tiếp vào môi trường phát triển như Cursor. Khác với các mô tả bằng ngôn ngữ tự nhiên vốn mơ hồ, prompt này được xây dựng như một tập lệnh kỹ thuật chi tiết, bao phủ toàn bộ logic nghiệp vụ và các trạng thái giao diện, đảm bảo tính đầy đủ. Đồng thời, prompt thể hiện rõ các tham chiếu chéo giữa UI, logic và dữ liệu từ API, giúp duy trì tính liên kết trong toàn bộ hệ thống. Các hướng dẫn về kiến trúc, coding style và quy chuẩn dự án được tích hợp trực tiếp, đảm bảo mã nguồn sinh ra tuân thủ chặt chẽ các nguyên tắc Clean Code và cấu trúc dự án đã định nghĩa. Bên cạnh đó, prompt được tối ưu hóa để các mô hình sinh code như Claude 3.5 Sonnet hoặc GPT-4o trong Cursor có thể hiểu và sinh mã chính xác ngay lần đầu, đạt hiệu quả Zero-shot hoặc Few-shot cao. Quá trình tổng hợp và chuyển đổi thông tin được minh họa qua sơ đồ: từ input tổng hợp, thông qua tool nội bộ để hình thành prompt hoàn chỉnh, sau đó nạp trực tiếp vào Cursor để sinh code tự động. Hình 3.3 mô tả giao diện của công cụ gọi API Gemini: người dùng nhập các input đầu vào và hệ thống trả về kết quả là Coding Prompt.

Đặc tả Usecase *

```

"email": "johndoe@email.com"
}
}

Response lỗi:
{ "error": "Email hoặc mật khẩu không đúng." }

```

2374 ký tự

Template Đầu Ra *

1. Prompt A: Sinh Mã Backend (NestJS Service/Controller)

Mục tiêu: Xây dựng dịch vụ API, logic nghiệp vụ, và xử lý lỗi server.
Tạo endpoint [Phương thức API] [Đường dẫn API] trong [Tên Controller] NestJS. Endpoint này PHẢI được bảo vệ bằng JwtAuthGuard.
ĐINH DẠNG REQUEST: [Dán Request Body Mẫu hoặc Query Params Mẫu].
[Tên Service]Service phải viết phương thức xử lý chính. Logic: [Mô tả chi tiết các bước nghiệp vụ, bao gồm validation].
ĐINH DẠNG RESPONSE THÀNH CÔNG: [Dán Response Mẫu thành công].

2186 ký tự

Tạo Kết Quả với Gemini

Kết Quả:

Community-/node-1d=13/-/4//&t=JW185rgcguz3ulcc-4**. Đảm bảo do cục và màu sắc knop 100% với thiết ke."

3. Prompt C: Sinh Mã Frontend Logic (State & Kết nối API)

Mục tiêu: Kết nối UI với API và xử lý luồng thành công.

Hình 3.3: Giao diện của công cụ gọi API Gemini.

3.3 Pha 2 – Sinh mã nguồn

3.3.1 Đầu vào pha 2

Coding Prompt

Trong giai đoạn Pha 2 của quy trình phát triển hệ thống bằng mô hình sinh mã, Coding Prompt giữ vai trò trọng yếu, được xem như hình thức đặc tả kỹ thuật tối ưu hóa cho quá trình tự động hoá mã nguồn. Coding Prompt không chỉ mô tả một tập hợp yêu cầu chức năng rời rạc, mà được cấu trúc như một văn bản kỹ thuật hoàn chỉnh, định hướng cả quá trình thực thi của mô hình. Việc trình bày Coding Prompt dưới dạng văn xuôi có tổ chức thay cho danh sách liệt kê giúp đảm bảo độ rõ ràng, tính mạch lạc và sự toàn vẹn về mặt ý nghĩa trong toàn bộ đặc tả.

Trước hết, Coding Prompt cần xác định rõ mục tiêu nghiệp vụ của chức năng được triển khai, bao gồm phạm vi tác động, vai trò của chức năng trong hệ thống, và kết quả mong đợi sau khi hoàn tất xử lý. Việc mô tả mục tiêu theo cách này nhằm thiết lập nền

tăng nhận thức thống nhất cho mô hình sinh mã, qua đó đảm bảo mã được tạo ra phù hợp với yêu cầu vận hành và cấu trúc của hệ thống.

Bên cạnh đó, Coding Prompt phải mô tả chi tiết luồng xử lý logic, thể hiện cách dữ liệu được tiếp nhận, kiểm tra, chuyển tiếp và xử lý trong các nhánh nghiệp vụ khác nhau. Phần này bao gồm cả các điều kiện hợp lệ, điều kiện sai phạm, phương thức hệ thống phản hồi trong trường hợp ngoại lệ, cũng như tính nhất quán trong xử lý dữ liệu giữa backend và frontend. Luồng logic được mô tả một cách có hệ thống giúp giảm thiểu sai lệch trong quá trình sinh mã, đồng thời tăng cường khả năng mô hình tái tạo chính xác hành vi nghiệp vụ.

Ngoài ra, Coding Prompt cần cung cấp mô tả giao diện và hành vi người dùng đối với các chức năng liên quan đến UI. Nội dung này bao gồm bố cục, đặc tính thẩm mỹ theo thiết kế Figma, các trạng thái giao diện, cách thức hệ thống phản hồi trước thao tác của người dùng và các quy tắc kiểm tra dữ liệu phía client. Việc mô tả nhất quán các yếu tố này nhằm bảo đảm kết quả sinh mã đáp ứng chuẩn thiết kế, duy trì tính thống nhất giữa đặc tả giao diện và hiện thực hoá kỹ thuật. Tổng thể, Coding Prompt đóng vai trò như văn kiện trung gian giữa phân tích nghiệp vụ và giai đoạn triển khai tự động hóa bằng mô hình sinh mã. Một Coding Prompt được xây dựng theo văn phong học thuật, chặt chẽ về cấu trúc và đầy đủ về ngữ nghĩa sẽ nâng cao độ chính xác của mã sinh ra, giảm nhu cầu chỉnh sửa vòng lặp, đồng thời tối ưu hóa hiệu quả của toàn bộ quy trình phát triển trong Pha 2.

Mã nguồn nền tảng

Bên cạnh Coding Prompt, Pha 2 còn dựa vào mã nguồn nền tảng (codebase ban đầu) như một thành phần cốt lõi, cung cấp ngữ cảnh kỹ thuật và kiến trúc đã được thiết lập trước. Mã nguồn nền tảng này không chỉ là điểm xuất phát để mở rộng các chức năng nghiệp vụ, mà còn là “bản đồ” giúp hệ thống sinh mã hiểu được cách tổ chức dự án, cấu trúc thư mục, quy ước đặt tên, và cách các thành phần giao tiếp với nhau. Nhờ có mã nguồn nền tảng, mã mới được sinh ra trong Pha 2 có thể tích hợp trực tiếp vào hệ thống hiện hữu mà không phá vỡ các chuẩn kiến trúc hoặc luồng dữ liệu đã được định hình.

Ở tầng máy chủ, mã nguồn nền tảng đã xác lập các thành phần cơ bản của hệ thống, bao gồm cấu trúc khởi tạo dự án, cơ chế kết nối với hạ tầng lưu trữ dữ liệu, các mô hình

dữ liệu nền tảng và bộ công cụ hỗ trợ xác thực, cấu hình, kiểm tra hợp lệ và quản lý lỗi. Hệ thống đã xây dựng một bộ mô hình dữ liệu cơ bản, bao quát các thực thể quản lý thông tin định danh, các thực thể lưu vết trạng thái và quan hệ, cũng như các thực thể phân loại, phục vụ việc tổ chức, phân tách và vận hành dữ liệu một cách tổng quát. Bên cạnh đó, các cấu hình hệ thống quan trọng như tham số môi trường, kết nối dữ liệu, thiết lập bảo mật, tài liệu hóa API, cơ chế kiểm soát truy cập và cấu hình giao tiếp giữa các mô-đun cũng đã được chuẩn bị. Mặc dù nhiều mô-đun nghiệp vụ vẫn chưa được triển khai chi tiết, mã nguồn nền tảng đã cung cấp khung sẵn có để mô hình sinh mã có thể tạo ra Controllers, Services và DTOs một cách thống nhất, đồng thời đảm bảo tính toàn vẹn và khả năng mở rộng của hệ thống.

Ở tầng giao diện người dùng, mã nguồn nền tảng đã hình thành cấu trúc nền của ứng dụng, bao gồm hệ thống định tuyến, quản lý trạng thái, cấu trúc trang và các thành phần UI dùng chung. Các dịch vụ gọi API đã được định hình về mặt cấu trúc, kèm theo cơ chế tiền xử lý và xử lý lỗi cho các yêu cầu mạng. Hệ thống giao diện đã chuẩn bị sẵn các thành phần tiện ích, lớp định dạng, cấu trúc thư mục, và cơ chế tổ chức dữ liệu hiển thị. Nhờ đó, các thành phần UI mới được sinh ra có thể tuân thủ chuẩn thiết kế, nhất quán về bố cục, tương tác và cách hiển thị dữ liệu với các thành phần đã tồn tại. Tổng thể, mã nguồn nền tảng hiện tại đã cung cấp một khung kiến trúc đầy đủ và ổn định, bao gồm mô hình dữ liệu nền tảng, cấu hình hệ thống, cơ chế bảo mật, cấu trúc giao diện và các phương thức trao đổi dữ liệu. Các thành phần này không chỉ đóng vai trò là nền tảng để mở rộng các chức năng nghiệp vụ mà còn là ngữ cảnh bắt buộc để đảm bảo rằng mã sinh ra trong Pha 2 tuân thủ chuẩn tổ chức, phong cách, quy tắc đặt tên và kiến trúc tổng thể của dự án. Nhờ vậy, quá trình sinh mã có thể diễn ra một cách đồng bộ, chính xác và giảm thiểu rủi ro xung đột hoặc lệch chuẩn trong toàn bộ hệ thống.

Quy tắc sinh mã của dự án

Trong phạm vi Pha 2, ngoài Coding Prompt và mã nguồn nền tảng, hệ thống còn tiếp nhận tập quy tắc sinh mã của dự án, được thiết kế như bộ định chuẩn bắt buộc để mọi mã sinh ra phải tuân theo. Khác với các nguyên tắc trừu tượng chỉ mang tính hướng dẫn chung, tập quy tắc sinh mã này xác định cả cấu trúc logic lẫn công nghệ và framework cụ thể mà dự án đang sử dụng. Điều này bao gồm việc quy định rõ mô hình tổ chức mã nguồn, cách phân tách tầng, chuẩn thư mục, cơ chế quản lý trạng thái, quy ước đặt tên,

cùng với việc chỉ định các công nghệ nền tảng đang áp dụng ở từng phần của hệ thống.

Trước hết, tập quy tắc sinh mã xác lập khung công nghệ và framework chủ đạo. Ở tầng xử lý phía máy chủ, quy tắc chỉ rõ hệ thống sử dụng mô hình module hoá kết hợp cơ chế tiêm phụ thuộc, chuẩn hóa cách tổ chức controller, service, cấu trúc dữ liệu và tầng truy xuất. Ở tầng giao diện, tập quy tắc sinh mã định nghĩa cơ chế xây dựng giao diện dựa trên component, quy tắc định tuyến, cách quản lý trạng thái và cách tổ chức các phần tử UI. Việc ràng buộc vào framework cụ thể giúp đảm bảo mã sinh ra luôn tương thích với kiến trúc hiện tại, đồng thời loại bỏ nguy cơ tạo ra các cấu trúc không phù hợp hoặc lệch chuẩn so với dự án.

Bên cạnh đó, tập quy tắc sinh mã đưa ra chuẩn phân tách tầng ở mức chi tiết: tầng giao tiếp dữ liệu, tầng xử lý nghiệp vụ, tầng giao diện, tầng tiện ích chung và tầng cấu hình hệ thống. Từng tầng được mô tả bằng vai trò, mối quan hệ phụ thuộc và phạm vi trách nhiệm, giúp mã sinh ra luôn duy trì sự tách biệt logic, dễ mở rộng và dễ kiểm thử.

Các quy tắc cũng chỉ định chuẩn tổ chức thư mục phản ánh đúng cấu trúc của dự án: mỗi module gồm mô hình dữ liệu, giao diện trao đổi dữ liệu, định nghĩa của các chức năng xử lý và phần khung của giao tiếp bên ngoài. Tương tự, mã phía giao diện người dùng được tổ chức theo component, page, hooks, service gọi API, cùng với chuẩn định danh và trách nhiệm rõ ràng giữa các thành phần.

Quy ước coding convention là thành phần cốt lõi của bộ quy tắc: chuẩn đặt tên class, thư mục, component, biến, hàm; quy chuẩn định dạng phản hồi dữ liệu; chuẩn commit khi làm việc với hệ thống quản lý mã nguồn; cùng với các tiêu chuẩn quản lý lỗi và ghi log thống nhất. Những quy tắc này đảm bảo mã sinh ra nhất quán giữa các phiên, tránh sự sai lệch về phong cách hoặc cấu trúc.

Ngoài ra, tập quy tắc sinh mã còn thiết lập quy trình quản lý trạng thái, xử lý lỗi và giao tiếp giữa các tầng, quy định cách sử dụng bộ công cụ xác thực, cách gọi API, cách tổ chức dữ liệu hiển thị và cơ chế tiền xử lý – hậu xử lý ở từng lớp. Việc chuẩn hóa cả mặt kỹ thuật lẫn công nghệ đảm bảo rằng mã sinh ra không chỉ khớp về mặt cú pháp, mà còn vận hành đúng ngữ cảnh công nghệ của toàn bộ dự án.

Nhìn chung, các quy tắc sinh mã đóng vai trò như “bản hợp đồng kỹ thuật” giữa Pha 2 và toàn bộ hệ thống, buộc mọi mã sinh ra phải tuân thủ framework đã chọn, cấu

trúc chuẩn, quy trình kỹ thuật và phong cách lập trình thống nhất. Điều này bảo đảm tính ổn định, nhất quán và khả năng mở rộng lâu dài của dự án, đồng thời giảm thiểu sai lệch giữa các phiên chạy sinh mã khác nhau.

3.3.2 Quy trình sinh mã

Quá trình sinh mã trong Pha 2 được vận hành theo một chuỗi xử lý tuần tự, có khả năng lặp lại tự động và đảm bảo tính nhất quán qua mỗi phiên sinh mã. Khi quá trình bắt đầu, hệ thống — thông qua Cursor — thực hiện bước khởi tạo bằng việc tải toàn bộ ngữ cảnh dự án. Ngữ cảnh này bao gồm Coding Prompt, cây thư mục của mã nguồn hiện có, các tệp cấu hình liên quan, những mô-đun đã được triển khai cũng như các quy tắc kiến trúc được định nghĩa trước đó. Việc nạp đầy đủ các nguồn dữ liệu này cho phép Cursor xây dựng một mô hình biểu diễn nội tại về cấu trúc hệ thống. Mô hình này đóng vai trò tương tự như quá trình mà một lập trình viên tiếp nhận và phân tích mã nguồn nền tảng để hiểu vị trí, mối liên kết và vai trò của từng thành phần trước khi phát triển chức năng mới.

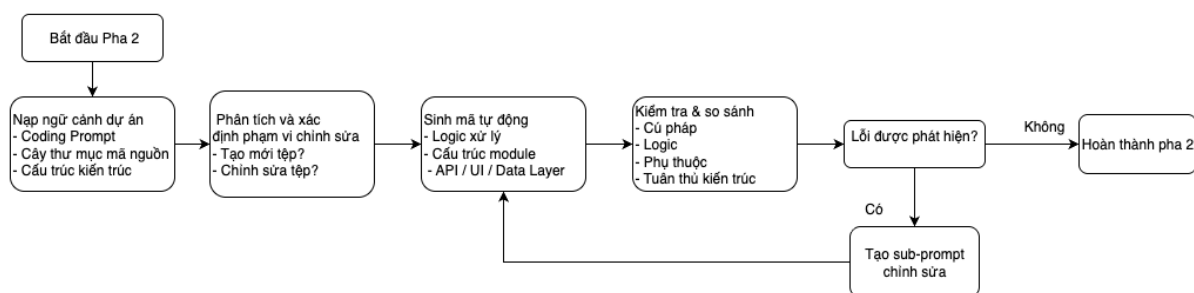
Sau khi hoàn thiện quá trình nạp ngữ cảnh, Cursor tiến hành giai đoạn định vị phạm vi công việc. Ở bước này, hệ thống tự động xác định phần mã cần sinh mới và phần mã cần được mở rộng hoặc điều chỉnh. Quyết định này dựa trên sự kết hợp giữa Coding Prompt (mô tả yêu cầu tính năng) và bộ quy tắc kiến trúc, bao gồm cách tổ chức dự án, chuẩn công nghệ, quy tắc đặt tên và nguyên tắc phân tách tầng. Nếu yêu cầu từ Coding Prompt tương ứng với một chức năng chưa tồn tại trong dự án, Cursor sẽ tự động khởi tạo các tệp mới và đặt chúng vào đúng vị trí trong cấu trúc thư mục, tuân theo các quy ước framework và kiến trúc đã thiết lập. Ngược lại, khi chức năng liên quan đã tồn tại, hệ thống sẽ tiến hành chỉnh sửa có chọn lọc, chỉ tác động vào những vị trí cần thay đổi mà không làm ảnh hưởng đến các phần khác trong dự án. Việc chỉnh sửa được thực hiện theo hướng tối thiểu hoá thay đổi, bảo toàn cấu trúc hiện hữu và tránh tạo ra xung đột hoặc sai lệch logic.

Khi phạm vi chỉnh sửa đã được xác định, hệ thống chuyển sang bước sinh mã. Tại đây, Cursor tạo ra các đoạn mã hoàn chỉnh dựa trên yêu cầu: xử lý nghiệp vụ, giao tiếp dữ liệu, tương tác API, trình bày giao diện hoặc cấu trúc tổ chức mô-đun. Việc sinh mã không chỉ dừng ở mức tạo ra các tệp rỗng và chèn nội dung tương ứng, mà còn bao gồm

quá trình hoàn thiện chi tiết kỹ thuật trong từng thành phần, chẳng hạn như:

- Cấu trúc import thống nhất và hợp lệ,
- Khai báo lớp, hàm và biến đúng quy ước đặt tên,
- Đảm bảo luồng dữ liệu giữa các tầng tuân theo kiến trúc đã quy định,
- Duy trì tính toàn vẹn của các mối quan hệ giữa các mô-đun,
- Tổ chức mã theo đúng tiêu chuẩn coding convention của dự án.

Một đặc điểm trọng yếu của Pha 2 là sự xuất hiện của vòng lặp tự hiệu chỉnh (self-correction loop). Sau mỗi lần sinh mã, Cursor sẽ tự động phân tích lại phần mã vừa được tạo hoặc cập nhật, so sánh chúng với Coding Prompt, bộ quy tắc kiến trúc và trạng thái mã nguồn nền tảng hiện tại. Nếu phát hiện sai lệch — bao gồm lỗi cú pháp, lỗi logic, lỗi phụ thuộc, cấu trúc không đúng chuẩn, hoặc bất kỳ dấu hiệu nào vi phạm quy tắc kiến trúc, hệ thống sẽ tự động sinh ra các sub-prompt, đóng vai trò như những yêu cầu chỉnh sửa mục tiêu. Các sub-prompt này hướng dẫn mô hình thực hiện điều chỉnh cụ thể, giúp mã được hoàn thiện từng bước qua nhiều vòng lặp liên tiếp cho đến khi đáp ứng đầy đủ yêu cầu chức năng, tuân thủ toàn bộ quy tắc kiến trúc, duy trì tính nhất quán trong phong cách viết mã và không còn lỗi cú pháp hay lỗi vận hành. Hình 3.4 mô tả quy trình xử lý ở Pha 2 để đạt được đầu ra là mã nguồn chạy được:



Hình 3.4: Các bước xử lý ở Pha 2.

Nhờ cơ chế tự lặp và tự điều chỉnh này, kết quả sinh mã trong Pha 2 đạt được sự ổn định, chính xác kỹ thuật và đồng nhất, tương đương với quy trình mà một lập trình viên thực hiện theo từng vòng refactor. Đồng thời, việc tự động hoá hoàn toàn quy trình

không chỉ rút ngắn thời gian phát triển mà còn giảm thiểu nguy cơ sai lệch về kiến trúc hoặc phong cách, đặc biệt trong những dự án có quy mô lớn và yêu cầu chặt chẽ về tiêu chuẩn lập trình.

3.3.3 Đầu ra của Pha 2

Kết quả đầu ra của pha 2 là một tập hợp mã nguồn hoàn chỉnh, bao gồm cả những tệp mới được sinh ra và những tệp đã được chỉnh sửa theo đúng yêu cầu chức năng. Các tệp này không chỉ tồn tại ở dạng khung mã cơ bản, mà đã bao gồm đầy đủ nội dung thực thi cần thiết: từ logic nghiệp vụ, cơ chế xử lý dữ liệu, các thành phần giao diện, cho đến cấu hình hệ thống liên quan. Toàn bộ nội dung mã được tổ chức theo đúng cấu trúc dự án và tuân thủ những nguyên tắc kiến trúc đã được quy định từ pha chuẩn bị, bảo đảm tính nhất quán và khả năng mở rộng trong các lần phát triển tiếp theo.

Song song với việc tạo mã, hệ thống cũng ghi nhận và tổng hợp danh sách các tệp mới xuất hiện trong dự án. Danh sách này đóng vai trò như một nhật ký thay đổi, giúp lập trình viên dễ dàng theo dõi sự biến động của mã nguồn nền tảng qua từng chu kỳ sinh mã. Việc theo dõi này đặc biệt quan trọng trong môi trường phát triển dựa trên mô hình tự động, nơi nhiều tệp có thể được tạo ra hoặc thay đổi trong một lần chạy duy nhất. Nhờ đó, nhóm phát triển có thể kiểm soát tốt hơn phạm vi chỉnh sửa, đánh giá tác động của mỗi lần sinh mã và đảm bảo tính minh bạch của quá trình phát triển phần mềm.

Ở góc độ tổng thể, pha 2 cung cấp một gói mã hoàn chỉnh và có khả năng thực thi ngay lập tức. Tùy theo đặc điểm và công nghệ của dự án, mã nguồn có thể được chạy trực tiếp hoặc tiến hành build mà không cần chỉnh sửa thêm. Khả năng này giúp quá trình phát triển không bị gián đoạn: lập trình viên có thể ngay lập tức kiểm thử, mở rộng chức năng hoặc tích hợp mã vào các thành phần hệ thống khác. Đồng thời, việc có được một phiên bản mã hoàn chỉnh sau mỗi vòng sinh mã tạo ra nền tảng ổn định để triển khai các pha tiếp theo, bảo đảm tính liên tục và chất lượng của chu trình phát triển phần mềm.

3.4 Xử lý ngoại lệ

Trong chu trình phát triển phần mềm hiện đại, việc phát sinh các ngoại lệ (exceptions) và lỗi (errors) là một thực tế tất yếu. Những vấn đề này không chỉ xuất hiện ngẫu

nhien, mà còn phản ánh các sai sót trong logic nghiệp vụ, cấu trúc dữ liệu, hoặc tương tác giữa các thành phần của hệ thống. Do đó, việc nhận diện, phân loại và xử lý ngoại lệ một cách kịp thời và chính xác là yếu tố quyết định hiệu quả phát triển và độ ổn định của ứng dụng. Trong bối cảnh này, các công cụ hỗ trợ lập trình hiện đại, điển hình như Cursor, đóng vai trò then chốt trong việc phân tích ngữ cảnh mã nguồn, gợi ý sửa lỗi, và tự động hóa một phần quá trình khắc phục vấn đề.

Việc phân loại lỗi theo đặc điểm và nguyên nhân giúp tối ưu hóa quá trình xử lý và bảo trì hệ thống. Công cụ hỗ trợ ngữ cảnh mã nguồn như Cursor có khả năng phân tích toàn bộ dự án để nhận diện lỗi, xác định vị trí phát sinh và đề xuất giải pháp sửa chữa phù hợp. Dưới đây là các nhóm lỗi phổ biến trong kiến trúc ứng dụng hiện đại, cùng với cơ chế xử lý tương ứng.

3.4.1 Lỗi liên quan đến giao tiếp mạng và bảo mật (CORS)

Lỗi CORS xảy ra khi trình duyệt chặn các yêu cầu HTTP từ một ứng dụng web (Client Origin) tới một tài nguyên tại nguồn gốc khác (Server Origin) do vi phạm chính sách bảo mật. Thông thường, lỗi được nhận diện thông qua thông báo trên console của trình duyệt và không luôn đi kèm mã lỗi HTTP cụ thể.

Để xử lý vấn đề này, lập trình viên cần can thiệp phía máy chủ, bổ sung các header HTTP cần thiết như `Access-Control-Allow-Origin` và `Access-Control-Allow-Methods`. Công cụ trợ lý lập trình Cursor có khả năng phân tích toàn bộ mã nguồn, xác định các thư mục cấu hình hoặc middleware liên quan (như `server.js` hoặc cấu hình Reverse Proxy), và tự động chèn hoặc điều chỉnh các header cần thiết, từ đó giải quyết triệt để vấn đề bảo mật nguồn gốc mà không làm gián đoạn quá trình phát triển.

3.4.2 Lỗi cấu hình và định tuyến API

Lỗi cấu hình và định tuyến API phát sinh khi đường dẫn API được gọi trong client (thường nằm ở Service Layer) không khớp với các định tuyến đã được thiết lập trên server (Controller/Router), dẫn đến lỗi HTTP 404 Not Found.

Để khắc phục, Cursor quét toàn bộ mã nguồn, đối chiếu các URL trong Service Layer với các decorator định tuyến của backend, và tự động gợi ý sửa đổi các URL sai

bằng đường dẫn chính xác. Quá trình này giúp đảm bảo đồng bộ giữa giao diện người dùng và logic nghiệp vụ, đồng thời giảm thiểu lỗi phát sinh do sự không nhất quán giữa client và server.

3.4.3 Lỗi mô hình dữ liệu

Lỗi mô hình dữ liệu xảy ra khi cấu trúc dữ liệu truyền đi (Request) hoặc nhận về (Response) không tương thích với lớp DTO (Data Transfer Object) hoặc Entity tương ứng, gây ra lỗi deserialization, type error hoặc mất dữ liệu trong quá trình xử lý logic nghiệp vụ. Cursor tiến hành phân tích luồng dữ liệu (Data Flow Analysis) để phát hiện các trường dữ liệu thiếu hoặc không đồng bộ, từ đó đề xuất cập nhật DTO. Sau khi lập trình viên xác nhận các trường dữ liệu mới theo ca sử dụng, Cursor có thể tự động thêm, xóa hoặc điều chỉnh kiểu dữ liệu, đảm bảo đồng bộ hóa hoàn toàn mô hình dữ liệu giữa client và server.

3.4.4 Lỗi xung đột mã nguồn và cấu trúc

Lỗi xung đột mã nguồn và cấu trúc xuất hiện khi việc sửa lỗi hoặc phát triển tính năng mới tạo ra mâu thuẫn với cấu trúc hoặc logic hiện có, ví dụ như cố gắng tạo một file đã tồn tại. Để duy trì tính toàn vẹn dữ liệu, Cursor áp dụng cơ chế xác nhận trước khi ghi đè file, đồng thời phân tích sự khác biệt giữa các hàm và gợi ý cách merge code an toàn. Cơ chế này giúp tránh mất dữ liệu, duy trì đồng bộ trong dự án và hỗ trợ lập trình viên xử lý các xung đột logic phức tạp một cách hiệu quả.

3.4.5 Lỗi sai lệch giữa giao diện và logic

Lỗi sai lệch giữa giao diện và logic xuất hiện khi hành vi của giao diện người dùng không phản ánh đúng trạng thái hoặc kết quả của logic nghiệp vụ đã thực thi, ví dụ dữ liệu đã được lưu trên backend nhưng frontend không cập nhật hoặc hiển thị sai trạng thái.

Để xử lý, lập trình viên cung cấp screenshot minh họa và Cursor đối chiếu output thực tế với output mong muốn. Công cụ phân tích các thành phần quản lý trạng thái (State Management) và các hàm callback hoặc event handler, từ đó xác định và gợi ý sửa

chữa các vấn đề như thiếu lệnh cập nhật trạng thái, điều kiện render sai hoặc lỗi trong quá trình data binding giữa logic và component.

Chương 4

Thực nghiệm

Chương này trình bày toàn bộ quá trình thực nghiệm của đề tài, bao gồm mô tả hệ thống và mã nguồn nền tảng ban đầu, liệt kê và phân tích chi tiết các đặc tả ca sử dụng đầu vào, kết quả sinh mã từ các coding prompt, các tình huống ngoại lệ tương ứng, cùng với việc minh họa phần mềm đã được tạo ra thông qua các giao diện và cấu trúc tệp sinh tự động.

4.1 Mô tả hệ thống

Hệ thống Financial Management là một ứng dụng quản lý tài chính cá nhân toàn diện, được thiết kế nhằm hỗ trợ người dùng theo dõi dòng tiền, quản lý tài sản, phân tích chi tiêu và đặt mục tiêu tài chính dài hạn. Hệ thống kết hợp dữ liệu giao dịch, thông tin tài khoản và các biểu đồ phân tích để cung cấp cái nhìn tổng thể về tình hình tài chính của cá nhân. Mục tiêu chính là giúp người dùng chủ động kiểm soát tài chính, tối ưu hóa thói quen chi tiêu và đạt được các mục tiêu tiết kiệm đã đề ra.

Cấu trúc chức năng của hệ thống được tổ chức thành bốn module cốt lõi quản lý tài khoản và sổ dư, quản lý giao dịch, phân tích chi tiêu và hóa đơn và quản lý mục tiêu. Mỗi module tương ứng với một nhóm ca sử dụng phục vụ trực tiếp nhu cầu theo dõi và quản lý tài chính của người dùng.

4.1.1 Danh sách ca sử dụng

Hệ thống bao gồm các ca sử dụng chính:

- **Đăng ký tài khoản:** Người dùng tạo tài khoản mới bằng cách cung cấp các thông tin xác thực cơ bản như email, mật khẩu và các dữ liệu nhận dạng cần thiết.
- **Đăng nhập:** Ca sử dụng cho phép người dùng truy cập vào hệ thống bằng cách nhập thông tin xác thực (email và mật khẩu). Hệ thống kiểm tra tính hợp lệ của thông tin đăng nhập và cấp quyền truy cập nếu thông tin chính xác.
- **Xem lịch sử giao dịch:** Người dùng có thể xem tất cả các giao dịch, bao gồm tên giao dịch, ngày tháng thực hiện, số tiền giao dịch, loại giao dịch.
- **Tạo giao dịch mới:** Ca sử dụng cho phép người dùng ghi nhận một giao dịch mới. Khi tạo, người dùng nhập số tiền, loại giao dịch và tài khoản. Hệ thống tự động cập nhật số dư tài khoản tương ứng.
- **Xem danh sách tài khoản ngân hàng:** Người dùng có thể xem tất cả tài khoản đang liên kết với hệ thống, bao gồm tên ngân hàng, số tài khoản, loại tài khoản và số dư hiện tại.
- **Thêm tài khoản ngân hàng:** Người dùng thêm mới một tài khoản bằng cách cung cấp thông tin cần thiết như tên tài khoản, ngân hàng và số dư ban đầu. Hệ thống lưu lại và cập nhật vào danh sách tài khoản.
- **Chỉnh sửa tài khoản:** Ca sử dụng cho phép người dùng thay đổi thông tin của một tài khoản đã tồn tại, chẳng hạn như tên hiển thị hoặc số dư ban đầu. Hệ thống ghi nhận và cập nhật dữ liệu mới.
- **Xoá tài khoản ngân hàng:** Người dùng xoá tài khoản không còn sử dụng. Hệ thống yêu cầu xác nhận trước khi thực hiện và cập nhật danh sách tài khoản sau khi xoá.
- **Xem chi tiết tài khoản ngân hàng:** Ca sử dụng này cung cấp thông tin chi tiết của một tài khoản cụ thể, bao gồm mô tả tài khoản và danh sách một số giao dịch gần nhất để người dùng theo dõi biến động.

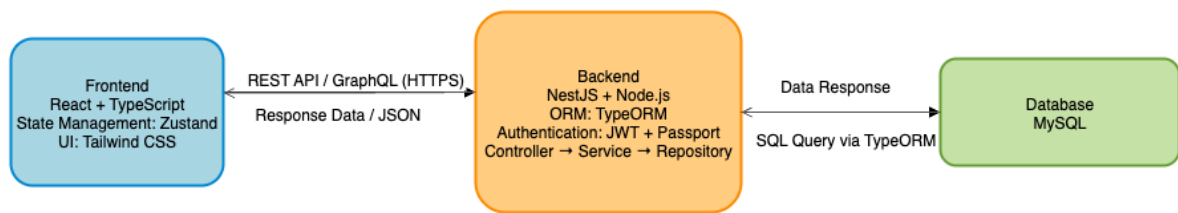
- Xem Chi tiêu Hàng tháng: Người dùng xem biểu đồ tổng chi tiêu của tháng hiện tại và các tháng trước. Biểu đồ giúp đánh giá xu hướng chi tiêu theo thời gian.
- Xem Chi tiết Chi tiêu theo Danh mục: Ca sử dụng hiển thị tổng chi tiêu được nhóm theo danh mục như ăn uống, nhà ở, vận chuyển, v.v. Người dùng dễ dàng thấy danh mục nào chiếm tỷ trọng lớn nhất trong tháng.
- Xem Danh sách Hóa đơn: Người dùng xem danh sách các hóa đơn định kỳ sắp đến hạn. Mỗi hóa đơn bao gồm tên, số tiền và ngày đáo hạn để giúp người dùng chủ động thanh toán.
- Xem Mục tiêu hàng tháng: Hệ thống hiển thị danh sách các mục tiêu tài chính hiện có, bao gồm mục tiêu tiết kiệm hoặc giới hạn chi tiêu theo danh mục. Người dùng theo dõi trạng thái hoàn thành của từng mục tiêu.
- Tạo Mục tiêu mới: Người dùng tạo mục tiêu tài chính mới bằng cách nhập giá trị mục tiêu và danh mục áp dụng. Hệ thống lưu trữ và đưa vào danh sách mục tiêu đang theo dõi.
- Điều chỉnh Mục tiêu hàng tháng: Ca sử dụng cho phép người dùng thay đổi giá trị mục tiêu, cập nhật số tiền đã thực hiện hoặc chỉnh sửa phạm vi áp dụng. Hệ thống cập nhật và phản ánh tiến độ mới.
- Xem Biểu đồ Tổng quan Tiết kiệm: Người dùng xem biểu đồ so sánh khoản tiết kiệm ròng theo từng tháng trong năm hiện tại và năm trước. Biểu đồ giúp đánh giá mức độ cải thiện tài chính theo thời gian

4.1.2 Mô tả mã nguồn nền tảng ban đầu

Mã nguồn nền tảng ban đầu bao gồm:

Các thư viện và công nghệ được sử dụng

Dự án áp dụng một bộ công nghệ hiện đại và đồng bộ, tập trung vào hệ sinh thái TypeScript/Node.js nhằm tối ưu hóa hiệu suất và khả năng bảo trì mã nguồn.



Hình 4.1: Mô tả sơ đồ kiến trúc tổng thể Backend - Frontend - Database của hệ thống.

Phía Backend được xây dựng bằng NestJS trên nền tảng Node.js. NestJS cung cấp một framework mạnh mẽ, có cấu trúc chặt chẽ, hỗ trợ mô hình Module/Controller/Service và áp dụng các mẫu thiết kế của lập trình hướng đối tượng (OOP). Việc này giúp kiến trúc backend trở nên dễ mở rộng và dễ kiểm soát, tương tự như các framework truyền thống như Spring Boot. Để tương tác với cơ sở dữ liệu, TypeORM được sử dụng kết hợp với MySQL, đóng vai trò là Lớp Ánh Xạ Quan Hệ Đối Tượng (ORM), cho phép định nghĩa các Entity để thao tác với CSDL một cách an toàn và tường minh. Việc xác thực được thực hiện thông qua JWT (JSON Web Token), được tích hợp qua module `@nestjs/jwt` và chiến lược Passport.

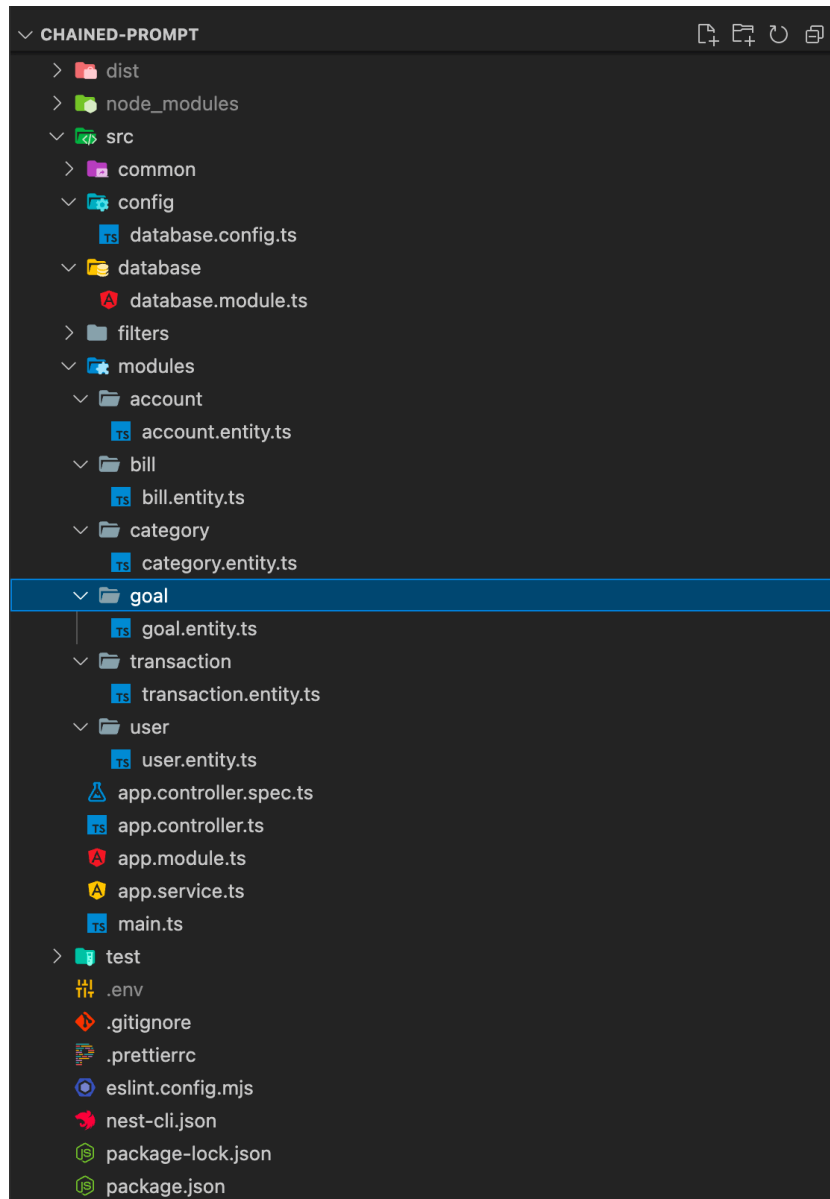
Phía Frontend được phát triển bằng thư viện React (sử dụng TypeScript), tuân thủ mô hình Component-Based, giúp tái sử dụng giao diện và quản lý UI hiệu quả. Vite được chọn làm công cụ xây dựng (bundler) nhờ tốc độ phát triển nhanh và khả năng tối ưu hóa. Việc quản lý trạng thái phức tạp được giao cho thư viện Zustand, nổi bật với tính tối giản, dễ hiểu và hiệu suất cao. Giao diện người dùng được tạo kiểu bằng Tailwind CSS, một framework CSS theo hướng tiện ích (utility-first), cho phép nhà phát triển xây dựng các thiết kế tùy chỉnh nhanh chóng mà không cần viết CSS tùy chỉnh mở rộng.

Cấu trúc thư mục và kiến trúc hệ thống

Cấu trúc thư mục của hệ thống được phân chia rõ ràng thành hai thư mục gốc, thể hiện sự tách biệt vật lý giữa hai ứng dụng chạy độc lập:

- **/be**: Chứa toàn bộ mã nguồn **Backend** (NestJS).
- **/fe**: Chứa toàn bộ mã nguồn **Frontend** (React + Vite).

Cấu Trúc Chi Tiết Backend (NestJS): Backend NestJS tuân thủ mô hình module hóa và tổ chức mã nguồn theo kiến trúc *feature-based*. Các thành phần chính trong thư mục **/be/src** được bố trí như sau:



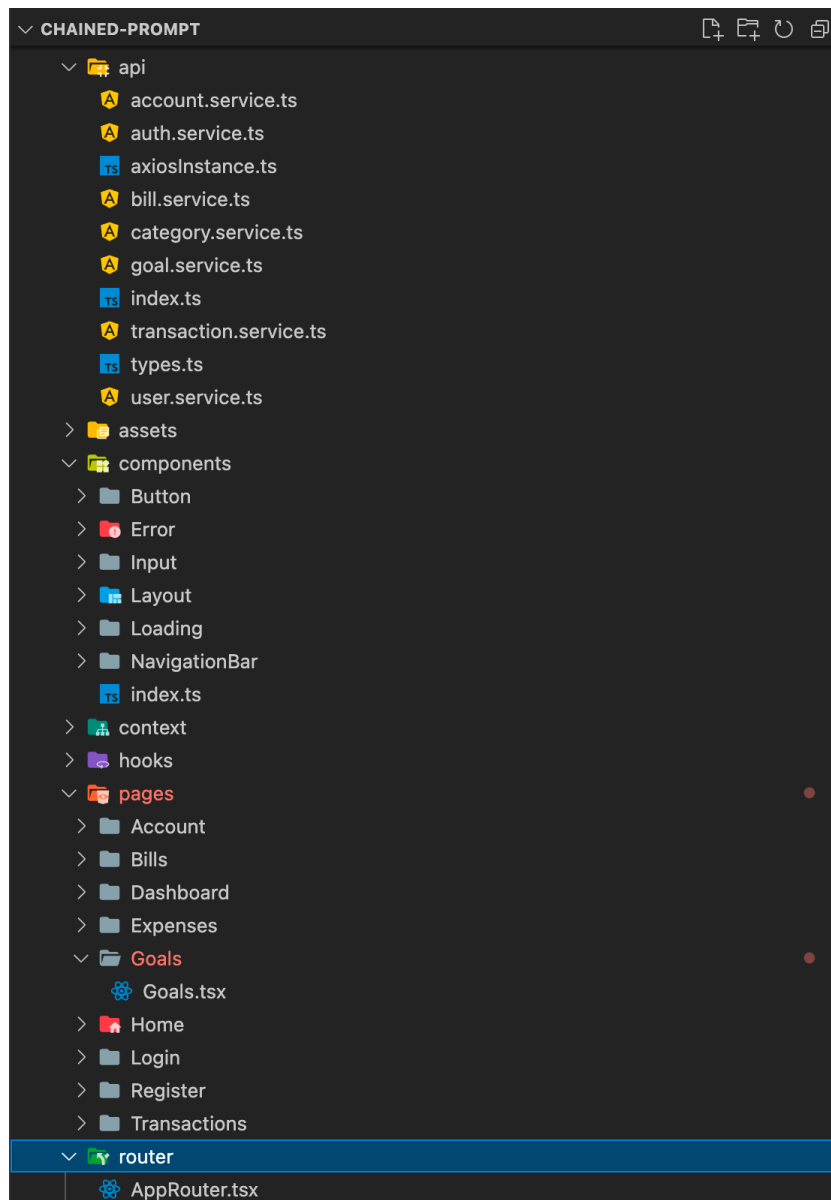
Hình 4.2: Mô tả cấu trúc thư mục Backend của dự án.

- **main.ts:** Tập khởi động ứng dụng; cấu hình CORS, Swagger và Global Validation Pipe.
- **app.module.ts:** Module gốc tổng hợp các module nghiệp vụ; cần import thêm DatabaseModule.
- **config/:** Chứa các cấu hình không liên quan đến nghiệp vụ.

- **database.config.ts**: Cấu hình kết nối MySQL từ biến môi trường.
- **database/**: Chứa các thành phần kết nối ORM.
 - **database.module.ts**: Cấu hình TypeORM kết nối MySQL, `synchronize: false`.
- **common/**: Chứa Guard, Interceptor, Pipe dùng chung; nơi triển khai JWT Strategy và Validation Pipes.
- **filters/**: Chứa formatter và bộ lọc lỗi toàn cục.
 - **http-exception.filter.ts**: Bộ lọc lỗi HTTP (cần được hoàn thiện thêm).
- **modules/**: Chứa toàn bộ module nghiệp vụ như **user**, **account**, **transaction....** Mỗi module gồm: *Controller, Service, Entity, DTO*.
- **entities/**: Định nghĩa các thực thể dùng chung. Bao gồm các lớp *User, Account, Transaction, Bill, Category, Goal*.

Cấu Trúc Chi Tiết Frontend (React + Vite) : Frontend React được thiết kế theo kiến trúc component kết hợp quản lý trạng thái tập trung. Các thư mục chính trong **/fe/src** gồm:

- **api/**: Lớp giao tiếp với Backend API.
 - **axiosInstance**: Đã cấu hình Base URL, Request Interceptor (tự động đính kèm JWT) và Response Interceptor (xử lý 401/logout).
- **components/**: Các thành phần UI tái sử dụng như Button, Input, Layout, Loading, Error, NavigationBar.
- **pages/**: Các trang chính của ứng dụng. Bao gồm: Home, Login, Register, Dashboard, và các trang nghiệp vụ. Áp dụng Lazy Loading.
- **context/**: Quản lý trạng thái toàn cục qua React Context.
 - **AuthContext**: Quản lý trạng thái đăng nhập và token.
 - **ThemeContext**: Quản lý chủ đề giao diện.



Hình 4.3: Mô tả cấu trúc thư mục Frontend của dự án.

- **router/**: Định nghĩa hệ thống định tuyến.
 - **ProtectedRoute**: Bảo vệ các route yêu cầu đăng nhập.
- **hooks/**: Chứa các custom hooks như `useDebounce`, `useLocalStorage`.
- **types/**: Định nghĩa các giao diện TypeScript như `User`, `Account`, `Transaction`, và `ApiResponse<T>`.

Các thành phần cố định

Để đảm bảo tính nhất quán, khả năng bảo mật và hiệu suất hoạt động, dự án đã thiết lập các thành phần cố định quan trọng ngay từ giai đoạn khởi tạo:

Phía Backend (NestJS) :các cấu hình nền tảng sau được thiết lập trong hàm khởi động chính (`main.ts`) và các file cấu hình tương ứng:

- **Database Configuration**: Kết nối TypeORM với MySQL được thiết lập, sử dụng biến môi trường để trích xuất thông tin đăng nhập. Cấu hình quan trọng là `synchronize: false` nhằm đảm bảo tính ổn định và an toàn của sơ đồ dữ liệu đã có sẵn.
- **Swagger API Documentation**: Tài liệu hóa API được tự động sinh ra và phục vụ tại endpoint `/api/docs`. Cấu hình này đã tích hợp Bearer Auth (JWT), cho phép kiểm thử các API cần xác thực trực tiếp trên giao diện Swagger.
- **CORS (Cross-Origin Resource Sharing)**: Được cấu hình mở rộng để cho phép truy cập API từ nhiều nguồn (client frontend) bằng cách sử dụng các giá trị từ biến môi trường hoặc mặc định. Nó hỗ trợ đầy đủ các phương thức HTTP cần thiết (GET, POST, PUT, DELETE, PATCH) và cho phép gửi thông tin xác thực (credentials).
- **Entities**: Các thực thể cốt lõi của hệ thống quản lý tài chính đã được định nghĩa (`User`, `Account`, `Transaction`, `Bill`, `Category`, `Goal`), đóng vai trò là giao diện giữa logic nghiệp vụ và cơ sở dữ liệu.
- **Exception Filter Structure**: File `HttpExceptionFilter` đã được tạo, đóng vai trò là cơ chế xử lý lỗi tập trung toàn cục (Global Exception Handling), đảm bảo mọi

phản hồi lỗi gửi về client đều tuân thủ một định dạng JSON nhất quán ("success": false, "message": "...").

Phía Frontend (React + Vite): Các thành phần cố định trong Frontend tập trung vào việc chuẩn hóa giao tiếp API và quản lý trạng thái xác thực:

- **Axios Instance:** Một instance tùy chỉnh của Axios đã được thiết lập. Nó bao gồm:
 - **Base URL** được đặt cố định tới /api thông qua Proxy của Vite, giúp đồng nhất đường dẫn API.
 - **Request Interceptor:** Tự động chèn JWT Token (lấy từ trạng thái xác thực) vào Header Authorization: Bearer <token> của mọi yêu cầu đi.
 - **Response Interceptor:** Xử lý phản hồi lỗi HTTP 401 (Unauthorized) một cách tự động, kích hoạt quy trình đăng xuất và chuyển hướng người dùng về trang đăng nhập.
- **Routing System:** Hệ thống định tuyến (React Router DOM) được phân chia rõ ràng thành Public Routes và Protected Routes. Component ProtectedRoute được sử dụng để bảo vệ các trang nghiệp vụ, dựa vào trạng thái xác thực từ *AuthContext*.
- **Context APIs:**
 - *AuthContext*: Là trung tâm quản lý trạng thái xác thực của người dùng, bao gồm trạng thái đăng nhập và lưu trữ/cung cấp JWT Token.
 - *ThemeContext*: Quản lý chủ đề giao diện người dùng, đảm bảo trải nghiệm người dùng nhất quán.
- **API Services Structure:** Cấu trúc các file service API (*auth.service.ts*, *user.service.ts*, v.v.) đã được tạo sẵn, đóng vai trò là lớp trừu tượng (abstraction layer) giữa giao diện người dùng và các endpoints của Backend.

Các quy tắc sinh mã của dự án

Dự án áp dụng quy ước đặt tên chuẩn và tiêu chuẩn hóa quy trình làm việc Git. Đặc biệt, mọi giao tiếp API đều tuân thủ định dạng phản hồi JSON chuẩn. Mọi logic nghiệp

vụ trong quá trình sinh mã tự động phải tuân thủ nguyên tắc Không để logic nghiệp vụ trong Controller/Component và Tạo đầy đủ 5 thành phần (*controller, service, entity, dto, module*) cho mỗi module NestJS mới.

Kiến trúc Backend tuân thủ nghiêm ngặt mô hình module hóa và phân tách nhiệm vụ :

- Logic nghiệp vụ phải chỉ nằm trong Service. Controller chỉ xử lý request và gọi Service.
- Sử dụng Repository pattern của TypeORM để truy cập CSDL.
- DTOs được sử dụng kết hợp với class-validator để kiểm tra tính hợp lệ của dữ liệu.

Frontend áp dụng kiến trúc Component-Based với các quy tắc chính: Sử dụng Functional Component và Hooks để phát triển., giao diện được tạo kiểu bằng Tailwind CSS, quản lý trạng thái bằng Context API (Auth, Theme) hoặc Zustand (dữ liệu nghiệp vụ), giao tiếp API luôn thông qua Axios Instance trong thư mục `src/api/` để tận dụng Interceptor.

4.2 Đặc tả ca sử dụng chi tiết

Trong phạm vi mục này, luận văn chỉ trình bày một số ca sử dụng tiêu biểu nhằm minh họa rõ ràng cách thức áp dụng phương pháp đã đề xuất. Những ca sử dụng còn lại, do trùng lặp về cấu trúc hoặc không ảnh hưởng đáng kể đến việc hiểu quy trình tổng thể, sẽ được đưa vào Phụ lục để tránh làm tài liệu trở nên quá dài dòng.

4.2.1 Đăng nhập

Đặc tả ca sử dụng

Bảng 4.1 trình bày đặc tả ca sử dụng Đăng nhập, cho phép người dùng truy cập hệ thống bằng thông tin xác thực hợp lệ.

Bảng 4.1: Đặc tả ca sử dụng Đăng nhập hệ thống

Tên ca sử dụng	Đăng nhập
Mô tả	• Cho phép Người dùng xác thực danh tính bằng Email và Mật khẩu để truy cập vào hệ thống. • Hỗ trợ tùy chọn “Lưu thông tin đăng nhập”.
Tác nhân	Người dùng (User)
Luồng chính	• Người dùng truy cập nút Đăng nhập trên Trang chủ. • Hệ thống chuyển đến trang Đăng nhập. • Người dùng nhập Email và Mật khẩu. • Nhấn nút “Đăng nhập”. • Hệ thống kiểm tra tính hợp lệ. • Nếu hợp lệ: thiết lập phiên đăng nhập. • Chuyển hướng đến Trang chủ và hiển thị “Đăng xuất”.
Luồng ngoại lệ	• Nếu Email hoặc Mật khẩu không đúng: hiển thị lỗi “Email hoặc mật khẩu không đúng.” • Người dùng vẫn ở màn hình Đăng nhập.
Luồng thay thế	• Người dùng chọn “Lưu thông tin đăng nhập”. • Nếu thành công, hệ thống ghi nhớ phiên đăng nhập.
Tiền điều kiện	• Người dùng đang ở Trang chủ. • Đã có tài khoản hợp lệ.
Hậu điều kiện	• Thành công: Đăng nhập và chuyển về Trang chủ. • Thất bại: Hiển thị lỗi.

Tích hợp UI Figma:

<https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits--Community-?node-id=137-7477&t=cxgXQfkGoLMfxW48-4>

Endpoint:

POST /api/auth/login

Request Body:

```
{ "email": "johndoe@email.com",  
  "password": "mypassword" }
```

Response khi thành công:

```
{"accessToken": "JWT_TOKEN",  
  "user": {"id": 1,  
            "fullName": "John Doe",  
            "email": "johndoe@email.com"} }
```

Response lỗi:

```
{ "error": "Email hoặc mật khẩu không đúng." }
```

Kết quả Coding Prompt Pha 1.

1. Prompt A: Sinh Mã Backend (NestJS Service/Controller)

Mục tiêu: Xây dựng dịch vụ API, logic nghiệp vụ và xử lý lỗi server cho chức năng đăng nhập.

Tạo endpoint POST `api/auth/login` trong `AuthController` của NestJS. Endpoint này là *public route*, không yêu cầu Guard xác thực.

Định dạng Request:

```
{ "email": "string",  
  "password": "string" }
```

Logic xử lý trong AuthService:

1. Nhận email và password từ request body.
2. Tìm người dùng trong cơ sở dữ liệu bằng email.
3. Nếu không tìm thấy hoặc mật khẩu không khớp, xử lý thất bại.

4. Nếu đúng, tạo JSON Web Token (JWT) chứa thông tin định danh người dùng.
5. Trả về token và thông tin cơ bản của người dùng.

Định dạng Response thành công:

```
{ "accessToken": "JWT_TOKEN",  
  "user": {"id": 1,  
            "fullName": "John Doe",  
            "email": "johndoe@email.com"}}
```

Xử lý lỗi: Nếu email không tồn tại hoặc mật khẩu không đúng, hệ thống ném UnauthorizedException với JSON lỗi:

```
{ "error": "Email hoặc mật khẩu không đúng." }
```

2. Prompt B: Sinh Mã Frontend (React Component & UI/MCP)

Mục tiêu: Xây dựng giao diện đăng nhập theo đúng thiết kế Figma.

Tạo component LoginForm bằng React TypeScript và Tailwind CSS. Form bao gồm: Trường nhập Email, Trường nhập Mật khẩu, Checkbox “Lưu thông tin đăng nhập”, Nút “Đăng nhập”, Liên kết “Quên mật khẩu?”.

Yêu cầu bám sát hoàn toàn thiết kế Figma MCP tại liên kết sau:

<https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits--Community-?node-id=137-7477&t=Jwl8SfgcguZ5Gicc-4>

3. Prompt C: Sinh Mã Frontend Logic (State & Kết nối API)

Mục tiêu: Kết nối form UI với API và xử lý luồng đăng nhập thành công.

Thêm các state: email: string, password: string, rememberMe: boolean

Triển khai hàm bất đồng bộ handleSubmit để gửi request POST đến api/auth/login.

Payload gửi lên:

```
{"email": "johndoe@email.com",  
  "password": "mypassword"}
```

Khi đăng nhập thành công:

1. Lưu accessToken vào localStorage.
2. Lưu thông tin user vào global state (Context API / Redux / Zustand).
3. Điều hướng người dùng về trang chủ /.

4. Prompt D: Xử lý Ngoại lệ (Client-side Validation & Error Handling)

Mục tiêu: Hoàn thiện xử lý lỗi, loading và validation phía client.

Nâng cấp hàm handleSubmit với các chức năng:

1. **Trạng thái Loading:** Thêm state isLoading. Khi đang xử lý, vô hiệu hóa nút Đăng nhập và hiển thị spinner.
2. **Xử lý lỗi API:** Nếu API trả lỗi 401, hiển thị thông báo: *“Email hoặc mật khẩu không đúng.”*
3. **Validation FE:**
 - Email không được để trống và phải đúng định dạng.
 - Mật khẩu không được để trống.
 - Nếu lỗi, hiển thị thông báo ngay dưới ô input.

4.2.2 Đăng kí

Đặc tả ca sử dụng

Bảng 4.2 mô tả chi tiết ca sử dụng Đăng ký tài khoản, giúp người dùng tạo mới tài khoản bằng cách cung cấp thông tin xác thực.

Bảng 4.2: Đặc tả ca sử dụng Đăng ký tài khoản

Tên ca sử dụng	Đăng ký tài khoản
Mô tả	Cho phép Người dùng mới cung cấp thông tin (Họ tên, Email, Mật khẩu) để tạo tài khoản mới trong hệ thống.
Tác nhân	Người dùng (User)
Tiền điều kiện	- Người dùng đang truy cập vào màn hình Đăng ký. - Người dùng chưa có tài khoản trong hệ thống.
Hậu điều kiện	- Thành công: Tài khoản được tạo, thiết lập phiên đăng nhập và chuyển hướng đến Trang chủ (/). - Thất bại: Hiển thị lỗi và Người dùng vẫn ở màn hình Đăng ký.
Luồng chính	- Người dùng truy cập Trang chủ và nhấn nút Đăng ký/Đăng nhập. - Hệ thống chuyển đến trang Đăng ký. - Người dùng nhập Họ tên, Email, Mật khẩu và Xác nhận Mật khẩu. - Nhấn nút “Sign Up”. - Hệ thống kiểm tra tính hợp lệ của dữ liệu nhập. - Hệ thống kiểm tra Email có tồn tại chưa. - Hệ thống kiểm tra Mật khẩu và Xác nhận Mật khẩu. - Nếu hợp lệ: tạo tài khoản mới. - Thiết lập phiên đăng nhập cho người dùng mới. - Chuyển hướng đến Trang chủ (/).
Luồng thay thế	A.1 – Email đã tồn tại - Sau bước 6 Luồng chính. - Nếu Email đã đăng ký: hiển thị lỗi “This email is already registered.” - Người dùng vẫn ở màn hình Đăng ký. A.2 – Mật khẩu không trùng khớp - Sau bước 7 Luồng chính. - Nếu không khớp: hiển thị lỗi “Passwords do not match.” - Người dùng vẫn ở màn hình Đăng ký.

A.3 – Dữ liệu không hợp lệ

- Nếu Email sai định dạng hoặc trường bị bỏ trống.
- Hệ thống hiển thị lỗi ngay dưới trường nhập liệu.
- Người dùng vẫn ở màn hình Đăng ký.

Tích hợp UI Figma:

<https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits--Community-?node-id=137-8071&t=cxgXQfkGoLMfxW48-4>

Endpoint:

POST /api/auth/register

Request Body:

```
{ "fullName": "Nguyen Van A",  
  "email": "example@email.com",  
  "password": "123456",  
  "confirmPassword": "123456" }
```

Response khi thành công:

```
{ "message": "Registration successful",  
  "user": { "id": 1,  
            "fullName": "Nguyen Van A",  
            "email": "example@email.com" },  
  "token": "JWT_TOKEN" }
```

Response lỗi:

```
{ "error": "This email is already registered." }  
{ "error": "Passwords do not match." }
```

Kết quả Coding Prompt Pha 1.

1. Prompt A: Sinh Mã Backend (NestJS Service/Controller)

Mục tiêu: Xây dựng dịch vụ API, logic nghiệp vụ và xử lý lỗi server cho chức năng đăng ký tài khoản.

Tạo endpoint POST `api/auth/register` trong `AuthController` của NestJS. Endpoint này là *public route*, không yêu cầu Guard.

Định dạng Request:

```
{ "fullName": "Nguyen Van A",  
  "email": "example@email.com",  
  "password": "123456",  
  "confirmPassword": "123456" }
```

Logic xử lý trong AuthService:

1. Sử dụng DTO và class-validator để kiểm tra các trường: `fullName`, `email`, `password`.
2. Kiểm tra `password` có trùng với `confirmPassword`.
3. Kiểm tra email đã tồn tại trong cơ sở dữ liệu hay chưa.
4. Nếu hợp lệ, băm password và tạo user mới.
5. Tạo JWT token cho người dùng mới.
6. Trả về thông tin người dùng và token.

Định dạng Response thành công:

```
{ "message": "Registration successful",  
  "user": { "id": 1,  
            "fullName": "Nguyen Van A",  
            "email": "example@email.com" },  
  "token": "JWT_TOKEN" }
```

Xử lý lỗi:

```
{ "error": "This email is already registered." }  
{ "error": "Passwords do not match." }
```

2. Prompt B: Sinh Mã Frontend (React Component & UI/MCP)

Mục tiêu: Xây dựng giao diện đăng ký theo đúng thiết kế Figma.

Tạo component `SignUpForm` bằng React TypeScript và Tailwind CSS. Form cần có: Email, Full Name, Password, Confirm Password, Nút *Sign Up* và liên kết chuyển đến trang đăng nhập.

Thiết kế phải bám sát Figma MCP tại liên kết:

<https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits--Community-?node-id=137-8071&t=cxgXQfkGoLMfxW48-4>

3. Prompt C: Sinh Mã Frontend Logic (State & Kết nối API)

Mục tiêu: Kết nối form UI với API và xử lý quá trình đăng ký thành công.

Thêm các state: `fullName`, `email`, `password`, `confirmPassword`.

Triển khai hàm `handleSignUp` để gửi request POST đến `api/auth/register`.

Payload gửi lên:

```
{"fullName": "...",  
  "email": "...",  
  "password": "...",  
  "confirmPassword": "..."} }
```

Khi đăng ký thành công:

1. Lưu token vào `LocalStorage` hoặc `Context`.
2. Lưu thông tin user vào global state.
3. Điều hướng người dùng về trang chủ `/`.

4. Prompt D: Xử lý Ngoại lệ (Client-side Validation & Error Handling)

Mục tiêu: Hoàn thiện xử lý lỗi, loading và validation phía client.

1. **Loading:** Thêm state `isLoading`. Khi đang xử lý, vô hiệu hóa nút Sign Up và hiển thị spinner.
2. **Xử lý lỗi API:** Hiển thị lỗi từ response như: *"This email is already registered."*
"Passwords do not match."
3. **Validation FE:**
 - Kiểm tra tất cả các trường không được để trống.
 - Kiểm tra mật khẩu và xác nhận mật khẩu có trùng khớp.
 - Nếu lỗi, hiển thị thông báo ngay dưới input.

4.2.3 Xem danh sách giao dịch

Đặc tả ca sử dụng

Bảng 4.3 mô tả ca sử dụng Xem danh sách giao dịch, cho phép người dùng theo dõi toàn bộ giao dịch đã thực hiện.

Bảng 4.3: Đặc tả ca sử dụng Xem danh sách giao dịch

Tên ca sử dụng	Xem danh sách giao dịch
Mô tả	• Cho phép người dùng xem danh sách các giao dịch đã thực hiện (Thu nhập và Chi tiêu). • Hiển thị các thông tin: tên mặt hàng, cửa hàng, ngày giao dịch, phương thức thanh toán, số tiền.
Tác nhân	Người dùng (User)
Luồng chính	• Người dùng truy cập trang Transactions qua sidebar (/transactions). • Hệ thống hiển thị tab All mặc định. • Front-end gọi API. • Backend xác thực <code>access_token</code> và lấy <code>user_id</code>.

	• Backend truy vấn giao dịch theo user_id, sắp xếp giảm dần theo ngày, áp dụng phân trang. • Backend trả về danh sách giao dịch. • Front-end hiển thị danh sách với các cột: Items, Shop Name, Date, Payment Method, Amount.
Luồng ngoại lệ	• Lỗi xác thực hoặc truy vấn: Backend trả về lỗi (401, 500). • Front-end hiển thị thông báo lỗi và không tải danh sách.
Luồng thay thế	• Lọc theo Thu nhập/Chi tiêu: Người dùng chọn Revenue hoặc Expenses, hệ thống trả danh sách đã lọc. • Tải thêm dữ liệu: Người dùng cuộn xuống hoặc chọn “Load More”, hệ thống tăng offset và tải thêm giao dịch. • Không có giao dịch: Nếu danh sách rỗng, hiển thị “Bạn chưa có giao dịch nào được ghi nhận.”
Tiền điều kiện	• Người dùng đã đăng nhập. • Người dùng đã có ít nhất một giao dịch.
Hậu điều kiện	• Thành công: Danh sách giao dịch được hiển thị (có thể được lọc hoặc phân trang). • Thất bại: Hiển thị thông báo lỗi hoặc không có dữ liệu.

Tích hợp UI Figma:

<https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits--Community-?node-id=66-5474>

Endpoint:

GET /api/v1/transactions

Query Parameters:

- type: All, Revenue, hoặc Expense
- limit: Số lượng bản ghi mỗi lần lấy (mặc định 10)

- offset: Vị trí bắt đầu (mặc định 0, phục vụ phân trang)

Response thành công (200):

```
{
  "data": [
    {
      "transactionId": 1,
      "accountId": 3,
      "transactionDate": "2023-05-17T00:00:00Z",
      "type": "Expense",
      "itemDescription": "GTR 5",
      "shopName": "Gadget & Gear",
      "amount": 160.00,
      "paymentMethod": "Credit Card",
      "status": "Complete",
      "receiptId": null,
      "createdAt": "2023-05-17T08:12:00Z"
    },
    {
      "transactionId": 2,
      "accountId": 3,
      "transactionDate": "2023-05-17T00:00:00Z",
      "type": "Expense",
      "itemDescription": "Polo Shirt",
      "shopName": "XL Fashions",
      "amount": 20.00,
      "paymentMethod": "Credit Card",
      "status": "Complete",
      "receiptId": null,
      "createdAt": "2023-05-17T08:12:00Z"
    }
  ],
  "total": 2,
  "hasMore": false
}
```

Response lỗi (400 / 500):

```
{
  "message": "Invalid type parameter"
}
```

Kết quả Coding Prompt Pha 1.

1. Prompt A: Sinh Mã Backend (NestJS Service/Controller)

Mục tiêu: Xây dựng dịch vụ API, logic nghiệp vụ và xử lý lỗi server cho chức năng lấy danh sách giao dịch.

Tạo endpoint GET `/api/v1/transactions` trong `TransactionsController`. Endpoint này phải được bảo vệ bởi `JwtAuthGuard`.

Định dạng Request:

- `type`: "All""Revenue""Expense"(bắt buộc)
- `limit`: number (mặc định: 10)
- `offset`: number (mặc định: 0)

Logic xử lý trong `TransactionsService`:

1. Lấy `user_id` từ request đã được xác thực.
2. Nhận các query params: `type`, `limit`, `offset`.
3. Kiểm tra `type` phải thuộc: "All", "Revenue", "Expense".
4. Xây dựng truy vấn lấy giao dịch theo `user_id`.
5. Nếu `type` là "Revenue" hoặc "Expense", thêm điều kiện lọc.
6. Sắp xếp theo `transactionDate` giảm dần.
7. Áp dụng phân trang: `limit`, `offset`.
8. Trả về danh sách giao dịch, tổng số bản ghi (`total`), và giá trị `hasMore`.

Định dạng Response thành công:

```
{"data": [  
  {"transactionId": 1,  
    "accountId": 3,
```



```
    "transactionDate": "2023-05-17T00:00:00Z",
    "type": "Expense",
    "itemDescription": "GTR 5",
    "shopName": "Gadget & Gear",
    "amount": 160.00,
    "paymentMethod": "Credit Card",
    "status": "Complete",
    "receiptId": null,
    "createdAt": "2023-05-17T08:12:00Z"}],
"total": 1,
"hasMore": false}
```

Xử lý lỗi:

```
{ "message": "Invalid type parameter" }
```

Prompt B: Sinh Mã Frontend (React Component & UI/MCP)

Mục tiêu: Xây dựng giao diện danh sách giao dịch theo đúng thiết kế Figma.

Tạo component TransactionsPage bằng React TypeScript và Tailwind CSS.

Giao diện phải bao gồm:

- Tiêu đề trang **Transactions**.
- Ba tab: All, Revenue, Expense (tab đang chọn phải có style nổi bật).
- Bảng hoặc danh sách hiển thị các giao dịch với các cột: Items, Shop Name, Date, Payment Method, Amount.
- Icon minh họa từng giao dịch.
- Nút **Load More** (hiển thị khi còn dữ liệu).
- Thông báo rỗng: *"Bạn chưa có giao dịch nào"*.

Thiết kế phải khớp 100% với bản Figma MCP:

<https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits--Community-?node-id=66-5474>

Prompt C: Sinh Mã Frontend Logic (State & Kết nối API)

Mục tiêu: Kết nối UI với API và xử lý luồng nhận dữ liệu giao dịch.

Thêm các state trong TransactionsPage:

- transactions: Transaction[]
- activeTab: 'All' | 'Revenue' | 'Expense'
- pagination: { offset, limit, hasMore }
- isLoading: boolean
- error: string | null

Triển khai hàm fetchTransactions(loadMore = false) gọi API:

- Nếu loadMore = false: offset = 0.
- Nếu loadMore = true: offset = pagination.offset.

Response dạng:

```
{"data": [...],  
  "total": 2,  
  "hasMore": false}
```

Xử lý thành công:

1. Nếu loadMore = true: nối dữ liệu vào mảng cũ.
2. Nếu false: thay thế toàn bộ danh sách.
3. Cập nhật pagination.
4. Gọi API mỗi khi activeTab thay đổi.

Prompt D: Xử lý Ngoại lệ (Client-side Validation & Error Handling)

Mục tiêu: Hoàn thiện xử lý lỗi, loading và hiển thị trạng thái danh sách.

- 1. **Loading:** Khi gọi API, bật `isLoading`. Khi đang tải, hiển thị *Spinner/Skeleton* thay cho danh sách.
- 2. **Lỗi API:** Trong `catch`, đặt `error = "Không thể tải danh sách giao dịch.- Vui lòng thử lại."` Hiển thị lỗi thay cho danh sách.
- 3. **Không có dữ liệu:** Nếu `transactions` rỗng và không loading, hiển thị: *"Bạn chưa có giao dịch nào cho mục activeTab"*.
- 4. **Validation FE:** Không áp dụng vì không có form nhập liệu.

4.2.4 Tạo giao dịch mới

Đặc tả ca sử dụng

Bảng 4.4 mô tả chi tiết ca sử dụng Tạo giao dịch mới, giúp người dùng ghi nhận một giao dịch và cập nhật số dư tài khoản.

Bảng 4.4: Đặc tả ca sử dụng Tạo giao dịch mới

Tên ca sử dụng	Thêm giao dịch mới
Mô tả	• Cho phép người dùng ghi nhận một giao dịch tài chính mới (thu nhập hoặc chi tiêu) bằng cách cung cấp các thông tin cần thiết như chi tiết giao dịch, số tiền, loại giao dịch và danh mục.
Tác nhân	Người dùng (User)
Tiền điều kiện	• Người dùng phải ở trạng thái đã đăng nhập. • Các danh mục (Categories) phải có sẵn để người dùng lựa chọn.
Hậu điều kiện	• Thành công: Giao dịch mới được lưu vào bảng Transactions, thông báo thành công hiển thị, danh sách giao dịch cập nhật.

	• Thất bại: Hệ thống hiển thị thông báo lỗi tương ứng với dữ liệu không hợp lệ, người dùng vẫn ở lại form Thêm giao dịch.
Luồng chính	1. Người dùng truy cập trang Giao dịch và nhấn nút “Thêm giao dịch”. 2. Hệ thống hiển thị form nhập liệu với các trường: Tên giao dịch, Tên mặt hàng, Danh mục, Số tiền, Loại giao dịch (Revenue / Expense), Tài khoản thanh toán, Ngày giao dịch, và các tùy chọn khác (Phương thức thanh toán / Tài khoản liên quan). 3. Người dùng nhập đầy đủ thông tin vào form. 4. Người dùng nhấn nút “Lưu” (Save). 5. Hệ thống Front-end gửi yêu cầu kèm dữ liệu đến API Backend. 6. Hệ thống Backend xác thực access_token và trích xuất user_id. 7. Hệ thống Backend kiểm tra tính hợp lệ của dữ liệu đầu vào (Tên giao dịch không trống, Số tiền > 0, Loại giao dịch hợp lệ, Danh mục tồn tại, Tài khoản thuộc user_id). 8. Nếu hợp lệ, lưu giao dịch mới vào bảng Transactions và cập nhật số dư tài khoản liên quan. 9. Backend trả về phản hồi thành công. 10. Front-end nhận phản hồi và hiển thị thông báo “Thêm giao dịch thành công.” 11. Front-end làm mới danh sách giao dịch hoặc điều hướng trở lại trang /transactions.
Luồng thay thế	A.1 – Dữ liệu không hợp lệ • Sau bước 7 Luồng chính. • Nếu dữ liệu bị thiếu hoặc sai định dạng: hiển thị lỗi tương ứng bên cạnh trường nhập liệu. • Người dùng vẫn ở lại form Thêm giao dịch. A.2 – Tài khoản không hợp lệ • Nếu tài khoản thanh toán không thuộc user_id hoặc số dư không đủ: hiển thị thông báo lỗi.

- | | |
|--|---|
| | <ul style="list-style-type: none">• Người dùng vẫn ở lại form Thêm giao dịch. |
|--|---|

Tích hợp UI Figma:

Form thêm giao dịch có layout giống phong cách tổng thể trang Giao dịch (sử dụng card bo góc, input có border mờ, nút Save màu xanh). Các trường được bố trí dọc, có label rõ ràng. Nút Cancel quay lại trang /transactions.

Endpoint:

POST /api/v1/transactions

Request Body:

```
{
  "accountId": 3,
  "transactionDate": "2023-05-17T00:00:00Z",
  "type": "Expense",
  "itemDescription": "Movie Ticket",
  "category_id": 3,
  "shopName": "Inox",
  "amount": 150000,
  "paymentMethod": "Credit Card",
  "status": "Complete"
}
```

Response thành công (201 Created):

```
{
  "message": "Transaction created successfully",
  "data": {
    "transactionId": 8,
    "accountId": 3,
    "transactionDate": "2023-05-17T00:00:00Z",
    "type": "Expense",
    "itemDescription": "Movie Ticket",
    "shopName": "Inox",
    "amount": 150000,
    "paymentMethod": "Credit Card",
  }
}
```

```
"status": "Complete",
"receiptId": null,
"createdAt": "2023-05-17T08:12:00Z",
"category_id": 3}}
```

Response lỗi (400 / 422):

```
{"message": "Invalid or missing transaction data"}
```

Kết quả Coding Prompt Pha 1.

1. Prompt A: Sinh Mã Backend (NestJS Service/Controller)

Mục tiêu: Xây dựng dịch vụ API, logic nghiệp vụ, và xử lý lỗi server cho chức năng thêm giao dịch.

Tạo endpoint POST /api/v1/transactions trong TransactionsController NestJS. Endpoint này phải được bảo vệ bằng *JwtAuthGuard*.

Định dạng Request:

```
{"accountId": 3,
"transactionDate": "2023-05-17T00:00:00Z",
"type": "Expense",
"itemDescription": "Movie Ticket",
"category_id": 3,
"shopName": "Inox",
"amount": 150000,
"paymentMethod": "Credit Card",
"status": "Complete"}
```

Logic xử lý trong TransactionsService:

1. Nhận dữ liệu giao dịch (DTO) và `user_id` từ *JwtAuthGuard*.
2. Kiểm tra tính hợp lệ của dữ liệu:
 - `itemDescription` không được trống

- amount phải lớn hơn 0
 - type phải là 'Revenue' hoặc 'Expense'
 - category_id phải tồn tại và thuộc sở hữu của người dùng
3. Kiểm tra accountId có thuộc về user_id không.
 4. Kiểm tra số dư tài khoản nếu loại giao dịch là 'Expense'.
 5. Thực hiện transaction trong cơ sở dữ liệu:
 - (a) Lưu giao dịch mới vào bảng Transactions.
 - (b) Cập nhật số dư trong bảng Accounts theo loại giao dịch.
 6. Commit transaction nếu thành công.

Định dạng Response thành công (201 Created):

```
{
  "message": "Transaction created successfully",
  "data": {
    "transactionId": 8,
    "accountId": 3,
    "transactionDate": "2023-05-17T00:00:00Z",
    "type": "Expense",
    "itemDescription": "Movie Ticket",
    "shopName": "Inox",
    "amount": 150000,
    "paymentMethod": "Credit Card",
    "status": "Complete",
    "receiptId": null,
    "createdAt": "2023-05-17T08:12:00Z",
    "category_id": 3
  }
}
```

Xử lý lỗi: Nếu dữ liệu không hợp lệ (ví dụ: amount $\leq$ 0, accountId không tồn tại), throw BadRequestException với JSON:

```
{
  "message": "Invalid or missing transaction data",
  "statusCode": 400
}
```

2. Prompt B: Sinh Mã Frontend (React Component & UI/MCP)

Mục tiêu: Xây dựng giao diện thêm giao dịch mới theo thiết kế Figma.

Tạo component `AddTransactionForm` bằng React TypeScript và Tailwind CSS.

Giao diện bao gồm:

- Input cho 'Tên giao dịch' (Transaction Name / Item Description)
- Dropdown/Select cho 'Danh mục' (Category)
- Input kiểu number cho 'Số tiền' (Amount)
- Radio button hoặc Select cho 'Loại giao dịch' (Type: Revenue / Expense)
- Dropdown/Select cho 'Tài khoản thanh toán' (Payment Account)
- Input kiểu date cho 'Ngày giao dịch' (Transaction Date)
- Nút 'Lưu' (Save) màu xanh
- Nút 'Hủy' (Cancel) để quay lại trang trước

Thiết kế strict theo UI Notes: card bo góc, input border mờ, trường bố trí dọc, label rõ ràng.

3. Prompt C: Sinh Mã Frontend Logic (State & Kết nối API)

Mục tiêu: Kết nối UI với API và xử lý luồng thành công.

Trong `AddTransactionForm`, thêm state cho các trường: `itemDescription`, `amount`, `categoryId`, `type`, `accountId`, `transactionDate`, `paymentMethod`.

Triển khai hàm bất đồng bộ `handleSubmit` gửi request tới `POST /api/v1/transactions`.

Payload request:

```
{"accountId": 3,  
  "transactionDate": "2023-05-17T00:00:00Z",
```



```
"type": "Expense",  
"itemDescription": "Movie Ticket",  
"category_id": 3,  
"amount": 150000}
```

Hành động sau khi thành công:

Hiển thị toast: "Thêm giao dịch thành công.", Reset form, Điều hướng về trang /transactions.

4. Prompt D: Xử lý Ngoại lệ (Client-side Validation & Error Handling)

Mục tiêu: Hoàn thiện xử lý lỗi, loading, và validation client-side.

1. **Trạng thái Loading:** Trước khi gọi API, set `isLoading = true`. Trong khi loading, disable nút 'Lưu' và hiển thị spinner. Set `isLoading = false` sau khi API hoàn tất.
2. **Lỗi API:** Sử dụng `try...catch` bắt lỗi từ API. Nếu trả về 400/422/500, hiển thị toast: "Không thể thêm giao dịch lúc này. Vui lòng thử lại sau."
3. **Validation FE:** Trước khi gọi API, kiểm tra: 'Tên giao dịch' không được trống, 'Số tiền' > 0, Phải chọn 'Danh mục' và 'Tài khoản thanh toán'

Nếu validation thất bại, hiển thị thông báo lỗi ngay dưới input và không gọi API.

4.2.5 Xem danh sách tài khoản ngân hàng

Đặc tả ca sử dụng

Bảng 4.5 mô tả ca sử dụng Xem danh sách tài khoản ngân hàng, hiển thị tất cả tài khoản liên kết của người dùng.

Bảng 4.5: Đặc tả ca sử dụng Xem danh sách tài khoản ngân hàng

Tên ca sử dụng	Xem danh sách tài khoản của người dùng
----------------	--

Mô tả	• Cho phép người dùng đã đăng nhập truy cập và xem danh sách tất cả các tài khoản ngân hàng hoặc ví điện tử mà họ đã liên kết với hệ thống, bao gồm thông tin chi tiết như số dư và loại tài khoản.
Tác nhân	Người dùng (User)
Luồng chính	• Người dùng đã đăng nhập và có access_token hợp lệ. • Người dùng truy cập vào trang "Tài khoản ngân hàng". • Hệ thống Front-end gọi API Backend. • Backend xác thực access_token và trích xuất user_id. • Backend truy vấn bảng Accounts dựa trên user_id. • Backend trả về danh sách tài khoản (Ngân hàng, Loại tài khoản, Chi nhánh, Số tài khoản, Số dư). • Front-end nhận dữ liệu và hiển thị danh sách tài khoản chi tiết.
Luồng thay thế	A.1. Không có tài khoản liên kết: • Nếu danh sách trả về rỗng: hiển thị thông báo "Bạn chưa liên kết tài khoản nào. Vui lòng thêm tài khoản." và cung cấp nút thêm tài khoản mới. A.2. Lỗi xác thực / Lỗi truy cập: • Nếu access_token không hợp lệ hoặc hết hạn: trả về lỗi HTTP 401, chuyển hướng người dùng đến trang Đăng nhập và hiển thị thông báo yêu cầu đăng nhập lại. A.3. Lỗi hệ thống Backend: • Nếu xảy ra lỗi truy vấn cơ sở dữ liệu hoặc xử lý nội bộ: hiển thị thông báo lỗi thân thiện "Đã xảy ra lỗi hệ thống, vui lòng thử lại sau."
Tiền điều kiện	• Người dùng phải đăng nhập (sở hữu access_token hợp lệ). • Người dùng đã liên kết ít nhất một tài khoản ngân hàng / ví điện tử.
Hậu điều kiện	• Thành công: Hệ thống hiển thị danh sách đầy đủ các tài khoản của người dùng trên trang "Tài khoản ngân hàng".

- | | |
|--|---|
| | <ul style="list-style-type: none">• Thất bại: Hệ thống hiển thị thông báo lỗi (ví dụ: lỗi truy cập, không có dữ liệu) và người dùng vẫn ở trang "Tài khoản ngân hàng" hoặc được chuyển hướng đến trang thông báo lỗi. |
|--|---|

Tích hợp UI Figma:

<https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits--Community-?node-id=66-5320&t=cxgXQfkGoLMfxW48-4>

Endpoint:

GET /api/v1/accounts

Request:

GET /api/v1/accounts

Authorization: Bearer <access_token>

Response mẫu:

```
{"user_id": 1,
  "accounts": [
    { "id": 1,
      "bank_name": "Vietcombank",
      "account_type": "Checking",
      "branch_name": "Quan 1",
      "account_number_last_4": "0123",
      "balance": 4000000},
    { "id": 2,
      "bank_name": "Techcombank",
      "account_type": "Savings",
      "branch_name": "Quan 3",
      "account_number_last_4": "0987",
      "balance": 4000000}]]}
```

1. Prompt A: Sinh Mã Backend (NestJS Service/Controller)

Mục tiêu: Xây dựng dịch vụ API, logic nghiệp vụ, và xử lý lỗi server cho chức năng lấy danh sách tài khoản.

Tạo endpoint GET `/api/v1/accounts` trong `AccountsController` NestJS. Endpoint này phải được bảo vệ bằng `JwtAuthGuard`.

Định dạng Request: Không có request body. Thông tin người dùng lấy từ `access_token`.

Logic xử lý trong `AccountsService`:

1. Nhận `user_id` từ payload JWT.
2. Truy vấn cơ sở dữ liệu để tìm tất cả các tài khoản có `user_id` tương ứng.
3. Trả về danh sách các tài khoản. Nếu không có tài khoản, trả về mảng rỗng.

Response thành công:

```
{"user_id": 1,
  "accounts": [
    { "id": 1,
      "bank_name": "Vietcombank",
      "account_type": "Checking",
      "branch_name": "Quan 1",
      "account_number_last_4": "0123",
      "balance": 4000000},
    {"id": 2,
      "bank_name": "Techcombank",
      "account_type": "Savings",
      "branch_name": "Quan 3",
      "account_number_last_4": "0987",
      "balance": 4000000}]]}
```

Xử lý lỗi:

- Lỗi truy vấn hoặc lỗi server khác: throw `InternalServerErrorException` với

```
{ "statusCode": 500,
  "message": "Đã xảy ra lỗi hệ thống, vui lòng thử lại sau." }
```

- Nếu `access_token` không hợp lệ hoặc hết hạn, `JwtAuthGuard` sẽ tự động throw `UnauthorizedException` (HTTP 401).

2. Prompt B: Sinh Mã Frontend (React Component & UI/MCP)

Mục tiêu: Xây dựng giao diện danh sách tài khoản theo thiết kế Figma.

Tạo component `AccountListPage` bằng React TypeScript và Tailwind CSS. Giao diện hiển thị một danh sách các thẻ (card), mỗi thẻ hiển thị thông tin: Tên ngân hàng (`bank_name`), Loại tài khoản (`account_type`), Số dư (`balance`), 4 số cuối của số tài khoản (`account_number_last_4`)

Thiết kế strict theo Figma MCP : <https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits--Community-?node-id=66-5320&t=cxgXQfkGoLMfxW48-4>, bố cục, màu sắc, font chữ khớp 100%.

3. Prompt C: Sinh Mã Frontend Logic (State & Kết nối API)

Mục tiêu: Kết nối UI với API và xử lý luồng thành công.

Trong `AccountListPage`, thêm state: `accounts` để lưu danh sách tài khoản, `isLoading` cho trạng thái tải, `error` để lưu thông báo lỗi

Triển khai hàm bất đồng bộ `fetchAccounts`:

1. Gửi request: `GET /api/v1/accounts`
kèm header: `Authorization: Bearer <access_token>`.
2. Nếu thành công, cập nhật state: `accounts = response.accounts`, `isLoading = false`, Xóa thông báo lỗi cũ
3. Nếu mảng `accounts` rỗng, hiển thị thông báo cho người dùng.

4. Prompt D: Xử lý Ngoại lệ (Client-side Validation & Error Handling)

Mục tiêu: Hoàn thiện xử lý lỗi, trạng thái Loading, và Validation FE.

1. **Trạng thái Loading:** Khi `isLoading = true`, hiển thị Spinner hoặc Skeleton Loader thay cho danh sách tài khoản.
2. **Lỗi API:**
 - 401 Unauthorized: điều hướng người dùng đến trang Đăng nhập.
 - Lỗi server (500): hiển thị thông báo: *"Đã xảy ra lỗi hệ thống, vui lòng thử lại sau."*
 - Thành công nhưng danh sách rỗng: hiển thị thông báo *"Bạn chưa liên kết tài khoản nào. Vui lòng thêm tài khoản."* kèm nút/link điều hướng đến trang thêm tài khoản.
3. **Validation FE:** Không áp dụng, vì đây là ca sử dụng xem dữ liệu, không có form nhập liệu từ người dùng.

4.3 Phần mềm được sinh ra

4.3.1 Giao diện phần mềm

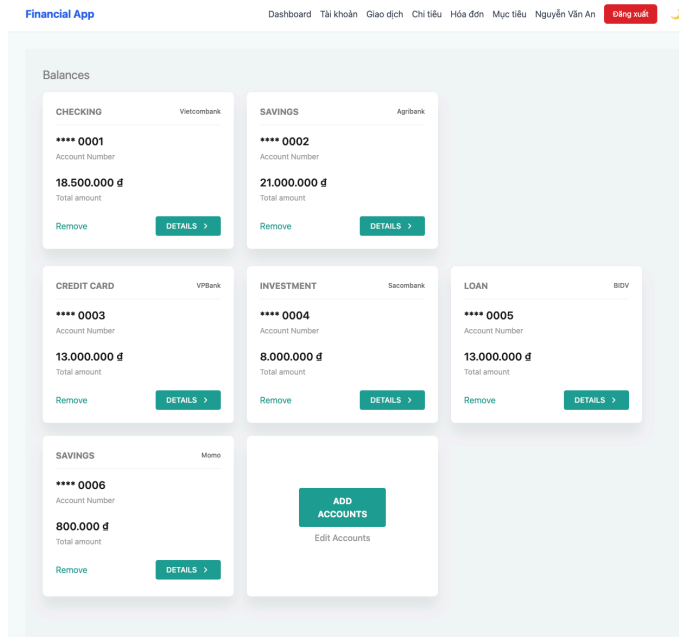
Bao gồm các hình ảnh minh họa các màn hình được sinh tự động, tương ứng với từng ca sử dụng.

Hình 4.4: Giao diện màn hình đăng nhập.

Hình 4.4 mô tả màn hình Đăng nhập tài khoản. Phiếu đăng nhập gồm hai trường bắt buộc là email và mật khẩu. Khi đăng nhập thành công, hệ thống sẽ tự động chuyển sang Trang chủ và hiển thị tên người dùng.

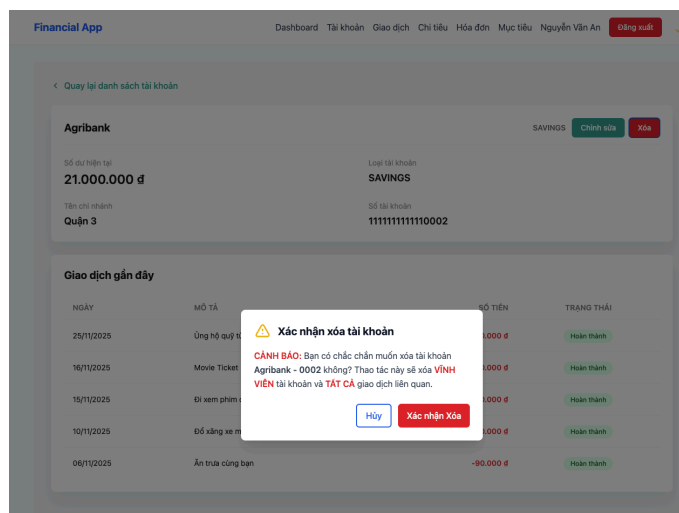
Hình 4.5: Giao diện màn hình đăng ký tài khoản.

Hình 4.5 mô tả màn hình Đăng ký tài khoản. Phiếu đăng ký gồm các trường thông tin bắt buộc bao gồm: họ và tên, email, mật khẩu và xác nhận mật khẩu.



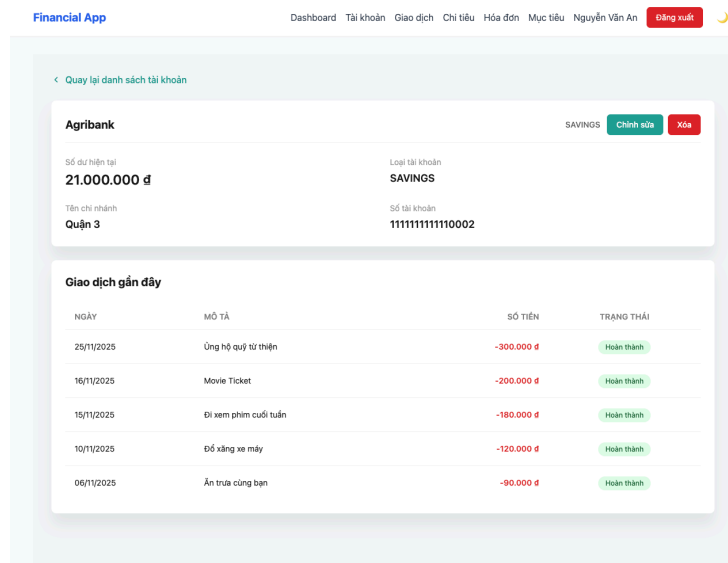
Hình 4.6: Giao diện màn hình xem danh sách tài khoản.

Hình 4.6 mô tả màn hình xem danh sách tài khoản. Tại đây người dùng có thể xem thông tin số tài khoản, chi nhánh, tên ngân hàng... tất cả các tài khoản hiện có cũng như các thao tác thêm, sửa, xóa tài khoản.



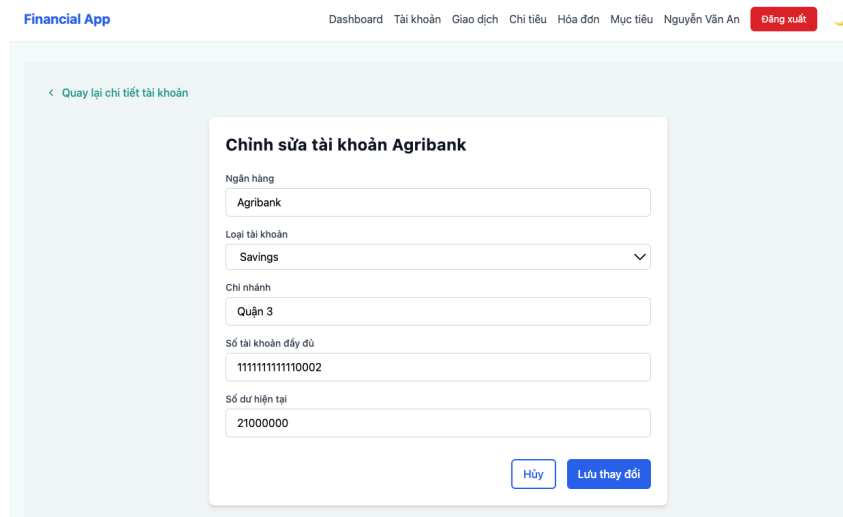
Hình 4.7: Giao diện màn hình xóa tài khoản.

Hình 4.7 mô tả màn hình Xóa tài khoản. Màn hình hiển thị cảnh báo, người dùng bấm "Xác nhận Xóa" nếu chắc chắn muốn xóa tài khoản.



Hình 4.8: Giao diện màn hình xem chi tiết tài khoản.

Hình 4.8 mô tả màn hình xem chi tiết tài khoản. Người dùng có thể xem thông tin chi tiết tài khoản bất kỳ của mình bao gồm: số tài khoản, tên chi nhánh, tên ngân hàng, loại tài khoản, số dư hiện tại cũng như 5 giao dịch gần nhất của tài khoản đó.



Hình 4.9: Giao diện màn hình chỉnh sửa thông tin tài khoản.

Hình 4.9 mô tả màn hình chỉnh sửa thông tin tài khoản. Màn hình bao gồm các trường có thể chỉnh sửa. Sau khi chỉnh sửa người dùng bấm Lưu thay đổi để hệ thống cập nhật thay đổi.

Financial App

DashboardTài khoảnGiao dịchChi tiêuHóa đơnMục tiêuNguyễn Văn AnĐăng xuất

Latest Transactions

All

Revenue

Expenses

ITEMS	SHOP NAME	DATE	PAYMENT METHOD	AMOUNT
— Ủng hộ chương trình thiện nguyện	VTV Charity	22/12/2025	Bank Transfer	-500.000 đ
+ Tiền lãi ngân hàng	Ngân hàng	18/12/2025	Bank Tranfer	+12.000.000 đ
— Ăn trưa công ty	Cơm gà Hải Nam	03/12/2025	Cash	-100.000 đ
— Tiền thuê nhà tháng 12	Chủ nhà	01/12/2025	Bank Transfer	-6.000.000 đ
— Tặng quà sinh nhật bạn	Miniso	26/11/2025	Cash	-250.000 đ
— Ủng hộ quỹ từ thiện	Quỹ Trái tim Việt	25/11/2025	Bank Transfer	-300.000 đ
— Mua váy	Canifa	20/11/2025	Credit Card	-600.000 đ
— Mua áo len mùa đông	H&M	18/11/2025	Credit Card	-750.000 đ
— Movie Ticket	CGV	16/11/2025	Bank tranfer	-200.000 đ
— Đi xem phim cuối tuần	CGV Vincom	15/11/2025	Credit Card	-180.000 đ

Tải thêm

Hình 4.10: Giao diện màn hình xem danh sách giao dịch.

Hình 4.10 mô tả màn hình Xem danh sách giao dịch. Màn hình hiển thị tất cả giao dịch sắp xếp theo ngày gần nhất, có các tab chia giao dịch thành Thu nhập và Chi tiêu.

Financial App

DashboardTài khoảnGiao dịchChi tiêuHóa đơnMục tiêuNguyễn Văn AnĐăng xuất

Thêm Giao Dịch Mới

Tên giao dịch

Ví dụ: Movie Ticket

Danh mục *

-- Chọn danh mục --

Số tiền

0

Loại giao dịch

Thu nhập (Revenue)

Chi tiêu (Expense)

Tài khoản thanh toán *

-- Chọn tài khoản --

Ngày giao dịch

29/11/2025

Tên cửa hàng (Tùy chọn)

Ví dụ: Inox

Phương thức thanh toán (Tùy chọn)

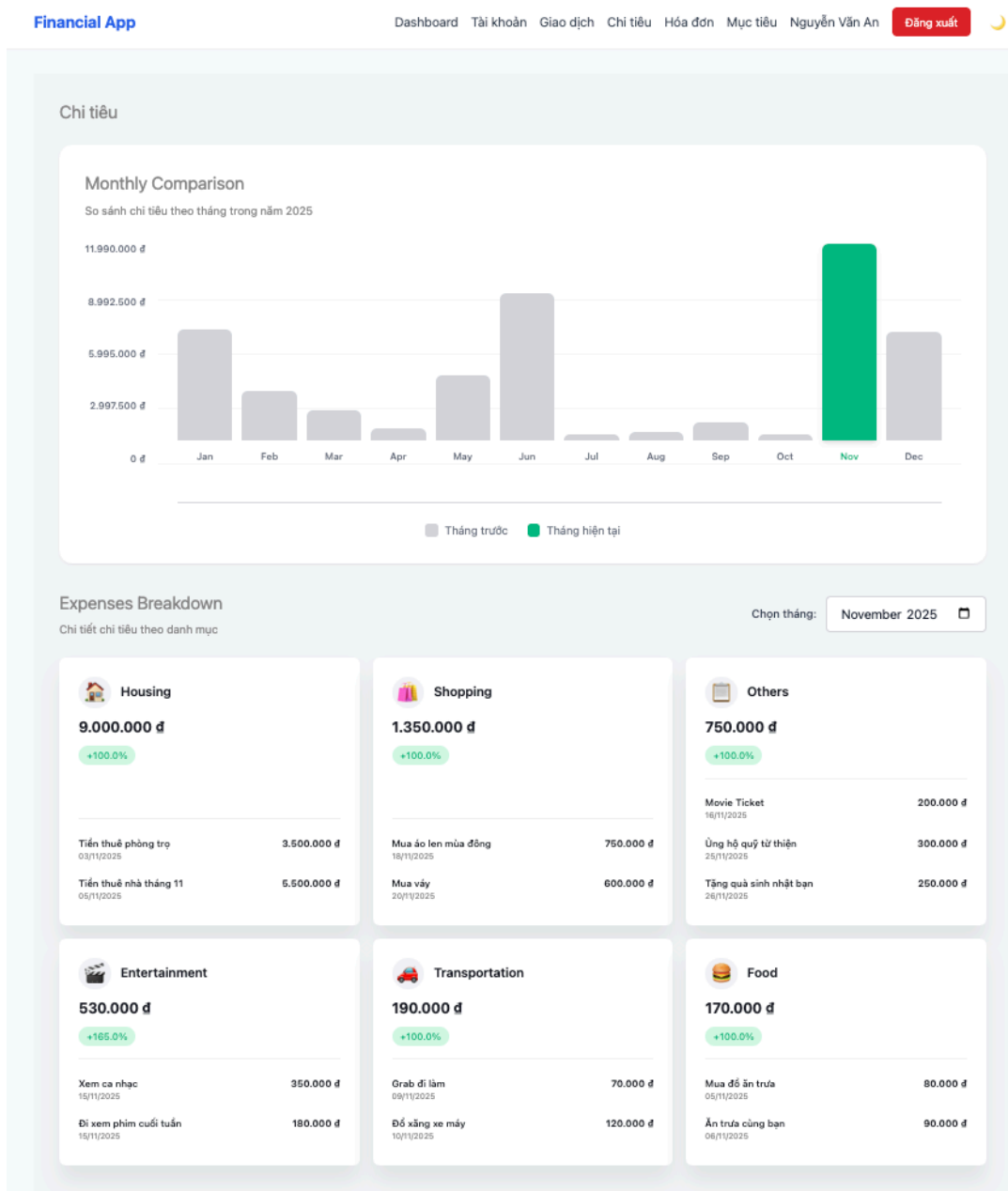
Ví dụ: Credit Card

Hủy

Lưu

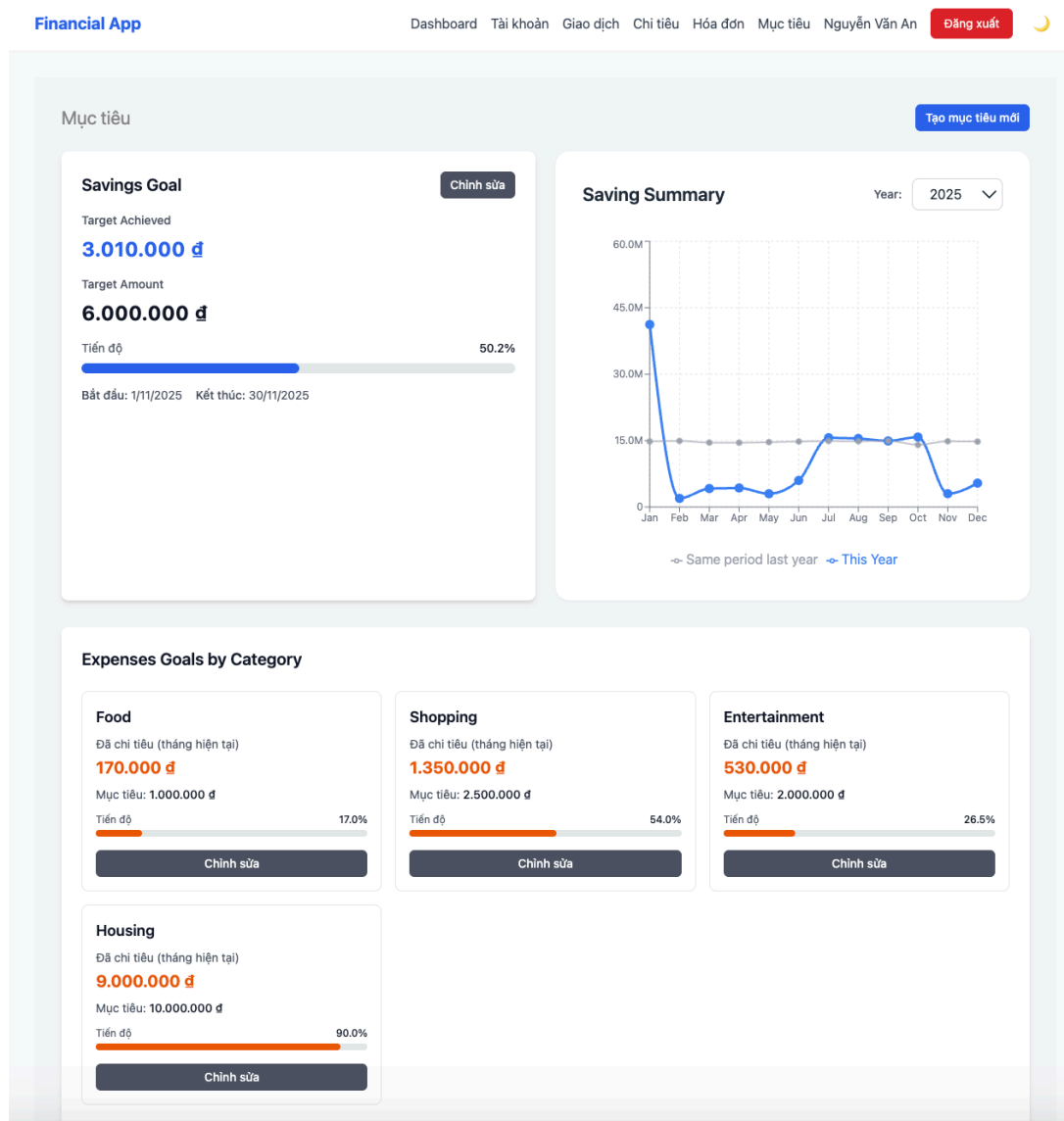
Hình 4.11: Giao diện màn hình thêm giao dịch mới.

Hình 4.11 mô tả màn hình Thêm giao dịch mới. Màn hình hiển thị Form cho người dùng nhập các thông tin như Tên giao dịch, Loại giao dịch, Danh mục, Tài khoản thanh toán,...Sau khi nhập đầy đủ thông tin cần thiết, người dùng bấm Lưu để thêm mới giao dịch.



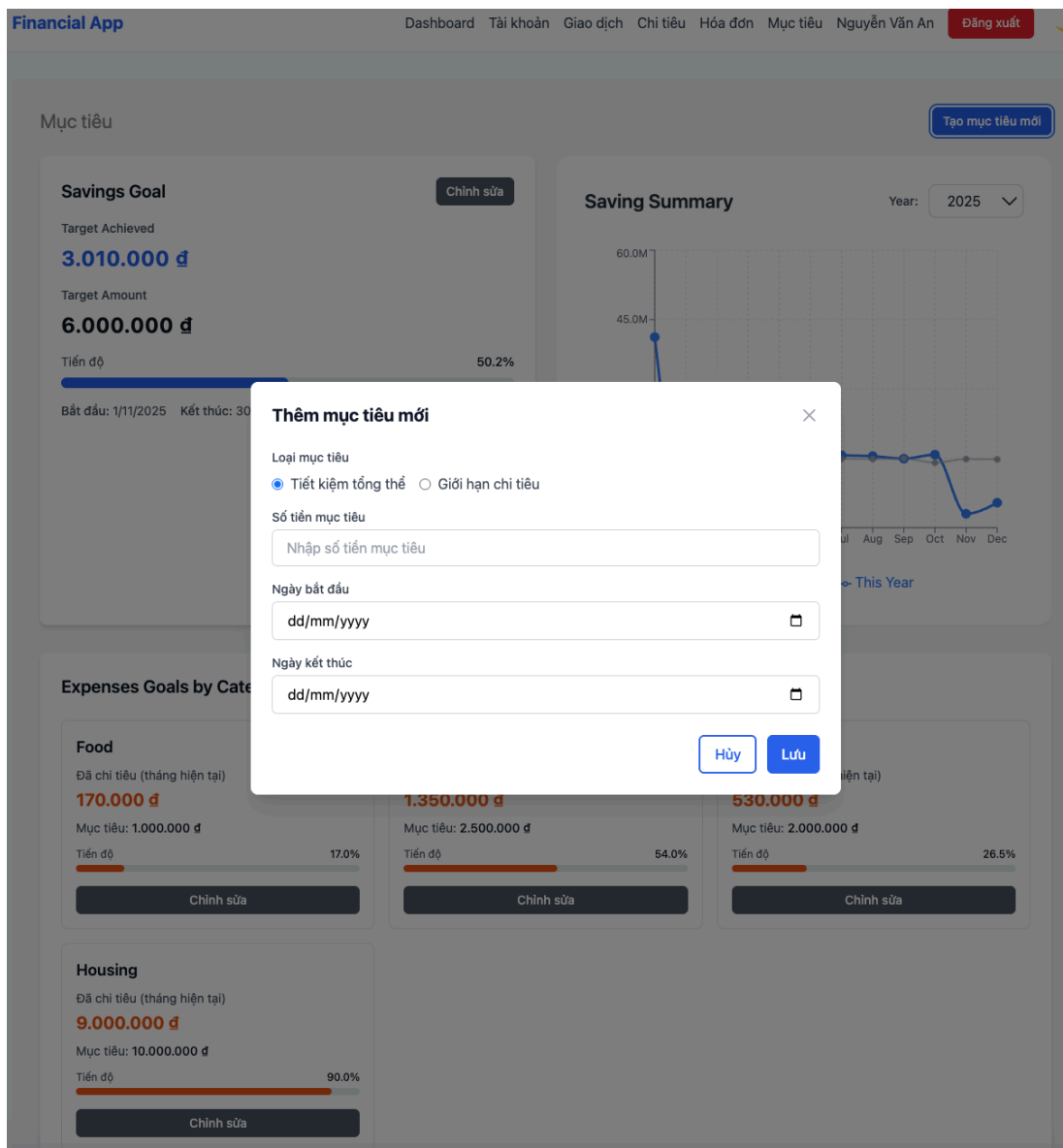
Hình 4.12: Giao diện màn hình tổng quan chi tiêu.

Hình 4.12 mô tả màn hình Xem Tổng quan chi tiêu và Xem chi tiêu theo danh mục. Màn hình hiển thị biểu đồ tổng chi tiêu của tháng hiện tại và các tháng trước, bên dưới hiển thị tổng chi tiêu trong tháng được nhóm theo danh mục, so sánh tỉ lệ tăng giảm so với các tháng trước đó.



Hình 4.13: Giao diện màn hình xem mục tiêu hàng tháng.

Hình 4.13 mô tả màn hình Xem mục tiêu hàng tháng. Màn hình hiển thị mục tiêu tiết kiệm, số tiền đã đạt được và so sánh tỉ lệ. Người dùng có thể xem và chỉnh sửa mục tiêu tiết kiệm và mục tiêu chi tiêu theo từng danh mục. Biểu đồ Tổng quan tiết kiệm giúp người dùng so sánh khoản tiết kiệm ròng theo từng tháng trong năm hiện tại và năm trước, giúp đánh giá mức độ cải thiện tài chính theo thời gian.



Hình 4.14: Giao diện màn hình thêm mục tiêu.

Hình 4.13 mô tả màn hình Thêm Mục tiêu. Màn hình hiển thị Form nhập các thông tin Loại mục tiêu, số tiền, ngày bắt đầu và kết thúc. Người dùng bấm Lưu để thêm mục tiêu mới vào hệ thống.

4.3.2 Mã nguồn và các file sinh tự động

Sau khi hoàn thành quá trình thực nghiệm, toàn bộ mã nguồn của hệ thống đã được tổng hợp và công bố tại đường dẫn (<https://github.com/thanhttruc/chained-prompt/tree/demo>) nhằm tạo điều kiện cho người đọc tham khảo và kiểm chứng kết quả.

Danh sách file sinh mới trong Backend

Bảng 4.6 liệt kê toàn bộ các file Backend mới tạo kèm theo mục đích sử dụng, trạng thái tạo/chỉnh sửa và use case tương ứng.

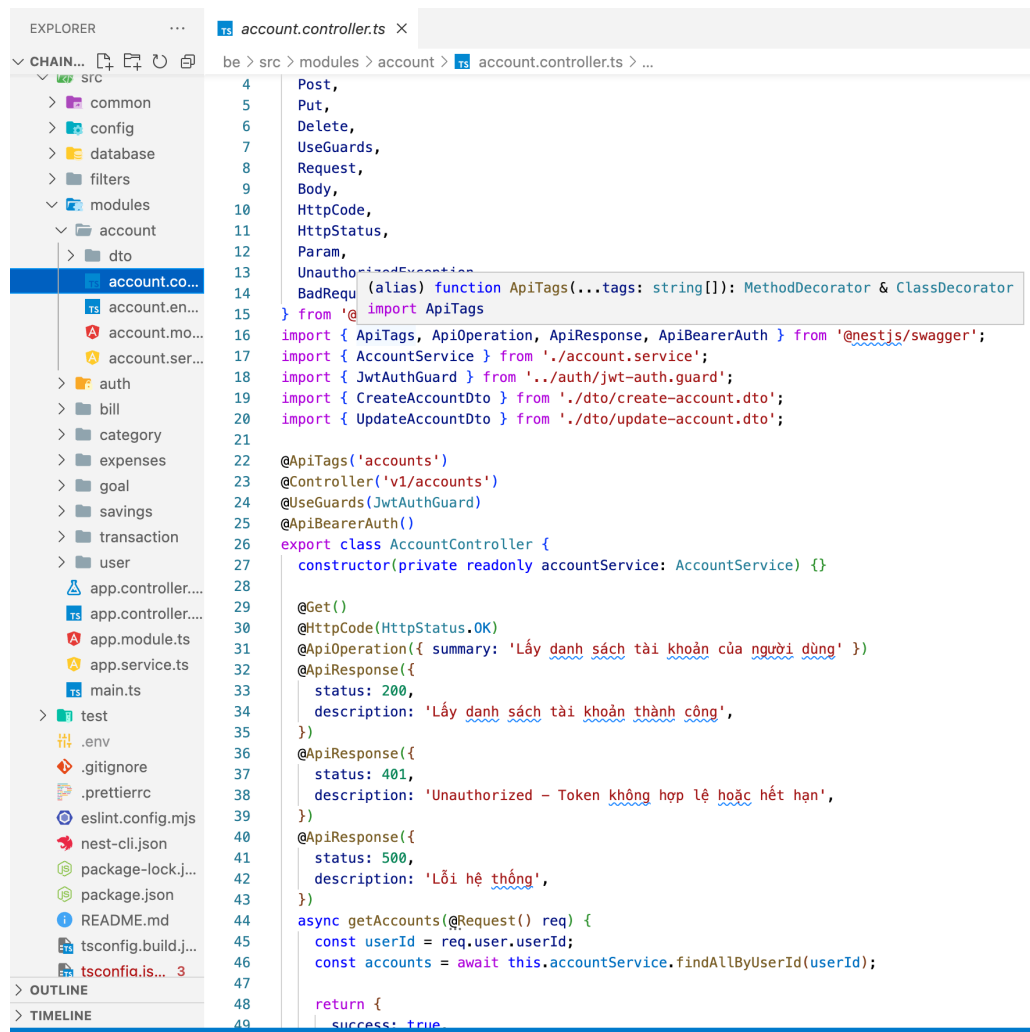
Bảng 4.6: Danh sách các thư mục và tệp mới được tạo trong Backend

File	Mục đích	Cá sử dụng liên quan
app.controller.ts	Route kiểm thử gốc	Không
app.service.ts	Logic cơ bản cho root controller	Không
app.module.ts	Module gốc ứng dụng, kết nối các module	Toàn bộ
main.ts	Khởi chạy server, cấu hình CORS/Swagger	Toàn bộ
config/database.config.ts	Cấu hình kết nối MySQL	Toàn bộ
database/ database.module.ts	Khởi tạo TypeORM	Toàn bộ
filters/ http-exception.filter.ts	Xử lý lỗi HTTP	Toàn bộ
modules/auth/ auth.controller.ts	API đăng nhập/đăng ký	Đăng nhập, Đăng ký
modules/auth/ auth.service.ts	Logic đăng nhập/đăng ký, hash mật khẩu	Đăng nhập, Đăng ký
modules/auth/ jwt-auth.guard.ts	Bảo vệ các route yêu cầu JWT	Mọi route cần bảo vệ

modules/auth/ jwt.strategy.ts	Logic giải mã và xác thực JWT	Mọi route cần bảo vệ
modules/auth/ dto/login.dto.ts	DTO cho request đăng nhập	Đăng nhập
modules/auth/ dto/register.dto.ts	DTO cho request đăng ký	Đăng ký
modules/account/ account.controller.ts	API thực hiện CRUD cho tài khoản	Xem/Thêm/Sửa/Xoá tài khoản
modules/account/ account.service.ts	Logic nghiệp vụ cho tài khoản	Xem/Thêm/Sửa/Xoá tài khoản
modules/account/ account.entity.ts	Lược đồ cho bảng Accounts	Xem/Thêm/Sửa/Xoá tài khoản
modules/transaction/ transaction.controller.ts	API cho giao dịch	Xem lịch sử, Tạo giao dịch
modules/transaction/ transaction.service.ts	Logic xử lý giao dịch	Xem lịch sử, Tạo giao dịch
modules/bill/ bill.controller.ts	API cho hoá đơn	Xem danh sách hoá đơn
modules/bill/ bill.service.ts	Logic truy vấn hoá đơn	Xem danh sách hoá đơn
modules/category/ category.controller.ts	API cho danh mục	Xem chi tiêu theo danh mục
modules/category/ category.service.ts	Logic xử lý danh mục	Xem chi tiêu theo danh mục
modules/expenses/ expenses.controller.ts	API phân tích chi tiêu	Xem chi tiêu hàng tháng, chi tiết theo danh mục
modules/expenses/ expenses.service.ts	Logic tổng hợp chi tiêu	Xem chi tiêu hàng tháng, chi tiết theo danh mục

modules/goal/ goal.controller.ts	API thực hiện CRUD cho mục tiêu	Xem, Tạo, Sửa mục tiêu
modules/goal/ goal.service.ts	Logic xử lý mục tiêu	Xem, Tạo, Sửa mục tiêu
modules/goal/ goal.entity.ts	Lược đồ cho bảng 'Goals'	Xem, Tạo, Sửa mục tiêu
modules/savings/ savings.controller.ts	API tổng hợp tiết kiệm	Xem biểu đồ Tổng quan Tiết kiệm
modules/savings/ savings.service.ts	Logic tính tiết kiệm ròng	Xem biểu đồ Tổng quan Tiết kiệm
modules/user/ user.entity.ts	Lược đồ cho bảng 'Users'	Đăng nhập, Đăng ký

Hình 4.17 mô tả kết quả mã nguồn sinh tự động trong thư mục Backend:



Hình 4.17: Mô tả màn hình mã nguồn sinh tự động trong thư mục Backend.

Danh sách thư mục sinh mới trong Frontend

Bảng 4.7 trình bày danh sách các file Frontend mới tạo, mô tả chức năng của từng file và mối liên hệ với các use case của hệ thống.

Bảng 4.7: Danh sách các thư mục và tệp mới được tạo trong Frontend

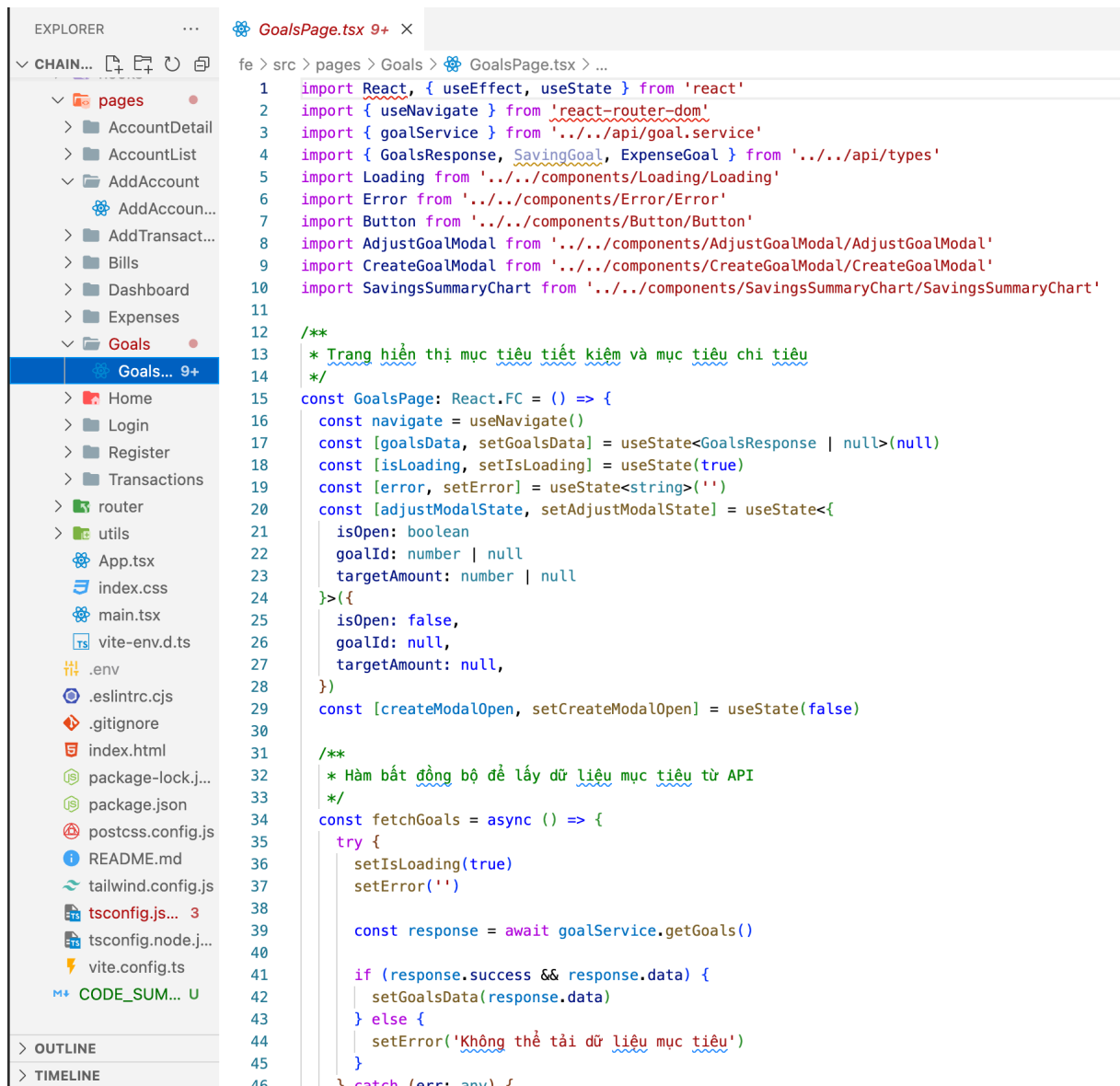
File	Mục đích	Cá sử dụng liên quan
App.tsx	Component gốc, bao quanh các provider	Tất cả
main.tsx	Render ứng dụng vào DOM	Tất cả
index.css	Style toàn cục / Tailwind	Tất cả

vite-env.d.ts	Khai báo kiểu biến môi trường	Tất cả
router/AppRouter.tsx	Quản lý routing toàn bộ ứng dụng	Tất cả
api/axiosInstance.ts	Cấu hình Axios với JWT token	Tất cả
api/account.service.ts	Gọi API tài khoản	Xem, Thêm, Sửa, Xóa, Chi tiết tài khoản
api/auth.service.ts	Gọi API đăng nhập / đăng ký	Đăng nhập, Đăng ký
api/transaction.service.ts	Gọi API giao dịch	Xem lịch sử, Tạo giao dịch
api/bill.service.ts	Gọi API hóa đơn	Xem danh sách hóa đơn
api/category.service.ts	Gọi API danh mục	Xem chi tiết chi tiêu theo danh mục
api/expense.service.ts	Gọi API tổng hợp chi tiêu	Xem chi tiêu hàng tháng, chi tiết theo danh mục
api/goal.service.ts	Gọi API mục tiêu	Xem, Tạo, Điều chỉnh mục tiêu
api/savings.service.ts	Gọi API tổng hợp tiết kiệm	Xem biểu đồ tiết kiệm
api/user.service.ts	Gọi API thông tin người dùng	Đăng nhập, Đăng ký
api/types.ts	Khai báo type TypeScript	Tất cả
api/index.ts	Export các service	Tất cả
LoginForm/ LoginForm.tsx	Form đăng nhập	Đăng nhập
SignUpForm/ SignUpForm.tsx	Form đăng ký	Đăng ký

AddTransactionForm/ AddTransactionForm.tsx	Form thêm giao dịch	Tạo giao dịch mới
AddAccountForm/ AddAccountForm.tsx	Form thêm tài khoản	Thêm tài khoản
AccountEditForm/ AccountEditForm.tsx	Form sửa tài khoản	Sửa tài khoản
DeleteAccountModal/ DeleteAccountModal.tsx	Modal xóa tài khoản	Xóa tài khoản
UpcomingBills/ UpcomingBills.tsx	Bảng hiển thị hóa đơn sắp tới	Xem danh sách hóa đơn
ExpensesBreakdown/ ExpensesBreakdown.tsx	Chi tiết chi tiêu theo danh mục	Xem chi tiết chi tiêu
ExpenseSummaryChart/ ExpenseSummaryChart.tsx	Biểu đồ chi tiêu tháng	Xem chi tiêu hàng tháng
SavingsSummaryChart/ SavingsSummaryChart.tsx	Biểu đồ tiết kiệm	Xem biểu đồ tiết kiệm
CreateGoalModal/ CreateGoalModal.tsx	Modal tạo mục tiêu	Tạo mục tiêu mới
AdjustGoalModal/ AdjustGoalModal.tsx	Modal điều chỉnh mục tiêu	Điều chỉnh mục tiêu
Layout/Layout.tsx	Layout chung toàn app	Tất cả
Input/Input.tsx	Component input tái sử dụng	Tất cả form
Button/Button.tsx	Component button tái sử dụng	Tất cả
Toast/Toast.tsx	Hiển thị thông báo nhanh (toast)	Tất cả
Login/Login.tsx	Trang đăng nhập	Đăng nhập
Register/Register.tsx	Trang đăng ký	Đăng ký

Transactions/ TransactionsPage.tsx	Trang danh sách giao dịch	Xem lịch sử giao dịch
AddTransaction/ AddTransactionPage.tsx	Trang thêm giao dịch	Tạo giao dịch mới
AccountList/ AccountListPage.tsx	Trang danh sách tài khoản	Xem danh sách tài khoản
AccountDetail/ AccountDetailPage.tsx	Trang chi tiết tài khoản	Xem chi tiết tài khoản
AddAccount/ AddAccountPage.tsx	Trang thêm tài khoản	Thêm tài khoản
Bills/BillsPage.tsx	Trang hóa đơn	Xem danh sách hóa đơn
Expenses/ExpensesPage.tsx	Trang phân tích chi tiêu	Xem chi tiêu hàng tháng, chi tiết theo danh mục
Goals/GoalsPage.tsx	Trang quản lý mục tiêu	Xem, Tạo, Điều chỉnh mục tiêu
Dashboard/Dashboard.tsx	Trang tổng quan	Tất cả
Home/Home.tsx	Trang home	Tất cả

Hình 4.18 dưới đây mô tả kết quả mã nguồn sinh tự động trong thư mục Frontend:



Hình 4.18: Mô tả màn hình mã nguồn sinh tự động trong thư mục Frontend.

4.4 Đánh giá kết quả

Chương này trình bày quá trình đánh giá phương pháp phát triển phần mềm tự động dựa trên đặc tả ca sử dụng thông qua các thí nghiệm đã được thực hiện ở Chương 3. Việc đánh giá được tiến hành theo hai hướng: (1) phân tích định lượng dựa trên các thông số đo được trong từng lần chạy sinh mã và (2) phân tích định tính dựa trên mức độ chính xác, tính nhất quán và chi phí chỉnh sửa của mã nguồn được tạo ra. Kết quả thu được là cơ sở để xác định mức độ khả thi, hiệu quả và tiềm năng ứng dụng của mô hình vào thực tiễn phát triển phần mềm.

4.4.1 Bảng thống kê các lần chạy

Trong nghiên cứu này, em đã tiến hành thực nghiệm trên **16 ca sử dụng**. Mỗi ca sử dụng được đo đạc các thông tin gồm: số lượng subprompt được tạo ra, mức độ chỉnh sửa thủ công (Sửa tay), mức độ chỉnh sửa bằng AI (Cursor sửa), thời gian xử lý thực tế. Bên cạnh đó, thời gian ước lượng nếu viết code thủ công được tổng hợp dựa trên đánh giá của bốn bạn cùng chuyên ngành mà em đã nhờ tham khảo. Kết quả chi tiết được trình bày trong Bảng 4.8.

Bảng 4.8: Thống kê thời gian thực thi cho từng ca sử dụng

Lần	Ca sử dụng	Số sub-prompt	Cursor (phút)	Fix tay (phút)	Code tay (giờ)
1	Đăng ký tài khoản	2	5	15	10
2	Đăng nhập	2	5	15	8
3	Xem lịch sử giao dịch	3	3	10	15
4	Tạo giao dịch mới	4	3	10	12
5	Xem danh sách tài khoản ngân hàng	2	4	15	15
6	Thêm tài khoản ngân hàng	3	4	10	10
7	Chỉnh sửa tài khoản ngân hàng	3	5	15	12
8	Xoá tài khoản ngân hàng	2	3	10	8
9	Xem chi tiết tài khoản ngân hàng	2	4	25	15
10	Xem Chi tiêu Hàng tháng	3	4	30	18
11	Xem Chi tiết Chi tiêu theo Danh mục	3	4	20	20
12	Xem Danh sách Hóa đơn	2	3	20	14

13	Xem Mục tiêu hàng tháng	2	3	25	17
14	Tạo Mục tiêu mới	3	4	15	10
15	Điều chỉnh Mục tiêu hàng tháng	3	3	15	12
16	Xem Biểu đồ Tổng quan Tiết kiệm	3	5	25	20

Từ kết quả trên, thời gian trung bình để xử lý một ca sử dụng đạt khoảng 18.7 phút. Các ca sử dụng có luồng nghiệp vụ phức tạp (ví dụ: thống kê, biểu đồ, tính toán tập hợp) thường có số subprompt cao hơn và yêu cầu nhiều bước chỉnh sửa hơn so với các ca sử dụng CRUD đơn giản.

4.4.2 Đánh giá hiệu quả

Dựa trên dữ liệu của 16 ca sử dụng, tổng thời gian xử lý bằng phương pháp sinh mã tự động (bao gồm thời gian sử dụng Cursor và thời gian chỉnh sửa thủ công) dao động trong khoảng **8–40 phút**, tùy theo độ phức tạp của từng nghiệp vụ. Thời gian trung bình của toàn bộ 16 ca sử dụng là khoảng **18.7 phút** cho mỗi ca.

Trong khi đó, thời gian phát triển thủ công từ đầu đến cuối được ghi nhận trong bảng cho thấy phần lớn các ca sử dụng yêu cầu 8–20 giờ để hoàn thiện (tương đương **480–1.200 phút**). Trung bình, thời gian code thủ công cho 16 ca sử dụng vào khoảng **13.1 giờ (786 phút)**.

Từ sự chênh lệch này có thể thấy phương pháp sinh mã tự động giúp rút ngắn thời gian phát triển khoảng **97,6%** so với cách lập trình truyền thống. Đây là mức giảm đáng kể, phản ánh tiềm năng lớn của mô hình trong việc tối ưu hóa quá trình xây dựng hệ thống phần mềm.

Khi phân loại 16 ca sử dụng thành bốn nhóm theo mức độ phức tạp, thời gian xử lý được ghi nhận như sau:

- CRUD đơn giản: 8–20 phút

- CRUD nâng cao: 15–30 phút
- Tổng hợp dữ liệu: 18–35 phút
- Thống kê phức tạp: 25–40 phút

Kết quả cho thấy, mặc dù độ phức tạp tăng dần giữa các nhóm nghiệp vụ, thời gian xử lý bằng phương pháp sinh mã tự động vẫn luôn thấp hơn từ **10 đến 20 lần** so với phát triển thủ công. Điều này cho thấy mô hình không chỉ hiệu quả trong các tác vụ CRUD mà còn duy trì được mức độ hiệu quả cao ngay cả với các chức năng tổng hợp hoặc thống kê đòi hỏi nhiều thao tác xử lý dữ liệu.

Tổng hợp các kết quả thực nghiệm, có thể khẳng định phương pháp sinh mã tự động dựa trên đặc tả ca sử dụng mang lại nhiều lợi ích đáng kể, bao gồm:

- Tiết kiệm hơn **97%** thời gian phát triển so với lập trình thủ công.
- Nâng cao tính đồng nhất của mã nguồn, nhờ áp dụng cấu trúc sinh mã tự động theo chuẩn định nghĩa trước.
- Giảm thiểu lỗi lặp lại trong quá trình triển khai các chức năng backend tương tự nhau.
- Tăng tốc đáng kể quá trình xây dựng hệ thống, đặc biệt trong các dự án quy mô lớn hoặc có số lượng API nhiều.

Các kết quả trên khẳng định mô hình có khả năng hỗ trợ hiệu quả trong việc tăng tốc và chuẩn hóa quy trình phát triển phần mềm dựa trên đặc tả ca sử dụng.

Chương 5

Kết luận

5.1 Tóm tắt kết quả đạt được

Khóa luận đã đề xuất và kiểm chứng một quy trình phát triển phần mềm tự động dựa trên đặc tả ca sử dụng. Quy trình này cho phép chuyển đổi đặc tả đầu vào thành chuỗi hướng dẫn cho mô hình tạo mã, từ đó sinh ra các thành phần phần mềm như tầng xử lý nghiệp vụ, tầng giao tiếp dữ liệu và cấu trúc cơ sở dữ liệu. Kết quả thực nghiệm cho thấy phương pháp có khả năng rút ngắn thời gian xây dựng hệ thống ban đầu, giảm khối lượng công việc lặp lại và duy trì mức độ thống nhất tương đối giữa các phần tử được sinh ra. Điều này chứng minh tiềm năng của hướng tiếp cận tự động hóa dựa trên đặc tả ca sử dụng.

5.2 Hạn chế và thách thức của Vibe Coding

Bên cạnh các kết quả đạt được, phương pháp phát triển phần mềm bằng Vibe Coding vẫn tồn tại nhiều hạn chế. Trước hết, khả năng duy trì tính nhất quán của luồng xử lý còn yếu; mã sinh ra có thể hoạt động nhưng đôi khi rời rạc, thiếu liên kết logic giữa các phần. Khi xuất hiện lỗi phức tạp liên quan đến luồng dữ liệu hoặc trạng thái hệ thống, mô hình khó nhận diện nguyên nhân và không hỗ trợ tốt việc truy vết. Cấu trúc dự án sinh ra cũng chưa thật sự phù hợp cho việc mở rộng hoặc tích hợp vào hệ thống lớn.

Ở góc độ người sử dụng, sự tương tác giữa con người và mô hình phát sinh nhiều vấn đề: người dùng dễ phụ thuộc vào mã sinh tự động, ít kiểm chứng kết quả, hoặc mô tả yêu cầu không chính xác, dẫn đến sai lệch trong mã sinh ra. Ở cấp độ tổ chức, cách phát triển này chưa phù hợp với quy trình phát triển phần mềm chuẩn hóa, gây khó khăn trong kiểm soát phiên bản, kiểm thử, triển khai và tuân thủ yêu cầu an toàn thông tin. Ngoài ra, việc sinh mã từ dữ liệu học đặt ra rủi ro liên quan đến bản quyền và trách nhiệm khi hệ thống tự sinh gây ra lỗi nghiêm trọng.

5.3 Định hướng phát triển trong tương lai

Nghiên cứu trong tương lai có thể phát triển theo các hướng: (1) chuẩn hóa đặc tả ca sử dụng nhằm giảm sự mơ hồ và tăng khả năng chuyển đổi sang hướng dẫn cho mô hình; (2) bổ sung cơ chế kiểm tra logic tự động để phát hiện bất nhất trong mã sinh ra; (3) tích hợp quy trình kiểm thử tự động vào luồng tạo mã; và (4) nghiên cứu kết hợp Vibe Coding với các phương pháp kiểm chứng hình thức để hạn chế lỗi logic. Bên cạnh đó, việc xây dựng một quy trình phát triển phần mềm mới phù hợp với thực tiễn phát triển có sự hỗ trợ của mô hình sinh mã cũng là hướng đi cần thiết để phương pháp này có thể ứng dụng ở quy mô công nghiệp.

Tài liệu tham khảo

Tiếng Anh

- [1] Cursor AI. *Cursor: The AI Code Editor*. <https://cursor.com/features>. Truy cập ngày 30/11/2025. 2024.
- [2] RandomCoding. *Is Cursor AI's Code Editor Any Good?* <https://randomcoding.com/blog/2024-09-15-is-cursor-ais-code-editor-any-good/>. Truy cập ngày 30/11/2025. 2024.
- [3] EngineLabs. *Cursor AI: An In-Depth Review in 2025*. <https://blog.enginelabs.ai/cursor-ai-an-in-depth-review/>. Truy cập ngày 30/11/2025. 2025.
- [4] Codeaholicguy. *My Engineering Workflow in CursorAI*. <https://codeaholicguy.com/2025/10/18/my-engineering-workflow-in-cursorai/>. Truy cập ngày 30/10/2025. 2025.
- [5] Kaustubh Saini. *What CTOs Really Think About Vibe Coding*. <https://www.finalroundai.com/blog/what-ctos-think-about-vibe-coding?ref=dailydev>. Truy cập ngày 30/10/2025. 2025.
- [6] Minh-Hue Chu **and** Anh-Hien Dao. “Automated Code Generation from Use cases and the Domain Model”. in *Annals of Computer Science and Information Systems, Volume 33*: Truy cập ngày 30/11/2025. 2023. URL: https://annals-csis.org/Volume_33/drp/27.html.
- [7] Zeeshan Rasheed **and others**. *CodePori: Large-Scale System for Autonomous Software Development Using Multi-Agent Technology*. Truy cập ngày 30/11/2025. 2024. URL: <https://arxiv.org/abs/2402.01411>.

- [8] Hina Mahmood, Atif Aftab Jilani **and** Abdul Rauf. *Code Swarm: A Code Generation Tool Based on the Automatic Derivation of Transformation Rule Set*. Truy cập ngày 30/11/2025. 2023. URL: <https://arxiv.org/abs/2312.01524>.
- [9] Lezhi Ma **and** others. *SpecGen: Automated Generation of Formal Program Specifications via Large Language Models*. Truy cập ngày 30/11/2025. 2024. URL: <https://arxiv.org/abs/2401.08807>.

Tiếng Việt

- [10] Thị Hạnh Quách **and** Tất Diệp Nguyễn Tố Uyên và Vũ. “VIBECODING trong phát triển phần mềm: Một tổng quan hệ thống về tác động đến tốc độ, chất lượng và nợ kỹ thuật”. *in Tạp chí Khoa học Trường Đại học Mở Hà Nội*: (2025). Truy cập ngày 30/11/2025. URL: <https://jshou.edu.vn/houjs/article/view/726>.
- [11] VnExpress. *Xu hướng viết code không cần hiểu lập trình*. <https://vnexpress.net/xu-huong-viet-code-khong-can-hieu-lap-trinh-4885788.html>. Truy cập ngày 30/11/2025. 2024.
- [12] Google. *Giới thiệu Gemini 2.0: Mô hình AI mới của Google*. <https://vietnamese.googleblog.com/2024/12/gioi-thieu-gemini-20-mo-hinh-ai-moi-cua.html>. Truy cập ngày 30/11/2025. 2024.
- [13] VnExpress. *Google ra Gemini 2.0 tạo nội dung đa phương thức*. <https://vnexpress.net/google-ra-gemini-2-0-tao-noi-dung-da-phuong-thuc-4826692.html>. Truy cập ngày 30/11/2025. 2024.
- [14] Cogover. *Gemini là gì? Hướng dẫn đầy đủ về Google Gemini AI*. <https://cogover.com/vi/blog/gemini-la-gi>. Truy cập ngày 30/11/2025. 2025.

Phụ lục

Đặc tả ca sử dụng chi tiết

A.0.1 Thêm tài khoản ngân hàng

Đặc tả ca sử dụng

Bảng 1.1 trình bày đặc tả ca sử dụng Thêm tài khoản ngân hàng, cho phép người dùng bổ sung một tài khoản mới vào hệ thống.

Bảng 1.1: Đặc tả ca sử dụng Thêm tài khoản ngân hàng mới

Tên ca sử dụng	Thêm tài khoản mới
Mô tả	Cho phép người dùng đã đăng nhập liên kết và thêm thông tin về một tài khoản ngân hàng hoặc ví điện tử mới vào hệ thống để theo dõi và quản lý tài chính.
Tác nhân	Người dùng (User)
Luồng chính	- Người dùng truy cập vào trang "Tài khoản" và nhấn "Thêm tài khoản". - Người dùng nhập thông tin tài khoản mới: Tên ngân hàng/Ví, Loại tài khoản, Số tài khoản, Chi nhánh (tùy chọn), Số dư ban đầu. - Nhấn nút "Thêm tài khoản" hoặc "Lưu". - Front-end gửi dữ liệu đến API Backend. - Backend xác thực access_token và trích xuất user_id. - Backend kiểm tra tính hợp lệ của dữ liệu đầu vào.

	• Nếu hợp lệ, Backend lưu thông tin tài khoản mới vào bảng Accounts, liên kết với user_id. • Backend trả về thông tin chi tiết của tài khoản vừa tạo. • Front-end thông báo thành công và chuyển hướng người dùng đến trang Xem danh sách tài khoản.
Luồng thay thế	A.1. Dữ liệu đầu vào không hợp lệ: • Nếu bất kỳ trường dữ liệu nào vi phạm quy tắc kiểm tra (ví dụ: số dư âm, tên ngân hàng trống): hiển thị thông báo lỗi chi tiết bên cạnh trường bị lỗi và giữ người dùng ở màn hình Thêm tài khoản. A.2. Lỗi trùng lặp Số tài khoản (tùy chọn): • Nếu số tài khoản đã tồn tại với người dùng: hiển thị thông báo "Tài khoản này đã tồn tại trong danh sách của bạn." A.3. Lỗi hệ thống Backend: • Nếu xảy ra lỗi trong quá trình lưu dữ liệu: hiển thị thông báo "Không thể thêm tài khoản lúc này. Vui lòng thử lại sau."
Tiền điều kiện	• Người dùng phải đăng nhập. • Người dùng đang truy cập vào trang "Tài khoản".
Hậu điều kiện	• Thành công: Tài khoản mới được lưu vào hệ thống, người dùng chuyển hướng đến trang danh sách tài khoản hoặc trang xác nhận. • Thất bại: Hiển thị thông báo lỗi về dữ liệu không hợp lệ hoặc lỗi hệ thống, người dùng vẫn ở màn hình Thêm tài khoản.

Tích hợp UI Figma:

Thiết kế phù hợp với giao diện trang Tài khoản ngân hàng với Form thêm mới : Ngân hàng(bank_name), Loại tài khoản (account_type), Chi nhánh (branch_name), Số tài khoản đầy đủ (account_number_full), Số dư khởi tạo (balance).

Endpoint:

POST /api/v1/accounts

Request mẫu:

```
{ "bank_name": "TPBank",  
  "account_type": "Checking",  
  "branch_name": "Quan 4",  
  "account_number_full": "9704221122334455667",  
  "balance": 2500000 }
```

Response thành công:

```
{ "message": "Account created successfully",  
  "account": {  
    "id": 9,  
    "user_id": 1,  
    "bank_name": "TPBank",  
    "account_type": "Checking",  
    "branch_name": "Quan 4",  
    "account_number_last_4": "5667",  
    "balance": 2500000 }}
```

1. Prompt A: Sinh Mã Backend (NestJS Service/Controller)

Mục tiêu: Xây dựng dịch vụ API, logic nghiệp vụ, và xử lý lỗi server cho chức năng thêm tài khoản.

Tạo endpoint POST /api/v1/accounts trong AccountsController NestJS. Endpoint này phải được bảo vệ bằng *JwtAuthGuard*.

Logic xử lý trong AccountsService:

1. Nhận DTO chứa dữ liệu tài khoản và `user_id` từ `request.user` (*JwtAuthGuard*).
2. Dùng *ValidationPipe* để xác thực DTO:

bank_name không được trống, account_number_full không được trống, balance là số và ≥ 0

3. Kiểm tra xem account_number_full đã tồn tại với người dùng này chưa.
4. Nếu hợp lệ và không trùng lặp, lưu thông tin tài khoản mới vào bảng Accounts liên kết với user_id.
5. Trả về đối tượng tài khoản vừa tạo.

Xử lý lỗi:

- Dữ liệu không hợp lệ: ValidationPipe tự động throw BadRequestException (400).
- Số tài khoản trùng: throw ConflictException (409) với

```
{ "statusCode": 409,  
  "message": "Tài khoản này đã tồn tại trong danh sách của bạn."}
```

- Lỗi lưu vào cơ sở dữ liệu: throw InternalServerErrorException (500) với

```
{ "statusCode": 500,  
  "message": "Không thể thêm tài khoản lúc này.  
  Vui lòng thử lại sau." }
```

2. Prompt B: Sinh Mã Frontend (React Component & UI/MCP)

Mục tiêu: Xây dựng giao diện thêm tài khoản chính xác theo thiết kế.

Tạo component AddAccountForm bằng React TypeScript và Tailwind CSS. Component này phải hiển thị form với các trường: Ngân hàng (bank_name), Loại tài khoản (account_type), Chi nhánh (branch_name, tùy chọn), Số tài khoản đầy đủ, Số dư khởi tạo (balance, number), Nút *Thêm tài khoản*

Thiết kế strict theo Figma MCP: bố cục, font chữ, màu sắc 100% khớp thiết kế.

3. Prompt C: Sinh Mã Frontend Logic (State & Kết nối API)

Mục tiêu: Kết nối UI với API và xử lý luồng thành công.

Trong component `AddAccountForm`:

- Tạo state cho các trường form:
`bank_name`, `account_type`, `branch_name`, `account_number_full`, `balance`.
- Triển khai hàm bất đồng bộ `handleSubmit` gửi request tới `POST /api/v1/accounts` với payload từ state.

Hành động sau khi thành công:

1. Hiển thị thông báo toast: "Thêm tài khoản thành công".
2. Điều hướng người dùng về trang danh sách tài khoản (`/accounts`).
3. (Tùy chọn) Cập nhật state global để thêm tài khoản mới mà không cần tải lại trang.

4. Prompt D: Xử lý Ngoại lệ (Client-side Validation & Error Handling)

Mục tiêu: Hoàn thiện việc xử lý lỗi, trạng thái Loading, và Validation FE.

1. **Trạng thái Loading:** Thêm state `isLoading`. Khi submit, set `isLoading = true` và vô hiệu hóa nút *Thêm tài khoản*. Sau khi API hoàn tất, set lại `isLoading = false`.
2. **Lỗi API:**
 - 409 Conflict: "Tài khoản này đã tồn tại trong danh sách của bạn."
 - 500 Internal Server Error: "Không thể thêm tài khoản lúc này. Vui lòng thử lại sau."
 - 400 Validation Error: hiển thị thông báo chi tiết bên dưới trường nhập liệu bị lỗi.
3. **Validation FE:** Trước khi gọi API:
 - 'Ngân hàng' và 'Số tài khoản' không được để trống.
 - 'Số dư khởi tạo' phải là số và ≥ 0 .

A.0.2 Chỉnh sửa tài khoản

Đặc tả ca sử dụng

Bảng 1.2 mô tả ca sử dụng Chỉnh sửa tài khoản, giúp người dùng cập nhật thông tin của tài khoản hiện có.

Bảng 1.2: Đặc tả ca sử dụng Chỉnh sửa thông tin tài khoản

Tên ca sử dụng	Chỉnh sửa thông tin tài khoản
Mô tả	• Cho phép người dùng cập nhật các trường thông tin của một tài khoản ngân hàng hoặc ví điện tử hiện có, bao gồm tên ngân hàng, loại tài khoản, chi nhánh, hoặc số dư, nhằm duy trì tính chính xác của dữ liệu trong hệ thống.
Tác nhân	Người dùng (User)
Luồng chính	• Người dùng truy cập trang "Tài khoản ngân hàng". • Người dùng chọn tài khoản muốn chỉnh sửa và nhấn "Chi tiết". • Tại trang "Chi tiết tài khoản", nhấn nút "Chỉnh sửa". • Hệ thống hiển thị form chỉnh sửa với dữ liệu hiện tại. • Người dùng thay đổi các trường cần thiết (branch_name, account_type, balance, v.v.). • Nhấn nút "Lưu thay đổi". • Front-end gọi API Backend: PUT /v1/accounts/:id với dữ liệu đã chỉnh sửa. • Backend xác thực access_token và trích xuất user_id. • Backend kiểm tra quyền sở hữu tài khoản. • Backend kiểm tra tính hợp lệ của dữ liệu mới (balance ≥ 0, số tài khoản hợp lệ). • Nếu hợp lệ, Backend cập nhật thông tin vào bảng Accounts. • Backend trả về thông tin tài khoản đã được cập nhật thành công (HTTP 200 OK). • Front-end hiển thị thông báo "Cập nhật thành công" và cập nhật giao diện chi tiết tài khoản.

Luồng thay thế	A.1. Dữ liệu đầu vào không hợp lệ: • Backend trả về lỗi Bad Request (HTTP 400) kèm chi tiết. • Front-end hiển thị thông báo lỗi dưới trường bị lỗi. • Người dùng vẫn ở màn hình chỉnh sửa để sửa lại. A.2. Lỗi ủy quyền (không có quyền chỉnh sửa): • Nếu tài khoản không thuộc sở hữu user_id: Backend trả về lỗi Forbidden (HTTP 403). • Front-end hiển thị thông báo "Bạn không có quyền chỉnh sửa thông tin tài khoản này." A.3. Lỗi hệ thống Backend: • Nếu xảy ra lỗi khi cập nhật cơ sở dữ liệu: hiển thị thông báo "Đã xảy ra lỗi khi lưu dữ liệu. Vui lòng thử lại sau."
Tiền điều kiện	• Người dùng phải đăng nhập. • Tài khoản cần chỉnh sửa phải tồn tại và thuộc sở hữu của người dùng. • Người dùng đang ở trang "Chi tiết tài khoản" của tài khoản đó.
Hậu điều kiện	• Thành công: Thông tin tài khoản được cập nhật, hiển thị thông báo thành công. • Thất bại: Hiển thị thông báo lỗi về dữ liệu không hợp lệ hoặc lỗi hệ thống, người dùng vẫn ở màn hình chỉnh sửa.

Tích hợp UI Figma:

Phù hợp với giao diện trang Chi tiết tài khoản, thiết kế form, có các trường để điền thông tin cần chỉnh sửa: Ngân hàng, Loại tài khoản, Chi nhánh, Số tài khoản đầy đủ, Số dư hiện tại (VND), [Hủy] [Lưu thay đổi].

Endpoint:

PUT /api/v1/accounts/:id

Request mẫu:

```
{"bank_name": "Vietcombank",  
  "account_type": "Checking",  
  "branch_name": "Quan 3",  
  "account_number_full": "9704221234567890123",  
  "account_number_last_4": "0123",  
  "balance": 4500000}
```

Response thành công:

```
{"message": "Account updated successfully",  
  "account": {  
    "account_id": 1,  
    "user_id": 1,  
    "bank_name": "Vietcombank",  
    "account_type": "Checking",  
    "branch_name": "Quan 3",  
    "account_number_full": "9704221234567890123",  
    "account_number_last_4": "0123",  
    "balance": 4500000,  
    "updated_at": "2025-11-01T15:25:00.000Z"}}
```

1. Prompt A: Sinh Mã Backend (NestJS Service/Controller)

Mục tiêu: Xây dựng dịch vụ API, logic nghiệp vụ, và xử lý lỗi server cho chức năng chỉnh sửa tài khoản.

Tạo endpoint PUT `/api/v1/accounts/:id` trong `AccountsController` NestJS. Endpoint này phải được bảo vệ bằng `JwtAuthGuard`.

Logic xử lý trong AccountsService:

1. Trích xuất `user_id` từ `request.user` (`JwtAuthGuard`).
2. Tìm tài khoản ngân hàng theo `:id`. Nếu không tìm thấy, throw `NotFoundException`.
3. Xác thực quyền sở hữu: kiểm tra `account.user_id === user_id`.

4. Xác thực dữ liệu đầu vào (DTO): các trường bắt buộc không rỗng, $\text{balance} \geq 0$.
5. Cập nhật thông tin tài khoản trong cơ sở dữ liệu.
6. Trả về đối tượng tài khoản đã được cập nhật.

Xử lý lỗi:

- Dữ liệu không hợp lệ: `BadRequestException` (400) với

```
{ "statusCode": 400,  
  "message": ["balance must not be less than 0"],  
  "error": "Bad Request" }
```

- Người dùng không sở hữu tài khoản: `ForbiddenException` (403) với

```
{ "statusCode": 403,  
  "message": "Bạn không có quyền chỉnh sửa thông tin tài khoản này.",  
  "error": "Forbidden" }
```

- Lỗi cập nhật cơ sở dữ liệu: `InternalServerErrorException` (500) với

```
{ "statusCode": 500,  
  "message": "Đã xảy ra lỗi khi lưu dữ liệu. Vui lòng thử lại sau.",  
  "error": "Internal Server Error" }
```

2. Prompt B: Sinh Mã Frontend (React Component & UI/MCP)

Mục tiêu: Xây dựng giao diện chỉnh sửa tài khoản chính xác theo thiết kế.

Tạo component `AccountEditForm` bằng React TypeScript và Tailwind CSS. Component này phải hiển thị:

- Tiêu đề: *Chỉnh sửa tài khoản [Tên ngân hàng]*
- Form với các trường: Ngân hàng, Loại tài khoản, Chi nhánh, Số tài khoản đầy đủ, Số dư hiện tại.

- Hai nút: *Hủy* và *Lưu thay đổi*.

Thiết kế strict theo Figma MCP, đảm bảo bố cục, khoảng cách và màu sắc khớp 100%.

3. Prompt C: Sinh Mã Frontend Logic (State & Kết nối API)

Mục tiêu: Kết nối UI với API và xử lý luồng thành công.

Trong component `AccountEditForm`:

- Tạo state quản lý dữ liệu form: `formData`, `isLoading`, `error`.
- Triển khai hàm bất đồng bộ `handleSaveChanges` gửi request `PUT /api/v1/accounts/:id` với payload từ state.

Hành động sau khi thành công:

1. Hiển thị thông báo toast: "Cập nhật thành công".
2. Cập nhật lại state chi tiết tài khoản với dữ liệu mới nhận về.
3. Đóng form/modal chỉnh sửa.

4. Prompt D: Xử lý Ngoại lệ (Client-side Validation & Error Handling)

Mục tiêu: Hoàn thiện việc xử lý lỗi, trạng thái Loading, và Validation FE.

1. **Trạng thái Loading:** Khi `isLoading = true`, vô hiệu hóa nút 'Lưu thay đổi' và hiển thị spinner.
2. **Lỗi API:**
 - 400 (Bad Request): hiển thị thông báo lỗi từ API ngay bên dưới trường nhập liệu.
 - 403 (Forbidden): toast "Bạn không có quyền chỉnh sửa thông tin tài khoản này."

- 500 (Server Error): toast "Đã xảy ra lỗi khi lưu dữ liệu. Vui lòng thử lại sau."

3. Validation FE:

- Trường 'Ngân hàng', 'Loại tài khoản' không được để trống.
- 'Số dư hiện tại' phải là số và ≥ 0 .

A.0.3 Xóa tài khoản ngân hàng

Đặc tả ca sử dụng

Bảng 1.3 mô tả chi tiết ca sử dụng Xóa tài khoản ngân hàng, cho phép người dùng loại bỏ một tài khoản không còn sử dụng.

Bảng 1.3: Đặc tả ca sử dụng Xóa thông tin tài khoản

Tên ca sử dụng	Xóa tài khoản và giao dịch liên quan
Mô tả	• Cho phép người dùng loại bỏ hoàn toàn một tài khoản ngân hàng hoặc ví điện tử không còn sử dụng. Thao tác này sẽ xóa vĩnh viễn tài khoản đó và tất cả các giao dịch đã được ghi nhận thông qua tài khoản này.
Tác nhân	Người dùng (User)
Luồng chính	• Người dùng truy cập trang "Tài khoản của tôi"(danh sách tất cả tài khoản). • Người dùng nhấn nút "Xóa"trên một tài khoản cụ thể hoặc vào "Chi tiết tài khoản"và bấm "Xóa". • Hệ thống hiển thị hộp thoại xác nhận cảnh báo việc xóa vĩnh viễn tài khoản và tất cả giao dịch liên quan. • Người dùng xác nhận muốn xóa. • Front-end gọi API Backend: DELETE api/v1/accounts/:id. • Backend xác thực access_token và trích xuất user_id. • Backend kiểm tra quyền sở hữu tài khoản. • Nếu hợp lệ, Backend thực hiện xóa tất cả giao dịch liên quan khỏi bảng Transactions và xóa tài khoản khỏi bảng Accounts. • Backend trả về phản hồi thành công (HTTP 200 OK).

	• Front-end cập nhật lại danh sách tài khoản hiển thị (tài khoản vừa xóa sẽ biến mất).
Luồng thay thế	A.1. Người dùng hủy bỏ thao tác: • Sau hộp thoại xác nhận, nếu người dùng nhấn "Hủy" hoặc đóng hộp thoại, thao tác xóa bị hủy và người dùng vẫn ở trang danh sách tài khoản. A.2. Lỗi ủy quyền (không có quyền xóa): • Nếu tài khoản không thuộc sở hữu user_id: Backend trả về lỗi Forbidden (HTTP 403), Front-end hiển thị thông báo lỗi "Bạn không có quyền thực hiện thao tác này." A.3. Lỗi hệ thống Backend: • Nếu xảy ra lỗi cơ sở dữ liệu hoặc lỗi xử lý khi xóa, hệ thống rollback các thay đổi (nếu có) và hiển thị thông báo "Đã xảy ra lỗi hệ thống, không thể xóa tài khoản và giao dịch liên quan."
Tiền điều kiện	• Người dùng phải đăng nhập. • Người dùng đang ở trang "Tài khoản của tôi". • Tài khoản cần xóa phải tồn tại và thuộc sở hữu người dùng.
Hậu điều kiện	• Thành công: Tài khoản và tất cả giao dịch liên quan được xóa, Front-end cập nhật danh sách tài khoản. • Thất bại: Hiển thị thông báo lỗi, tài khoản và giao dịch không bị thay đổi.

Tích hợp UI Figma:

Phù hợp với giao diện trang Tài khoản của tôi.

Endpoint:

DELETE /api/v1/accounts/:id

Response thành công:

```
{"message": "Account deleted successfully",
```

```
"deleted_account_id": 1}
```

Response khi lỗi:

```
{"statusCode": 404,  
  "message": "Account not found or not owned by current user"}
```

1. Prompt A: Sinh Mã Backend (NestJS Service/Controller)

Mục tiêu: Xây dựng dịch vụ API, logic nghiệp vụ, và xử lý lỗi server cho chức năng xóa tài khoản.

Tạo endpoint DELETE `/api/v1/accounts/:id` trong `AccountsController` NestJS. Endpoint này phải được bảo vệ bằng `JwtAuthGuard`.

Logic xử lý trong AccountsService:

1. Nhận `accountId` từ path parameter và `userId` từ `request.user` (`JwtAuthGuard`).
2. Tìm tài khoản trong cơ sở dữ liệu theo `accountId`.
3. Kiểm tra quyền sở hữu: nếu tài khoản tồn tại nhưng `account.userId !== userId`, throw lỗi.
4. Thực hiện xóa trong một transaction:
 - Xóa tất cả bản ghi liên quan trong bảng `Transactions` với `accountId`.
 - Sau đó, xóa bản ghi tài khoản trong bảng `Accounts`.
5. Trả về kết quả thành công.

Xử lý lỗi:

- Tài khoản không tồn tại hoặc không thuộc sở hữu người dùng: `NotFoundException` (404) với JSON:

```
{"statusCode": 404,  
  "message": "Account not found or not owned by current user"}
```

- Lỗi cơ sở dữ liệu khi xóa: `InternalServerErrorException` (500) với message: "Đã xảy ra lỗi hệ thống, không thể xóa tài khoản và giao dịch liên quan."

2. Prompt B: Sinh Mã Frontend (React Component & UI/MCP)

Mục tiêu: Xây dựng giao diện xác nhận xóa tài khoản chính xác theo Figma MCP.

Tạo component `DeleteAccountModal` bằng React TypeScript và Tailwind CSS với các phần tử:

- Tiêu đề: "Xác nhận xóa tài khoản"
- Biểu tượng cảnh báo
- Nội dung cảnh báo động: "CẢNH BÁO: Bạn có chắc chắn muốn xóa tài khoản [Tên Ngân hàng] - [**** Số cuối] không? Thao tác này sẽ xóa VĨNH VIỄN tài khoản và TẤT CẢ giao dịch liên quan."
- Hai nút hành động:
 - "Hủy" để đóng modal
 - "Xác nhận Xóa" (màu đỏ) thực hiện hành động

Thiết kế strict theo Figma MCP, đảm bảo bố cục, font chữ, màu sắc và khoảng cách khớp 100%.

3. Prompt C: Sinh Mã Frontend Logic (State & Kết nối API)

Mục tiêu: Kết nối UI với API và xử lý luồng thành công.

Trong component `DeleteAccountModal`: Tạo state quản lý: `isLoading`, `error`. Triển khai hàm bất đồng bộ `handleDeleteAccount(accountId)` gửi request `DELETE /api/v1/accounts/:id`.

Hành động sau khi thành công:

1. Đóng modal `DeleteAccountModal`.
2. Hiển thị toast/notification: "Đã xóa tài khoản thành công."
3. Cập nhật lại state global hoặc gọi lại hàm `fetch` danh sách tài khoản để loại bỏ tài khoản vừa xóa khỏi giao diện.

4. Prompt D: Xử lý Ngoại lệ (Client-side Validation & Error Handling)

Mục tiêu: Hoàn thiện việc xử lý lỗi, trạng thái Loading, và Validation FE.

1. Trạng thái Loading:

- Trước khi gọi API, set `isLoading = true`.
- Sau khi API hoàn thành, set `isLoading = false`.
- Khi `isLoading = true`, vô hiệu hóa cả hai nút và hiển thị spinner trong nút "Xác nhận Xóa".

2. **Lỗi API:** Bắt lỗi bằng `try...catch` và hiển thị thông báo lỗi chính xác trả về từ backend (ví dụ: 403, 404, 500) ngay bên trong modal, phía trên các nút.

3. **Validation FE:** Không yêu cầu validation đầu vào từ client trong ca sử dụng này.

A.0.4 Xem chi tiết tài khoản ngân hàng bất kì

Đặc tả ca sử dụng

Bảng 1.4 mô tả ca sử dụng Xem chi tiết tài khoản ngân hàng, hiển thị thông tin chi tiết và các giao dịch gần nhất của tài khoản.

Bảng 1.4: Đặc tả ca sử dụng Xem chi tiết thông tin tài khoản

Tên ca sử dụng	Xem chi tiết tài khoản
Mô tả	• Cho phép người dùng xem tất cả thông tin chi tiết của một tài khoản ngân hàng hoặc ví điện tử cụ thể, bao gồm thông tin cơ bản về tài khoản và danh sách các giao dịch gần đây nhất.
Tác nhân	Người dùng (User)
Luồng chính	• Người dùng truy cập trang "Tài khoản ngân hàng"(danh sách tài khoản). • Người dùng chọn một tài khoản và nhấn nút "Chi tiết". • Front-end gọi API Backend.

	• Backend xác thực access_token và trích xuất user_id. • Backend kiểm tra quyền sở hữu tài khoản. • Backend truy vấn thông tin chi tiết tài khoản và 5 giao dịch gần nhất. • Backend trả về dữ liệu chi tiết tài khoản và danh sách giao dịch. • Front-end hiển thị trang chi tiết tài khoản lên giao diện.
Luồng thay thế	A.1. Không tìm thấy tài khoản (ID không hợp lệ): • Backend trả về lỗi Not Found (HTTP 404). • Front-end hiển thị thông báo "Không tìm thấy tài khoản này." và chuyển hướng người dùng về trang danh sách tài khoản. A.2. Tài khoản không có giao dịch: • Nếu danh sách giao dịch rỗng, hiển thị thông tin chi tiết bình thường và thông báo "Chưa có giao dịch nào được ghi nhận cho tài khoản này." A.3. Lỗi ủy quyền (không có quyền xem): • Nếu tài khoản không thuộc sở hữu user_id: Backend trả về lỗi Forbidden (HTTP 403). • Front-end hiển thị thông báo "Bạn không có quyền xem thông tin tài khoản này."
Tiền điều kiện	• Người dùng phải đăng nhập. • Tài khoản cần xem chi tiết phải tồn tại và thuộc sở hữu người dùng hiện tại.
Hậu điều kiện	• Thành công: Hiển thị trang chi tiết tài khoản, bao gồm thông tin cơ bản và danh sách các giao dịch gần nhất. • Thất bại: Hiển thị thông báo lỗi, người dùng vẫn ở trang trước hoặc được chuyển hướng về danh sách tài khoản.

<https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits--Community-?node-id=416-7878&t=cxgXQfkGoLMfxW48-4>

API Backend: Xem chi tiết tài khoản ngân hàng

Endpoint: GET /api/v1/accounts/:id

Header: Authorization: Bearer <access_token>

Response thành công:

```
{
  "id": 1,
  "bank_name": "Vietcombank",
  "account_type": "Checking",
  "branch_name": "Quan 1",
  "account_number_full": "9704221234567890123",
  "balance": 5200000,
  "recent_transactions": [
    {
      "date": "2025-10-31",
      "amount": -500000,
      "description": "Chuyen tu Momo",
      "status": "Complete",
      "receipt_id": "RCP001",
      "type": "Expense"
    },
    {
      "date": "2025-10-30",
      "amount": 1000000,
      "description": "Chuyen tu Momo",
      "status": "Complete",
      "receipt_id": "RCP002",
      "type": "Expense"
    }
  ]
}
```

Xử lý lỗi:

- Không tìm thấy tài khoản: NotFoundException (404)

```
{
```

```
"statusCode": 404,  
"message": "Không tìm thấy tài khoản này.",  
"error": "Not Found"  
}
```

- Tài khoản không thuộc sở hữu người dùng: `ForbiddenException` (403)

```
{"statusCode": 403,  
  "message": "Bạn không có quyền xem thông tin tài khoản này.",  
  "error": "Forbidden"}
```

1. Prompt A: Sinh Mã Backend (NestJS Service/Controller)

Mục tiêu: Xây dựng dịch vụ API, logic nghiệp vụ và xử lý lỗi server.

Logic xử lý trong `AccountsService`:

1. Nhận `accountId` từ path param và `userId` từ `req.user` (`JwtAuthGuard`).
2. Truy vấn cơ sở dữ liệu để tìm tài khoản theo `accountId`.
3. Nếu không tìm thấy tài khoản, throw `NotFoundException`.
4. Kiểm tra quyền sở hữu: nếu `account.userId !== userId`, throw `ForbiddenException`.
5. Nếu xác thực thành công, truy vấn 5 giao dịch gần đây nhất liên quan đến `accountId`.
6. Kết hợp thông tin tài khoản và danh sách giao dịch thành một object duy nhất và trả về.

2. Prompt B: Sinh Mã Frontend (React Component & UI/MCP)

Mục tiêu: Xây dựng giao diện chính xác theo thiết kế Figma.

Tạo component `AccountDetailComponent` bằng React TypeScript và Tailwind CSS, bao gồm:

- Khu vực thẻ (card) hiển thị thông tin chi tiết tài khoản: tên ngân hàng, số dư, loại tài khoản, chi nhánh, số tài khoản đầy đủ.
- Khu vực bảng (table) hoặc danh sách (list) hiển thị 5 giao dịch gần đây với các cột: Ngày, Mô tả, Số tiền, Trạng thái.

STRICTLY thiết kế theo Figma MCP: <https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits--Community-?node-id=416-7878&t=cxgXQfkGoLMfxW48-4>

3. Prompt C: Sinh Mã Frontend Logic (State & Kết nối API)

Mục tiêu: Kết nối UI với API và xử lý luồng thành công.

Trong AccountDetailComponent:

- Thêm state: `accountData`, `isLoading`, `error`.
- Triển khai hàm bất đồng bộ `fetchAccountDetails(accountId)` gửi request `GET /api/v1/accounts/:id` kèm header `Authorization`.
- Khi API trả về thành công: Gọi `setAccountData(responseData)`, Đặt `setIsLoading(false)`, Xóa thông báo lỗi cũ: `setError(null)`

4. Prompt D: Xử lý Ngoại lệ (Client-side Validation & Error Handling)

Mục tiêu: Hoàn thiện xử lý lỗi, trạng thái Loading và hiển thị thông tin.

1. **Trạng thái Loading:** Trước khi gọi API, `setIsLoading(true)`. Khi `isLoading = true`, hiển thị Spinner hoặc Skeleton thay cho nội dung.
2. **Lỗi API:** Bọc API trong `try...catch`. Hiển thị thông báo lỗi dựa vào status code:
 - 404: "Không tìm thấy tài khoản này."
 - 403: "Bạn không có quyền xem thông tin tài khoản này."
 - Các lỗi khác: "Đã có lỗi xảy ra, không thể tải dữ liệu."
3. **Validation FE:** Không cần validation đầu vào. Nếu mảng giao dịch rỗng, hiển thị: "Chưa có giao dịch nào được ghi nhận cho tài khoản này."

A.0.5 Xem Chi tiêu Hàng tháng

Đặc tả ca sử dụng

Bảng 1.5 trình bày ca sử dụng Xem Chi tiêu Hàng tháng, cung cấp biểu đồ tổng quan mức chi tiêu theo từng tháng.

Bảng 1.5: Đặc tả ca sử dụng Xem chi tiêu hàng tháng

Tên ca sử dụng	Xem chi tiêu hàng tháng
Mô tả	• Cho phép người dùng trực quan hóa và so sánh tổng chi tiêu của tháng hiện tại với các tháng trước đó trong năm thông qua biểu đồ, nhằm nắm bắt xu hướng và thói quen tài chính.
Tác nhân	Người dùng (User)
Luồng chính	• Tại Trang chủ, người dùng truy cập vào trang "Chi tiêu" trên thanh điều hướng. • Front-end gọi API Backend để lấy dữ liệu tổng chi tiêu theo tháng. • Backend xác thực access_token và trích xuất user_id. • Backend truy vấn bảng Transactions để lấy tổng số tiền chi tiêu (SUM(Amount)) theo từng tháng với type = 'Expense' và user_id hiện tại trong phạm vi 12 tháng. • Backend trả về dữ liệu tổng chi tiêu theo định dạng Tháng - Tổng số tiền. • Front-end nhận dữ liệu và vẽ Biểu đồ cột so sánh chi tiêu. • Trục X hiển thị các tháng trong năm, trục Y hiển thị số tiền chi tiêu. • Cột biểu đồ: tháng hiện tại màu xanh lá, các tháng trước màu xám. • Giao diện hiển thị biểu đồ hoàn chỉnh.
Luồng thay thế	A.1. Không có dữ liệu chi tiêu: • Nếu truy vấn không trả về dữ liệu: hiển thị thông báo "Chưa có dữ liệu chi tiêu nào được ghi nhận để phân tích." A.2. Dữ liệu chỉ có cho tháng hiện tại:

	• Biểu đồ chỉ hiển thị cột màu xanh lá cho tháng hiện tại, có thể hiển thị thông báo khuyến khích ghi nhận giao dịch. A.3. Lỗi hệ thống / lỗi truy cập: • Nếu xảy ra lỗi khi truy vấn hoặc xác thực: Backend trả về lỗi (HTTP 500), Front-end hiển thị thông báo lỗi chung và không thể vẽ biểu đồ.
Tiền điều kiện	• Người dùng phải đăng nhập. • Người dùng đã ghi nhận các giao dịch chi tiêu (type = 'Expense') trong tháng hiện tại và các tháng trước.
Hậu điều kiện	• Thành công: Hiển thị biểu đồ so sánh chi tiêu hàng tháng trên trang "Chi tiêu". • Thất bại: Hiển thị thông báo lỗi hoặc thông báo không có dữ liệu chi tiêu.

Tích hợp UI Figma:

<https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits--Community-?node-id=66-5698&t=cxgXQfkGoLMfxW48-4>

Endpoint:

GET /api/v1/expenses/summary?userId={user_id}

Header: Authorization: Bearer <access_token>

Response thành công:

```
{
  "data": [
    { "month": "Jan", "totalExpense": 1250.00 },
    { "month": "Feb", "totalExpense": 2100.50 },
    { "month": "Mar", "totalExpense": 1900.00 }
  ]
}
```

Xử lý lỗi:

```
{ "error": "Cannot load data for Expense." }
```

1. Prompt A: Sinh Mã Backend (NestJS Service/Controller)

Mục tiêu: Xây dựng dịch vụ API, logic nghiệp vụ và xử lý lỗi server.

Logic xử lý trong ExpensesService:

1. Nhận user_id từ payload JWT (@Request()).
2. Truy vấn bảng Transactions trong cơ sở dữ liệu.
3. Lọc giao dịch theo: type = 'Expense', user_id = current user, Năm giao dịch = năm hiện tại
4. Nhóm theo tháng (GROUP BY month) và tính tổng (SUM(amount)).
5. Chuyển đổi kết quả thành mảng đối tượng: { month: string, totalExpense: number } với tên tháng viết tắt.
6. Trả về JSON với key data.

Xử lý lỗi: Nếu truy vấn DB thất bại, throw InternalServerErrorException với:

```
{ "error": "Cannot load data for Expense" }
```

2. Prompt B: Sinh Mã Frontend (React Component & UI/MCP)

Mục tiêu: Xây dựng giao diện theo thiết kế Figma.

Tạo component ExpenseSummaryChart (React TypeScript + Tailwind CSS) gồm: Tiêu đề khu vực biểu đồ. Trục X: Tháng trong năm. Trục Y: Tổng chi tiêu. Cột biểu đồ với tooltip hiển thị số tiền khi hover.

STRICTLY theo thiết kế Figma MCP: <https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits--Community-?node-id=66-5698&t=cxgXQfkGoLMfxW48-4>

Màu sắc cột: Tháng hiện tại màu xanh lá. Các tháng trước màu xám.

3. Prompt C: Sinh Mã Frontend Logic (State & Kết nối API)

Mục tiêu: Kết nối UI với API và xử lý dữ liệu.

- Thêm state: `summaryData`: dữ liệu biểu đồ, `isLoading`: trạng thái tải, `error`: thông báo lỗi
- Hàm bất đồng bộ `fetchExpenseSummary` bên trong `useEffect`:
 1. Gọi API GET `/api/v1/expenses/summary` với `Authorization: Bearer <access_token>`
 2. Nếu thành công, cập nhật `summaryData` với `response.data`
 3. Render biểu đồ dựa vào `summaryData`, cột tháng hiện tại màu xanh lá, các tháng khác màu xám

4. Prompt D: Xử lý Ngoại lệ (Client-side Validation & Error Handling)

Mục tiêu: Hoàn thiện xử lý lỗi và trạng thái Loading.

1. Loading:

- Trước khi gọi API, set `isLoading = true`
- Trong khối `finally`, set `isLoading = false`
- Khi `isLoading = true`, hiển thị `ChartSkeletonLoader` hoặc `Spinner`

2. Lỗi API:

- Bọc request trong `try...catch`
- Nếu lỗi (500), set `error = error.response.data.error` và hiển thị ở khu vực biểu đồ

3. Trường hợp mảng rỗng:

- Nếu `data.length === 0`, hiển thị: “Chưa có dữ liệu chi tiêu nào được ghi nhận để phân tích.”

- Nếu chỉ có một phần tử (tháng hiện tại), vẫn hiển thị biểu đồ với một cột màu xanh lá

4. **Validation FE:** Không áp dụng vì không có form đầu vào

A.0.6 Xem Chi tiết Chi tiêu theo Danh mục

Đặc tả ca sử dụng

Bảng 1.6 mô tả chi tiết ca sử dụng Xem Chi tiêu theo Danh mục, nhóm các khoản chi theo từng loại để người dùng tiện theo dõi.

Bảng 1.6: Đặc tả ca sử dụng Xem chi tiết chi tiêu theo danh mục

Tên ca sử dụng	Xem chi tiết chi tiêu theo danh mục
Mô tả	• Cho phép người dùng phân tích tổng chi tiêu trong một khoảng thời gian cụ thể (thường là tháng hiện tại) bằng cách chia nhỏ số tiền đã chi theo các danh mục đã thiết lập (Housing, Food, Entertainment, v.v.).
Tác nhân	Người dùng (User)
Luồng chính	• Người dùng truy cập vào trang "Chi tiêu" từ thanh điều hướng. • Front-end gọi API Backend để lấy chi tiết chi tiêu nhóm theo danh mục. • Backend xác thực access_token và trích xuất user_id. • Backend truy vấn bảng Transactions (type = 'Expense') kết hợp bảng Categories để nhóm tổng chi tiêu theo category_id cho tháng hiện tại và tháng trước. • Backend trả về danh sách các danh mục, tổng chi tiêu tháng hiện tại, tổng chi tiêu tháng trước, và các item giao dịch nhỏ lẻ cho mỗi danh mục. • Front-end hiển thị danh sách chi tiết chi tiêu theo từng danh mục: Tên danh mục và icon, tổng số tiền chi trong tháng hiện tại, tỷ lệ tăng/giảm so với tháng trước.

	Dưới mỗi danh mục, hiển thị danh sách các Item giao dịch nhỏ thuộc danh mục đó trong tháng hiện tại.
Luồng thay thế	A.1. Không có chi tiêu trong tháng hiện tại: - Nếu tổng chi tiêu tháng hiện tại bằng 0, hiển thị thông báo: "Bạn chưa có chi tiêu nào được ghi nhận trong tháng này." A.2. Lỗi hệ thống / lỗi truy cập: - Nếu xảy ra lỗi khi truy vấn hoặc xác thực: Backend trả về lỗi (HTTP 500), Front-end hiển thị thông báo lỗi chung và không thể phân tích dữ liệu.
Tiền điều kiện	- Người dùng phải đăng nhập. - Người dùng đã ghi nhận các giao dịch chi tiêu (type = 'Expense') và các giao dịch này đã được gán category_id hợp lệ. - Hệ thống có dữ liệu chi tiêu ít nhất tháng hiện tại để tính tỷ lệ so sánh.
Hậu điều kiện	- Thành công: Hiển thị trang Chi tiết chi tiêu với danh sách các danh mục, tổng chi tiêu của từng danh mục, và tỷ lệ so sánh với tháng trước. - Thất bại: Hiển thị thông báo lỗi hoặc thông báo không có dữ liệu chi tiêu.

Tích hợp UI Figma:

<https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits--Community-?node-id=66-5698&t=cxgXQfkGoLMfxW48-4>

Endpoint:

GET /api/v1/expenses/breakdown?month=YYYY-MM

Header: Authorization: Bearer <access_token>

Query Parameters: month - định dạng 'YYYY-MM', ví dụ: '2025-11'

Response thành công:

```
{ "data": [  
  {  
    "category": "Housing",  
    "total": 250.00,  
    "changePercent": 15,  
    "subCategories": [  
      { "item_description":  
        "House Rent", "amount": 230.00, "date": "2025-05-17" },  
      { "item_description":  
        "Parking", "amount": 20.00, "date": "2025-05-17" }  
    ]  
  }, {  
    "category": "Food",  
    "total": 350.00,  
    "changePercent": -8,  
    "subCategories": [  
      { "item_description":  
        "Grocery", "amount": 230.00, "date": "2025-05-17" },  
      { "item_description":  
        "Restaurant Bill", "amount": 120.00, "date": "2025-05-17" }  
    ]  
  }  
]}
```

Xử lý lỗi:

```
{ "error": "No input data for this month." }
```

1. Prompt A: Sinh Mã Backend (NestJS Service/Controller)

Mục tiêu: Xây dựng dịch vụ API, logic nghiệp vụ và xử lý lỗi server.

Logic xử lý trong ExpensesService:

1. Nhận `userId` từ payload JWT (`@Request()`) và `month` từ query params.
2. Tính `previousMonth` dựa trên `month` hiện tại.
3. Thực hiện 2 truy vấn song song trên bảng `Transactions`:
 - Truy vấn các giao dịch `type='Expense'` trong tháng hiện tại.
 - Truy vấn các giao dịch `type='Expense'` trong `previousMonth`.
4. Nhóm kết quả theo `category_id` và tính tổng `total` cho mỗi danh mục.
5. Lặp qua danh mục tháng hiện tại:
 - Tính $\text{changePercent} = \frac{\text{currentTotal} - \text{previousTotal}}{\text{previousTotal}} \times 100$. Nếu `previousTotal = 0`, có thể là 100 hoặc null.
 - Lấy danh sách giao dịch con (`subCategories`) thuộc danh mục.
6. Nếu không có giao dịch trong tháng hiện tại, throw `NotFoundException`.
7. Trả về JSON với key `data`.

Xử lý lỗi: Nếu không có bất kỳ giao dịch nào, throw `NotFoundException` với:

```
{ "error": "No input data for this month." }
```

2. Prompt B: Sinh Mã Frontend (React Component & UI/MCP)

Mục tiêu: Xây dựng giao diện theo thiết kế Figma.

- Tạo component `ExpensesBreakdown` bằng React TypeScript + Tailwind CSS.
- Mỗi card hiển thị:
 - Icon và tên danh mục
 - Tổng chi tiêu của danh mục
 - Tỷ lệ phần trăm thay đổi (`changePercent`) màu xanh lá nếu tăng, đỏ nếu giảm

- Danh sách các giao dịch con, mỗi dòng gồm: `item_description`, ngày, và số tiền

STRICTLY theo thiết kế Figma MCP: <https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits--Community-?node-id=66-5698>

3. Prompt C: Sinh Mã Frontend Logic (State & Kết nối API)

Mục tiêu: Kết nối UI với API và xử lý luồng thành công.

- Thêm các state:
 - `expensesData`: dữ liệu breakdown
 - `isLoading`: trạng thái tải
 - `error`: thông báo lỗi
- Triển khai hàm bất đồng bộ `fetchExpensesBreakdown(month: string)`:
 1. Gọi API GET `/api/v1/expenses/breakdown?month=YYYY-MM` với header `Authorization`.
 2. Nếu thành công, set `expensesData = response.data`.
 3. Đặt `isLoading = false` và xóa lỗi cũ (`error = null`).

4. Prompt D: Xử lý Ngoại lệ (Client-side Validation & Error Handling)

Mục tiêu: Hoàn thiện xử lý lỗi và trạng thái Loading.

1. **Loading:** Trước khi gọi API, set `isLoading = true`. Khi đang load, hiển thị Skeleton Loader giống layout card thật.
2. **Lỗi API:** Bọc request trong `try...catch`. Nếu lỗi 404, set `error = "Không có dữ liệu chi tiêu cho tháng này."` và hiển thị thông báo trong giao diện. Nếu không có dữ liệu, thông báo là: "Bạn chưa có chi tiêu nào được ghi nhận trong tháng này."

3. **Validation FE:** Chỉ gọi `fetchExpensesBreakdown` với chuỗi `month` hợp lệ (YYYY-MM) từ Date Picker.

A.0.7 Xem Danh sách Hóa đơn sắp tới

Đặc tả ca sử dụng

Bảng 1.7 mô tả ca sử dụng Xem Danh sách Hóa đơn, hiển thị các hóa đơn định kỳ sắp đến hạn để người dùng quản lý.

Bảng 1.7: Đặc tả ca sử dụng Xem danh sách hoá đơn sắp tới

Tên ca sử dụng	Xem danh sách hóa đơn sắp tới
Mô tả	Cho phép người dùng theo dõi và quản lý các hóa đơn định kỳ (Bills) sắp đến hạn thanh toán, giúp người dùng chủ động chuẩn bị tài chính và tránh thanh toán chậm.
Tác nhân	Người dùng (User)
Luồng chính	- Người dùng truy cập trang Bills (Hóa đơn) thông qua menu điều hướng. - Front-end gọi API Backend để lấy danh sách hóa đơn sắp tới (GET /api/v1/bills). - Backend xác thực <code>access_token</code> và trích xuất <code>user_id</code>. - Backend truy vấn danh sách các hóa đơn định kỳ thuộc <code>user_id</code> với ngày đến hạn (due date) trong tương lai gần. - Danh sách hóa đơn được sắp xếp theo ngày đến hạn tăng dần (due date ASC). - Backend trả về danh sách hóa đơn, gồm: Tên gói, Logo nhà cung cấp, Mô tả, Ngày đến hạn, Ngày thanh toán cuối, và Số tiền. - Front-end nhận dữ liệu và hiển thị danh sách hóa đơn với đầy đủ thông tin chi tiết và logo tương ứng.
Luồng thay thế	A.1. Không có hóa đơn sắp đến hạn: Nếu danh sách hóa đơn trả về rỗng: hiển thị thông báo "Bạn không có hóa đơn nào sắp đến hạn thanh toán."

	• Cung cấp liên kết để Người dùng thiết lập hóa đơn mới. A.2. Lỗi hệ thống / lỗi truy cập: • Nếu xảy ra lỗi xác thực hoặc truy vấn dữ liệu: Backend trả về lỗi (HTTP 401, HTTP 500), Front-end hiển thị thông báo lỗi chung và không hiển thị danh sách hóa đơn.
Tiền điều kiện	• Người dùng phải đăng nhập. • Người dùng đã thiết lập ít nhất một hóa đơn định kỳ sắp đến hạn.
Hậu điều kiện	• Thành công: Hiển thị danh sách các hóa đơn sắp tới trên trang Hóa đơn. • Thất bại: Hiển thị thông báo lỗi hoặc thông báo không có hóa đơn sắp đến hạn.

Tích hợp UI Figma:

<https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits--Community-?node-id=66-5609>

Endpoint:

GET /api/v1/bills

Header: Authorization: Bearer <access_token>

Request Body: Không có (GET request)

Response thành công:

```
{
  "data": [
    {
      "billId": 1,
      "userId": 1,
      "itemDescription": "Figma - Yearly Plan",
      "logoUrl": "https://cdn.figma.com/logo.png",
      "dueDate": "2025-05-15",
    }
  ]
}
```

```

      "lastChargeDate": "2024-05-14",
      "amount": 150.00
    }, {
      "billId": 2,
      "userId": 1,
      "itemDescription": "Adobe Creative Cloud",
      "logoUrl": "https://cdn.adobe.com/logo.png",
      "dueDate": "2025-06-10",
      "lastChargeDate": "2024-06-10",
      "amount": 559.00
    }
  ]
}
```

Xử lý lỗi:

```
{"message": "Failed to fetch bills"}
```

1. Prompt A: Sinh Mã Backend (NestJS Service/Controller)

Mục tiêu: Xây dựng dịch vụ API, logic nghiệp vụ và xử lý lỗi server.

Logic xử lý trong BillsService:

1. Nhận `user_id` từ payload JWT (`@Request()`).
2. Truy vấn cơ sở dữ liệu để lấy tất cả hóa đơn thuộc `user_id`.
3. Lọc các hóa đơn có `dueDate` $\geq$ ngày hiện tại.
4. Sắp xếp kết quả theo `dueDate` ASC.
5. Trả về JSON với key `data`.

Xử lý lỗi: Nếu có lỗi trong quá trình truy vấn cơ sở dữ liệu, throw `InternalServerErrorException` với:

```
{"message": "Failed to fetch bills"}
```

2. Prompt B: Sinh Mã Frontend (React Component & UI/MCP)

Mục tiêu: Xây dựng giao diện chính xác theo thiết kế Figma.

- Component: UpcomingBills (React TypeScript + Tailwind CSS)
- Hiển thị danh sách hóa đơn sắp tới:
Logo nhà cung cấp, Mô tả hóa đơn (itemDescription), Ngày đến hạn (dueDate), Số tiền (amount), Nút Pay Now

3. Prompt C: Sinh Mã Frontend Logic (State & Kết nối API)

Mục tiêu: Kết nối UI với API và xử lý luồng thành công.

- Thêm các state: bills: danh sách hóa đơn, isLoading: trạng thái tải, error: thông báo lỗi
- Hàm bắt đầu bộ fetchUpcomingBills:
 1. Gọi API GET /api/v1/bills với header Authorization.
 2. Nếu thành công:
 - set bills = response.data
 - Nếu data.length === 0, hiển thị thông báo: "Bạn không có hóa đơn nào sắp đến hạn thanh toán."
 3. set isLoading = false và error = null.

4. Prompt D: Xử lý Ngoại lệ (Client-side Validation & Error Handling)

Mục tiêu: Hoàn thiện xử lý lỗi, trạng thái Loading.

1. **Loading:** Trước khi gọi API, set isLoading = true. Khi kết thúc request, set isLoading = false. Hiển thị spinner hoặc skeleton loader trong thời gian tải.
2. **Lỗi API:** Nếu API trả về lỗi (ví dụ: 500), cập nhật state error với thông báo: "Không thể tải danh sách hóa đơn. Vui lòng thử lại.". Hiển thị thông báo tại vị trí danh sách hóa đơn.

3. **Validation FE:** Không cần, vì đây là chức năng chỉ đọc dữ liệu.

A.0.8 Xem danh sách Mục tiêu

Đặc tả ca sử dụng

Bảng 1.8 trình bày ca sử dụng Xem Mục tiêu hàng tháng, hiển thị các mục tiêu tài chính và trạng thái hoàn thành.

Bảng 1.8: Đặc tả ca sử dụng Xem danh sách Mục tiêu

Tên ca sử dụng	Xem danh sách mục tiêu
Mô tả	Cho phép người dùng đã đăng nhập xem tổng quan tất cả các mục tiêu tài chính mà họ đã thiết lập trong hệ thống, bao gồm mục tiêu tiết kiệm tổng thể và giới hạn chi tiêu theo danh mục.
Tác nhân	Người dùng (User)
Luồng chính	- Người dùng đã đăng nhập và có access_token hợp lệ. - Người dùng truy cập trang Mục tiêu (Goals) từ thanh điều hướng (/goals). - Front-end gọi API Backend: GET /api/v1/goals. - Backend xác thực access_token và trích xuất user_id. - Backend truy vấn bảng Goals và Transactions dựa trên user_id, tính Target Achieved = (Tổng Thu nhập - Tổng Chi tiêu) của tháng hiện tại. - Backend trả về danh sách các mục tiêu của người dùng (bao gồm mục tiêu tiết kiệm và giới hạn chi tiêu) của tháng hiện tại. - Front-end nhận dữ liệu và hiển thị danh sách các mục tiêu chi tiết lên giao diện người dùng.
Luồng thay thế	A.1. Không có mục tiêu nào được thiết lập: Nếu danh sách mục tiêu trả về rỗng, hiển thị thông báo: "Bạn chưa thiết lập mục tiêu nào. Hãy tạo mục tiêu đầu tiên!"

	• Cung cấp nút hoặc liên kết để Người dùng bắt đầu tạo mục tiêu mới. A.2. Lỗi xác thực / lỗi truy cập: • Nếu access_token không hợp lệ hoặc đã hết hạn: Backend trả về lỗi Unauthorized (HTTP 401), Front-end chuyển hướng Người dùng đến trang Đăng nhập. A.3. Lỗi hệ thống Backend: • Nếu xảy ra lỗi truy vấn cơ sở dữ liệu hoặc lỗi xử lý nội bộ, hiển thị thông báo lỗi thân thiện: "Đã xảy ra lỗi hệ thống khi tải mục tiêu, vui lòng thử lại sau."
Tiền điều kiện	• Người dùng phải đăng nhập (có access_token hợp lệ).
Hậu điều kiện	• Thành công: Hiển thị danh sách các mục tiêu đã thiết lập trên trang Mục tiêu. • Thất bại: Hiển thị thông báo lỗi hoặc thông báo không có mục tiêu nào được thiết lập.

Tích hợp UI Figma:

<https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits--Community-?node-id=66-5829&t=cxgXQfkGoLMfxW48-4>

Mục Savings Goal hiển thị: Target Achieved, Target Amount, Button “Adjust Goal”, Mục Expenses Goals by Category hiển thị các thẻ danh mục (Housing, Food, Transportation, ...)

Endpoint:

GET /api/v1/goals

Header: Authorization: Bearer <access_token>

Request Body: Không có

Response thành công:


```
{ "savingGoal": {
  "goal_id": 1,
  "goal_type": "Saving",
  "target_amount": 60000000,
  "target_achieved": 10000000,
  "start_date": "2025-01-01",
  "end_date": "2025-12-31"
},
"expenseGoals": [
  {
    "goal_id": 2,
    "category": "Food",
    "target_amount": 3000000
  },{
    "goal_id": 3,
    "category": "Transportation",
    "target_amount": 2000000
  }]
}
```

Xử lý lỗi:

```
{"statusCode": 500,
  "message": "500 Internal Server Error. Try again."}
```

1. Prompt A: Sinh Mã Backend (NestJS Service/Controller)

Mục tiêu: Xây dựng dịch vụ API, logic nghiệp vụ và xử lý lỗi server.

1. Endpoint GET /api/v1/goals trong GoalsController và bảo vệ bằng JwtAuthGuard.
2. Lấy user_id từ payload JWT.
3. Truy vấn cơ sở dữ liệu để lấy:
 - Mục tiêu tiết kiệm (savingGoal)

- Mục tiêu chi tiêu (expenseGoals)

cho user_id trong tháng hiện tại.

4. Tính target_achieved cho savingGoal:

$\text{target_achieved} = \text{Tổng Thu Nhập (tháng hiện tại)} - \text{Tổng Chi Tiêu (tháng hiện tại)}$

5. Trả về dữ liệu theo định dạng response.
6. Nếu có lỗi trong quá trình truy vấn cơ sở dữ liệu hoặc xử lý logic, throw `InternalServerErrorException` với JSON lỗi.

2. Prompt B: Sinh Mã Frontend (React Component & UI/MCP)

Mục tiêu: Xây dựng giao diện chính xác theo thiết kế Figma.

- Component: `GoalsPage` (React TypeScript + Tailwind CSS)
- Hiển thị:
 - Khu vực "Savings Goal": Target Achieved, Target Amount, nút "Adjust Goal"
 - Khu vực "Expenses Goals by Category": danh sách thẻ mục tiêu chi tiêu theo từng danh mục
- STRICTLY theo Figma MCP: <https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits---Community-?node-id=66-5829>

3. Prompt C: Sinh Mã Frontend Logic (State & Kết nối API)

Mục tiêu: Kết nối UI với API và xử lý luồng thành công.

- Thêm các state: `goalsData` – dữ liệu mục tiêu, `isLoading` – trạng thái tải, `error` – thông báo lỗi

- Hàm bất đồng bộ `fetchGoals`:

1. Gọi API GET `/api/v1/goals` với header `Authorization`.

2. Nếu thành công:

- `set goalsData = response.data`
- `set isLoading = false`
- Nếu `savingGoal` và `expenseGoals` đều rỗng, hiển thị thông báo: "Bạn chưa thiết lập mục tiêu nào. Hãy tạo mục tiêu đầu tiên!"

4. Prompt D: Xử lý Ngoại lệ (Client-side Validation & Error Handling)

Mục tiêu: Hoàn thiện xử lý lỗi, trạng thái Loading.

1. **Loading:** Trước khi gọi API, `set isLoading = true`. Trong khi tải, hiển thị Spinner hoặc Skeleton Loader.

2. **Lỗi API:**

- Nếu lỗi 401 Unauthorized, điều hướng về trang Đăng nhập.
- Nếu lỗi hệ thống (ví dụ 500), cập nhật `error` và hiển thị: "Đã xảy ra lỗi hệ thống khi tải mục tiêu, vui lòng thử lại sau."

3. **Validation FE:** Không áp dụng, vì đây là chức năng xem dữ liệu.

A.0.9 Tạo Mục tiêu mới

Đặc tả ca sử dụng

Bảng 1.9 mô tả chi tiết ca sử dụng Tạo Mục tiêu mới, cho phép người dùng thiết lập một mục tiêu tài chính mới.

Bảng 1.9: Đặc tả ca sử dụng Tạo Mục tiêu mới

Tên ca sử dụng	Tạo mục tiêu mới
----------------	------------------

Mô tả	• Cho phép người dùng thiết lập một mục tiêu tài chính mới, có thể là Mục tiêu tiết kiệm tổng thể hoặc Giới hạn chi tiêu theo danh mục, bằng cách nhập các thông số như loại mục tiêu, số tiền và thời gian thực hiện.
Tác nhân	Người dùng (User)
Luồng chính	• Người dùng truy cập trang Mục tiêu từ thanh điều hướng và nhấn nút “Thêm mục tiêu mới”. • Hệ thống hiển thị form nhập liệu với các thông tin: Loại mục tiêu (goal_type), Danh mục (category, nếu cần), Số tiền mục tiêu (target_amount), Ngày bắt đầu (start_date), Ngày kết thúc (end_date). • Người dùng nhập thông tin và nhấn nút “Lưu” (Save). • Front-end gửi yêu cầu kèm dữ liệu đến API Backend. • Backend xác thực access_token và trích xuất user_id. • Backend kiểm tra tính hợp lệ của dữ liệu đầu vào: số tiền > 0, start_date < end_date, danh mục hợp lệ nếu cần. • Nếu hợp lệ, Backend lưu mục tiêu mới vào bảng Goals, liên kết với user_id. • Backend trả về phản hồi thành công. • Front-end nhận phản hồi, hiển thị thông báo “Tạo mục tiêu thành công” và cập nhật danh sách mục tiêu.
Luồng thay thế	A.1. Dữ liệu đầu vào không hợp lệ (Validation Error): • Nếu bất kỳ trường dữ liệu vi phạm quy tắc, Backend trả về lỗi Bad Request (HTTP 400). • Front-end hiển thị thông báo lỗi tương ứng dưới trường bị lỗi và Người dùng chỉnh sửa lại. A.2. Mục tiêu chi tiêu không chọn Danh mục: • Nếu goal_type là Giới hạn chi tiêu nhưng trường Danh mục trống, hiển thị thông báo lỗi: "Vui lòng chọn danh mục chi tiêu."
Tiền điều kiện	• Người dùng phải đăng nhập.

	• Danh mục (Categories) phải có sẵn nếu tạo mục tiêu chi tiêu.
Hậu điều kiện	• Thành công: Mục tiêu mới được lưu vào bảng Goals, hệ thống cập nhật danh sách mục tiêu. • Thất bại: Hiển thị thông báo lỗi về dữ liệu không hợp lệ, Người dùng vẫn ở form Tạo mục tiêu mới.

Tích hợp UI Figma:

<https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits--Community-?node-id=416-6052>

Endpoint:

POST /api/v1/goals

Header: Authorization: Bearer <access_token>

Request Body mẫu:

```
{ "goal_type": "Saving",
  "category_id": null,
  "start_date": "2025-05-01",
  "end_date": "2025-12-31",
  "target_amount": 80000000}
```

Response thành công (HTTP 201):

```
{"message": "Goal created successfully",
  "goal_id": 9}
```

Xử lý lỗi:

```
{"statusCode": 400,
  "message": "End date must be < start date."}
```

```
{"statusCode": 500,  
  "message": "Cannot add Goals right now. Try again."}
```

1. Prompt A: Sinh Mã Backend (NestJS Service/Controller)

Mục tiêu: Xây dựng dịch vụ API, logic nghiệp vụ và xử lý lỗi server.

1. Endpoint POST /api/v1/goals trong GoalsController, bảo vệ bằng JwtAuthGuard.
2. Nhận DTO chứa dữ liệu mục tiêu; user_id lấy từ access_token.
3. Validation: target_amount > 0, end_date sau start_date, nếu goal_type = "Expense", category_id không được null và phải hợp lệ
4. Nếu hợp lệ, tạo bản ghi mới trong bảng Goals liên kết với user_id và lưu vào DB.
5. Xử lý lỗi:
 - Validation thất bại: BadRequestException (HTTP 400)
 - Lỗi hệ thống: InternalServerErrorException (HTTP 500)

2. Prompt B: Sinh Mã Frontend (React Component & UI/MCP)

Mục tiêu: Xây dựng giao diện chính xác theo thiết kế Figma.

- Component: CreateGoalModal (React TypeScript + Tailwind CSS)
- Form trong modal:
 - Tiêu đề: "Thêm mục tiêu mới"
 - Chọn Loại mục tiêu (Radio/Dropdown: "Tiết kiệm tổng thể", "Giới hạn chi tiêu")
 - Chọn Danh mục (hiển thị nếu mục tiêu là "Giới hạn chi tiêu")
 - Nhập Số tiền mục tiêu (currency input)
 - Chọn Ngày bắt đầu và Ngày kết thúc (Date Picker)

- Nút hành động: "Lưu", "Hủy"
- STRICTLY theo Figma MCP: <https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits--Community-?node-id=416-6052>

3. Prompt C: Sinh Mã Frontend Logic (State & Kết nối API)

Mục tiêu: Kết nối UI với API và xử lý luồng thành công.

- Các state: goalType, categoryId, targetAmount, startDate, endDate, isLoading, error
- Hàm handleSubmit:
 1. Gửi request POST /api/v1/goals với payload lấy từ state.
 2. Nếu thành công:
 - Hiển thị toast: "Tạo mục tiêu thành công."
 - Đóng modal
 - Gọi callback để refresh danh sách mục tiêu

4. Prompt D: Xử lý Ngoại lệ (Client-side Validation & Error Handling)

Mục tiêu: Hoàn thiện việc xử lý lỗi, trạng thái Loading, và Validation cuối cùng.

1. Loading:

- Biến state isLoading
- Khi bắt đầu gọi API: isLoading = true, vô hiệu hóa nút "Lưu", hiển thị spinner
- Khi kết thúc API: isLoading = false

2. Lỗi API:

- Bọc trong try...catch

- Hiển thị thông báo lỗi từ `error.response.data.message` nếu có, Nếu không có, hiển thị: "Không thể tạo mục tiêu lúc này. Vui lòng thử lại sau."

3. **Validation FE:** Trước khi gọi API:

- `target_amount` phải > 0
- `start_date` và `end_date` phải được chọn
- `end_date` phải sau `start_date`
- Nếu mục tiêu là "Expense", `category_id` phải được chọn
- Nếu validation thất bại, hiển thị thông báo lỗi dưới trường tương ứng, không gọi API

A.0.10 Điều chỉnh Mục tiêu hàng tháng

Đặc tả ca sử dụng

Bảng 1.10 mô tả đặc tả ca sử dụng Điều chỉnh Mục tiêu hàng tháng, hỗ trợ cập nhật giá trị và tiến độ của mục tiêu.

Bảng 1.10: Đặc tả ca sử dụng Điều chỉnh Mục tiêu hàng tháng

Tên ca sử dụng	Điều chỉnh mục tiêu (Adjust Goal)
Mô tả	• Cho phép người dùng cập nhật số tiền mục tiêu (Target Amount) hoặc số tiền đã đạt được (Archived Amount) của một mục tiêu hiện có (tiết kiệm hoặc chi tiêu) để phản ánh các thay đổi trong kế hoạch tài chính.
Tác nhân	Người dùng (User)
Luồng chính	• Người dùng truy cập trang Mục tiêu (Goals) từ thanh điều hướng và chọn mục tiêu muốn điều chỉnh. • Người dùng nhấn nút “Điều chỉnh mục tiêu” (Adjust Goal) trên thẻ danh mục tương ứng. • Hệ thống hiển thị popup form điền sẵn thông tin hiện tại, bao gồm các trường có thể chỉnh sửa (ví dụ: Target Amount).

	• Người dùng thay đổi giá trị các trường cần thiết. • Người dùng nhấn nút “Lưu” (Save). • Front-end gửi dữ liệu mới đến API Backend. • Backend xác thực access_token và trích xuất user_id. • Backend kiểm tra quyền sở hữu và tính hợp lệ dữ liệu mới (Target Amount > 0). • Nếu hợp lệ, Backend cập nhật mục tiêu trong bảng Goals. • Backend trả về phản hồi thành công. • Front-end hiển thị thông báo “Điều chỉnh mục tiêu thành công”. • Popup đóng lại và giao diện mục tiêu được cập nhật với giá trị mới.
Luồng thay thế	A.1. Dữ liệu đầu vào không hợp lệ (Validation Error): • Nếu bất kỳ trường dữ liệu vi phạm quy tắc (ví dụ: nhập giá trị âm), Backend trả về lỗi Bad Request (HTTP 400). • Front-end hiển thị thông báo lỗi trong popup, Người dùng chỉnh sửa lại. A.2. Lỗi ủy quyền (Không có quyền chỉnh sửa): • Nếu mục tiêu không thuộc sở hữu của user_id hiện tại, Backend trả về lỗi Forbidden (HTTP 403). • Front-end hiển thị thông báo lỗi: "Bạn không có quyền chỉnh sửa mục tiêu này." A.3. Lỗi hệ thống Backend: • Nếu xảy ra lỗi trong quá trình cập nhật dữ liệu, hệ thống hiển thị thông báo chung: "Không thể lưu thay đổi lúc này. Vui lòng thử lại sau."
Tiền điều kiện	• Người dùng phải đăng nhập. • Mục tiêu cần điều chỉnh tồn tại và thuộc sở hữu của người dùng.
Hậu điều kiện	• Thành công: Thông tin Target Amount hoặc Archived Amount được cập nhật trong bảng Goals, giao diện mục tiêu được cập nhật.

- | | |
|--|---|
| | <ul style="list-style-type: none">• Thất bại: Hiển thị thông báo lỗi, Người dùng vẫn ở popup điều chỉnh mục tiêu. |
|--|---|

Tích hợp UI Figma:

Phù hợp với thiết kế giao diện trang Mục tiêu.

Endpoint:

PUT /api/v1/goals/{goalId}

Header: Authorization: Bearer <access_token>

Request Body mẫu:

```
{  
  "target_amount": 70000000  
}
```

Response thành công (HTTP 200):

```
{  
  "message": "Goal updated successfully",  
  "updated_goal": {  
    "goal_id": 1,  
    "target_amount": 70000000  
  }  
}
```

Xử lý lỗi:

```
{  
  "statusCode": 400,  
  "message": ["target_amount must be a positive number"],  
  "error": "Bad Request"  
}
```

```
{
  "statusCode": 403,
  "message": "You do not have permission."
}

{
  "statusCode": 500,
  "message": "Cannot update now. Try again."
}
```

1. Prompt A: Sinh Mã Backend (NestJS Service/Controller)

Mục tiêu: Xây dựng dịch vụ API, logic nghiệp vụ và xử lý lỗi server.

1. Endpoint PUT `/api/v1/goals/{goalId}` trong `GoalsController`, bảo vệ bằng `JwtAuthGuard`.
2. Nhận `goalId` từ `params`, `user_id` từ JWT, và DTO chứa `target_amount`.
3. Tìm mục tiêu trong DB theo `goalId`. Nếu không tìm thấy, throw `NotFoundException`.
4. Kiểm tra quyền sở hữu: `goal.userId` phải trùng với `user_id`. Nếu không, throw `ForbiddenException`.
5. Validation: `target_amount > 0` (dùng `class-validator` trong DTO).
6. Nếu hợp lệ, cập nhật `target_amount` trong DB.
7. Trả về response thành công.
8. Xử lý lỗi:
 - DTO validation thất bại: `BadRequestException` (HTTP 400)
 - Không thuộc quyền sở hữu: `ForbiddenException` (HTTP 403)
 - Lỗi DB: `InternalServerErrorException` (HTTP 500)

2. Prompt B: Sinh Mã Frontend (React Component & UI/MCP)

Mục tiêu: Xây dựng giao diện chính xác theo thiết kế Figma.

- Component: `AdjustGoalModal` (React TypeScript + Tailwind CSS)
- Modal chứa:
 - Tiêu đề: "Điều chỉnh mục tiêu"
 - Trường input số cho "Số tiền mục tiêu", điền sẵn giá trị hiện tại
 - Nút primary: "Lưu"
 - Nút secondary hoặc icon 'X' để đóng modal

3. Prompt C: Sinh Mã Frontend Logic (State & Kết nối API)

Mục tiêu: Kết nối UI với API và xử lý luồng thành công.

- State: `targetAmount`, `isLoading`, `error`
- Hàm `handleSave`:
 1. Gửi request `PUT /api/v1/goals/{goalId}` với payload: `{ target_amount: targetAmount }`
 2. Nếu thành công:
 - Hiển thị toast: "Điều chỉnh mục tiêu thành công."
 - Gọi `onClose()` để đóng modal
 - Gọi callback `onSuccess()` để refresh danh sách mục tiêu

4. Prompt D: Xử lý Ngoại lệ (Client-side Validation & Error Handling)

Mục tiêu: Hoàn thiện việc xử lý lỗi, trạng thái Loading, và Validation cuối cùng.

1. Loading:

- Biến state: `isLoading`

- Khi bắt đầu gọi API: `isLoading = true`, vô hiệu hóa nút "Lưu", hiển thị spinner
- Khi kết thúc API: `isLoading = false`

2. Lỗi API:

- Bọc trong `try...catch`
- Status 400: hiển thị lỗi "Số tiền mục tiêu phải lớn hơn 0." dưới input
- Status 403: hiển thị lỗi "Bạn không có quyền chỉnh sửa mục tiêu này." ở khu vực chung
- Các lỗi khác (500): hiển thị "Không thể lưu thay đổi lúc này. Vui lòng thử lại sau."

3. Validation FE: Trước khi gọi API:

- `targetAmount` phải lớn hơn 0
- Nếu không, hiển thị thông báo lỗi dưới input và không gọi API

A.0.11 Xem Biểu đồ Tổng kết Tiết kiệm

Đặc tả ca sử dụng

Bảng 1.11 mô tả ca sử dụng Xem Biểu đồ Tổng quan Tiết kiệm, hiển thị biểu đồ so sánh khoản tiết kiệm ròng theo từng tháng.

Bảng 1.11: Đặc tả ca sử dụng Xem Biểu đồ Tổng kết Tiết kiệm

Tên ca sử dụng	Xem biểu đồ Tổng quan tiết kiệm
Mô tả	• Cho phép người dùng trực quan hóa tổng số tiền tiết kiệm ròng (Tổng Thu nhập - Tổng Chi tiêu) theo từng tháng trong một năm đã chọn, đồng thời so sánh với kết quả tiết kiệm cùng kỳ năm trước.
Tác nhân	Người dùng (User)
Luồng chính	• Người dùng truy cập trang Mục tiêu từ thanh điều hướng.

	• Người dùng chọn năm muốn xem dữ liệu ở mục Tổng quan Tiết kiệm (nếu không chọn, mặc định là năm hiện tại). • Front-end gửi yêu cầu đến API Backend. • Backend xác thực access_token và trích xuất user_id. • Backend tính toán: - Tiết kiệm tháng hiện tại: đối với mỗi tháng trong năm được chọn, tính (Tổng Thu nhập - Tổng Chi tiêu) từ bảng Transactions. - Tiết kiệm so sánh: tính tương tự cho các tháng cùng kỳ năm trước. • Backend nhóm dữ liệu theo tháng và trả về kết quả biểu đồ (bao gồm tiết kiệm theo tháng của năm hiện tại và năm trước). • Front-end nhận dữ liệu và hiển thị Biểu đồ đường (Line Chart): - Trục X: các tháng (1 đến 12). - Trục Y: Số tiền tiết kiệm tròn. - Biểu đồ hiển thị ít nhất hai đường: năm được chọn và năm trước. • Giao diện hiển thị biểu đồ hoàn chỉnh cho người dùng.
Luồng thay thế	A.1. Không có dữ liệu giao dịch trong năm được chọn: • Nếu không có giao dịch Revenue hoặc Expense trong năm được chọn, Front-end hiển thị thông báo: "Chưa có dữ liệu giao dịch trong năm này để tính toán tiết kiệm."
Tiền điều kiện	• Người dùng phải đăng nhập. • Người dùng đã ghi nhận các giao dịch Thu nhập (Revenue) và Chi tiêu (Expense) trong các tháng của năm được chọn.
Hậu điều kiện	• Thành công: Biểu đồ đường thể hiện tổng tiết kiệm tròn theo tháng trên trang Tổng quan tiết kiệm. • Thất bại: Hiển thị thông báo lỗi hoặc thông báo không có dữ liệu để hiển thị.

Tích hợp UI Figma :

<https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits--Community-?node-id=66-5829&t=cxgXQfkGoLMfxW48-4>

Dropdown chọn năm (VD: “2024”, “2025”)

Biểu đồ đường: Trục X: Tháng 1 → Tháng 12. Trục Y: Tổng tiền tiết kiệm. Đường đậm (màu xanh): “This Year”

Đường mờ (xám): “Same period last year”

Endpoint:

GET /api/v1/savings/summary

Request mẫu: GET /api/v1/savings/summary?year=2025

Query Params: year (tùy chọn): Năm muốn xem tổng quan tiết kiệm. Nếu không có, mặc định là năm hiện tại.

Response mẫu:

```
{
  "user_id": 1,
  "year": 2025,
  "summary": {
    "this_year": [
      {"month": "01", "amount": 1500000},
      {"month": "02", "amount": 2000000},
      {"month": "03", "amount": 2500000},
      {"month": "04", "amount": 2200000},
      {"month": "05", "amount": 2800000},
      {"month": "06", "amount": 3100000},
      {"month": "07", "amount": 3300000},
      {"month": "08", "amount": 2700000},
      {"month": "09", "amount": 2400000},
```

```

    {"month": "10", "amount": 2600000},
    {"month": "11", "amount": 3000000},
    {"month": "12", "amount": 3500000}
  ],
  "last_year": [
    {"month": "01", "amount": 1200000},
    {"month": "02", "amount": 1800000},
    {"month": "03", "amount": 2000000},
    {"month": "04", "amount": 1900000},
    {"month": "05", "amount": 2300000}
  ]
}
}

```

Xử lý lỗi:

```

{
  "statusCode": 500,
  "message": "An internal server error occurred while processing the
  savings summary."
}

```

1. Prompt A: Sinh Mã Backend (NestJS Service/Controller)

Mục tiêu: Xây dựng dịch vụ API, logic nghiệp vụ và xử lý lỗi server.

1. Tạo endpoint GET /api/v1/savings/summary trong SavingsController, được bảo vệ bằng JwtAuthGuard.
2. Lấy user_id từ JWT qua @Request() req.
3. Nhận tham số year từ query. Nếu không có, mặc định là năm hiện tại.
4. Truy vấn bảng Transactions để lấy toàn bộ giao dịch Thu (Revenue) và Chi (Expense) trong các tháng thuộc năm year.

5. Tính tiết kiệm ròng mỗi tháng: $\text{saving} = \text{total_revenue} - \text{total_expense}$.
Nếu không có giao dịch, giá trị là 0.
6. Lặp lại các bước trên cho năm trước đó ($\text{year} - 1$) và tạo mảng `last_year`.
7. Trả về response đúng định dạng yêu cầu.
8. Nếu có lỗi phát sinh, throw `InternalServerErrorException`.

2. Prompt B: Sinh Mã Frontend (React Component & UI/MCP)

Mục tiêu: Xây dựng giao diện hiển thị biểu đồ Tổng quan Tiết kiệm theo đúng thiết kế Figma.

- Component: `SavingsSummaryChart` (React + TypeScript + Tailwind CSS)
- Bố cục của Component:
 - Tiêu đề: “Saving Summary”
 - Dropdown chọn năm (ví dụ 2023, 2024, 2025)
 - Biểu đồ đường (LineChart) sử dụng thư viện `recharts`
- Các yêu cầu giao diện:
 - Trục X hiển thị 12 tháng (Jan–Dec)
 - Đường 1: Năm hiện tại, màu xanh đậm, label “This Year”
 - Đường 2: Năm trước, màu xám nhạt, label “Same period last year”
- Thiết kế bắt buộc tuân theo Figma MCP: <https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits--Community-?node-id=66-5829>

3. Prompt C: Sinh Mã Frontend Logic (State & Kết nối API)

Mục tiêu: Kết nối component với API và quản lý trạng thái dữ liệu.

- State:

- `selectedYear`: năm được chọn (mặc định: năm hiện tại)
 - `chartData`: chứa `this_year` và `last_year`
 - `isLoading`: trạng thái loading
 - `error`: thông báo lỗi
- Hàm `fetchSavingsSummary(year)`:
 1. Gửi request GET `/api/v1/savings/summary?year=...`
 2. Nếu thành công, cập nhật state `chartData` bằng dữ liệu từ API
 3. Dùng `useEffect` để gọi API mỗi khi `selectedYear` thay đổi

4. Prompt D: Xử lý Ngoại lệ (Client-side Validation & Error Handling)

Mục tiêu: Cải thiện UX khi gọi API và khi thiếu dữ liệu.

1. Loading:

- Trước khi gọi API: `isLoading = true`
- Sau khi hoàn tất: `isLoading = false`
- Khi loading: hiển thị Spinner thay biểu đồ

2. Lỗi API:

- Bắt lỗi với `try...catch`
- Nếu API lỗi, hiển thị thông báo: “Không thể tải dữ liệu. Vui lòng thử lại sau.”

3. Không có dữ liệu:

- Nếu cả `this_year` và `last_year` rỗng → hiển thị: “*Chưa có dữ liệu giao dịch trong năm này để tính toán tiết kiệm.*”